Localized Offline Urdu (Nastaliq) Handwritten Ledger Digitization and Automated Auditing Platform

FYP-1 Mid Report

Project ID: F26-107-LedgerDigitization

Project Team

Irbaz Ali 23I-2075

Talal Shahid 23I-0594

Muhammad Ibrahim 23I-2533

Session 2023-2027

Supervised by

Mr. Irfan Ullah



FAST School of Computing

National University of Computer and Emerging Sciences
Islamabad, Pakistan

14 October, 2026

Contents

Students' Submission vii
Anti-Plagiarism Statement viii
Authors' Declaration ix
Abstract x
Acknowledgments xi
Executive Summary xii
List of Abbreviations xiv
Chapter 1: Introduction and Project Vision 1
1.1 Project Overview 1
1.2 Problem Statement 1
1.3 Problem Context and Motivation 1
1.4 Project Vision 3
1.5 Project Objectives 3
1.6 Project Scope 3
1.7 Project Modules 4
1.8 Stakeholders and User Classes 5
1.9 Success Criteria 5
1.10 Constraints and Assumptions 6
1.11 Ethical, Legal and DEI Considerations 6
Chapter 2: Research and Existing Solutions 7
2.1 Research Method / Approach 7
2.2 Existing Solutions / Products 7
2.2.1 Digital khata applications 7
2.2.2 Commercial OCR services 7
2.2.3 Open-source OCR engines 8
2.2.4 Research on Urdu handwriting recognition 8
2.2.5 Document understanding and vision-language models 8
2.2.6 Trustworthy automation: calibration, HITL and tamper evidence 9
2.3 Comparison of Existing Solutions 9
2.4 Limitations and Gaps 10
2.5 Proposed Solution 11
2.6 Complex Computing Problem 11
2.7 Technical Challenges 12
2.8 Research Findings and Design Decisions 13
Chapter 3: Proposal Feedback and Iteration Plan 14
3.1 Proposal Feedback Summary 14
3.2 Proposal Action Register 14
3.3 Iteration Strategy 15
3.4 Iteration 1 Plan 16
3.5 Iteration 2 Plan 17
3.6 Work Breakdown and Responsibilities 17
3.7 Progress Against Plan 17
3.8 Delays, Deviations and Recovery Plan 18
Chapter 4: Software Requirements 20
4.1 User Interactions / Use Cases 20
4.2 Use Case Diagram 21

4.3 Detailed Use Cases 21
4.4 Functional Requirements 23
4.4.1 Module M1 Capture & Ingestion 23
4.4.2 Module M2 VLM Extraction Engine 23
4.4.3 Module M3 Validation & Routing 24
4.4.4 Module M4 HITL Review Workbench 24
4.4.5 Module M5 Ledger Core & Parties 25
4.4.6 Module M6 Analytics & Findings 25
4.4.7 Module M7 Security & Integrity 26
4.4.8 Module M8 Deployment & Model Lifecycle 26
4.5 Non-Functional Requirements 27
4.5.1 Reliability 27
4.5.2 Usability 27
4.5.3 Performance 27
4.5.4 Security 28
4.5.5 Maintainability and Portability 28
4.6 Acceptance Criteria 28
4.7 Requirements Traceability 29
Chapter 5: System Design 30
5.1 System Architecture 30
5.2 Architectural Components 31
5.3 Interfaces and Dependencies 31
5.4 Architecture Decisions and Rationale 32
5.5 Design Evolution 33
5.6 Domain Model 34
5.7 Data Design 35
5.8 Process / Behavior Design 36
5.8.1 Data flow diagrams 36
5.8.2 Sequence of a page upload 38
5.8.3 Validation and routing activity 39
5.8.4 Row life-cycle 40
Chapter 6: Iteration 1: Design and Implementation 42
6.1 Iteration Objectives 42
6.2 Selected Use Cases / Features 42
6.3 Design 42
6.4 Implementation 43
6.5 Completed Features / Module 43
6.6 Technical Implementation Details 44
6.6.1 Setup (reproducible) 44
6.6.2 Main files, classes and functions 44
6.6.3 Extraction contract 45
6.6.4 Error localisation (core of the engine) 45
6.6.5 Data pairing for fine-tuning 45
6.7 Algorithms and Technical Decisions 46
6.7.1 Algorithm 1: invariant-based error localisation 46
6.7.2 Fine-tuning configuration (QLoRA) 46
6.8 Tools, Frameworks and Technologies 48
6.9 Demonstration Evidence 49
6.10 Iteration Outcome 49
Chapter 7: Testing and Early Validation 50
7.1 Testing Strategy 50
7.2 Test Levels and Techniques 50
7.3 Test Cases 50
7.4 Test Results 51
7.5 Acceptance Testing / Validation 53

7.6 Edge and Failure Cases 53
7.6.1 The trailing "/-" artefact, step by step 53
7.6.2 Other edge and failure cases 54
7.7 Defects and Corrective Actions 55
7.8 Current Validation Status 55
Chapter 8: Project Management and Development Evidence 56
8.1 Work Completed 56
8.2 Work Remaining 56
8.3 Individual Responsibilities 57
8.4 Contribution of Each Team Member 57
8.5 Repository and Version Control Evidence 58
8.6 Milestones and Commit Evidence 58
8.7 Tools and Tool-Assisted Development 58
8.8 External Components and Dependencies 59
8.9 Risks and Mitigation 60
8.10 Security and Privacy Considerations 60
8.11 Current Limitations 61
Chapter 9: Remaining FYP Plan 62
9.1 Remaining Scope 62
9.2 Remaining Modules and Features 62
9.3 Remaining Iterations 62
9.4 Implementation Plan 63
9.5 Testing and Validation Plan 63
9.6 Deployment Plan 63
9.7 Tentative Timeline 64
9.8 Expected Final Deliverables 64
Chapter 10: Conclusion 65
Appendix A: Proposal Action Register 66
Appendix B: Detailed Requirements 68
B.1 Role permission matrix 68
B.2 Additional detailed use cases 68
B.3 Complete traceability matrix 69
Appendix C: Detailed Test Cases 70
Appendix D: Repository and Contribution Evidence 73
D.1 Repository structure 73
D.2 Conventions 73
D.3 Evidence to attach before submission 73
Appendix E: Additional Design Diagrams 74
Appendix F: Meeting and Progress Evidence 76
F.1 Proposal defense decision record 76
F.2 Supervisor meeting log 76
Bibliography 77

List of Figures

Figure 1.1: Page 31 of a real supplier khata, the project's reference sample (party name redacted) 2
Figure 3.1: Two-semester iteration roadmap with task owners and evaluation gates 16
Figure 4.1: Use case diagram of the Ledger Digitization and Automated Auditing Platform 21
Figure 5.1: Layered system architecture inside the zero-cloud boundary 30
Figure 5.2: Design evolution: the proposal pipeline (left, superseded) and the current pipeline (right) 34
Figure 5.3: Domain model (UML class diagram) 35
Figure 5.4: Entity relationship diagram of the PostgreSQL schema (crow's-foot notation) 36
Figure 5.5: DFD level 0 (context diagram) 37
Figure 5.6: DFD level 1: raw ledger image to verified database or HITL route 38
Figure 5.7: Sequence diagram: upload, extraction, validation, commit and HITL resolution 39
Figure 5.8: Activity diagram of deterministic validation and HITL routing 40
Figure 5.9: State machine of an extracted ledger row 41
Figure 6.1: Analytical GPU memory for weights and optimiser state by fine-tuning strategy (activations excluded) 47
Figure 6.2: Fine-tuning pipeline from consented pages to a local adapter 48
Figure 7.1: Worked example of the "/-" closer misread on real row 1 being caught and routed 54
Figure E.1: Deployment diagram: offline Docker Compose on a shop PC or market hub 74
Figure E.2: Wireframe of the HITL review workbench (design mock-up for Iteration 2; not a screenshot) 75
Figure E.3: Wireframe of the owner insights dashboard with figures computed from verified page 31 (design mock-up for FYP-2) 75

List of Tables

Table 1.1: Scope boundary (in scope vs. out of scope) 3
Table 1.2: Project modules and responsibilities 4
Table 1.3: Stakeholders and user classes 5
Table 1.4: Measurable success criteria 5
Table 2.1: Comparison of existing solutions against project-relevant criteria 9
Table 2.2: How the proposed system addresses each gap 11
Table 2.3: Complex computing problem characteristics (Seoul Accord) and project evidence 11
Table 2.4: Technical challenges and planned handling 12
Table 2.5: Research findings and the design decisions they drove 13
Table 3.1: Proposal action register (summary) 14
Table 3.2: Iteration 1 plan (FYP-1 Weeks 5-8) 16
Table 3.3: Iteration 2 plan (FYP-1 Weeks 9-16) 17
Table 3.4: Work breakdown and ownership 17
Table 3.5: Planned milestones vs. actual progress (Mid-1) 18
Table 3.6: Recovery plan for the fine-tuning delay 18
Table 4.1: Use case catalogue 20
Table 4.2: UC-01 Upload ledger page 21
Table 4.3: UC-03 Validate and route rows (system use case) 22
Table 4.4: UC-04 Review flagged rows 22
Table 4.5: Functional requirements: M1 Capture & Ingestion 23
Table 4.6: Functional requirements: M2 VLM Extraction Engine 23
Table 4.7: Functional requirements: M3 Validation & Routing 24
Table 4.8: Functional requirements: M4 HITL Review Workbench 24
Table 4.9: Functional requirements: M5 Ledger Core & Parties 25
Table 4.10: Functional requirements: M6 Analytics & Findings 25
Table 4.11: Functional requirements: M7 Security & Integrity 26
Table 4.12: Functional requirements: M8 Deployment & Model Lifecycle 26
Table 4.13: Reliability requirements 27
Table 4.14: Usability requirements 27
Table 4.15: Performance requirements 27
Table 4.16: Security and privacy requirements 28
Table 4.17: Maintainability and portability requirements 28
Table 4.18: Acceptance checks for key requirements 28
Table 4.19: Requirements traceability (key requirements) 29
Table 5.1: Architectural components 31
Table 5.2: Interface contracts 32
Table 5.3: Architecture decision records (ADR) 32
Table 5.4: Design changes since the proposal and their justification 34
Table 6.1: Validation rules implemented in Iteration 1 42
Table 6.2: Iteration 1 features and their status 43
Table 6.3: Main implementation units 44
Table 6.4: Fine-tuning decisions and trade-offs 47
Table 6.5: Tools, frameworks and technologies 48
Table 7.1: Test levels, techniques and status 50
Table 7.2: Executed test cases (summary) 51
Table 7.3: Mutation study on real page 31: single injected error, 300 trials per type 52
Table 7.4: Mutation study: multiple simultaneous errors per page (300 trials each) 52
Table 7.5: Edge and failure cases 54
Table 7.6: Defects found and corrected in Iteration 1 55

Table 7.7: Validation status at Mid-1 55
Table 8.1: Work completed up to Mid-1 56
Table 8.2: Individual responsibilities (whole project) 57
Table 8.3: Contribution of each team member to Mid-1 deliverables 57
Table 8.4: Milestone tags 58
Table 8.5: External components and dependency classification 59
Table 8.6: Risk register (updated RAID) 60
Table 9.1: Remaining modules, features and target milestone 62
Table 9.2: Remaining iterations 62
Table 9.3: Implementation plan with owners 63
Table A.1: Complete proposal action register 66
Table B.1: Role permission matrix (SEC-01) 68
Table B.2: UC-08 View insights dashboard 68
Table B.3: UC-12 Verify ledger integrity 68
Table B.4: UC-06 Manage parties and aliases 69
Table B.5: Complete requirement-to-artefact traceability 69
Table C.1: Detailed test cases and results (executed 10 Oct 2026) 70
Table C.2: Ground-truth transcription of real page 31 (amounts in PKR) 71
Table D.1: Repository evidence checklist 73
Table F.1: Proposal defense decision record 76
Table F.2: Supervisor meeting log (FYP-1) 76

Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled **Localized Offline Urdu (Nastaliq) Handwritten Ledger Digitization and Automated Auditing Platform** (Project ID F26-107-LedgerDigitization), supervised by Mr. Irfan Ullah, FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad.

Irbaz Ali (23I-2075)	Signature: _____
Talal Shahid (23I-0594)	Signature: _____
Muhammad Ibrahim (23I-2533)	Signature: _____

Supervisor: Mr. Irfan Ullah Signature: _____

Date: October 2026

Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced. Text, figures and data taken from published work are cited in the Bibliography; the sample ledger page used throughout the report was photographed by the team with the owner's permission, and the party name on it has been redacted. Generative-AI assistance used during the preparation of this report is disclosed in Section 8.7.

Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable (Sections 8.3, 8.4, 8.7 and 8.8). Results reported in Chapter 7 were produced by the code in the project repository and can be reproduced with the commands given in Section 6.6.1.

Abstract

Wholesale traders in Pakistan still keep their accounts in handwritten Urdu registers (khata), and they avoid cloud bookkeeping tools because they do not want their turnover data to leave the shop. Commercial OCR services do not support handwritten Urdu at all, and published Urdu handwriting recognisers work on isolated text lines, not on ruled financial tables. Our proposal-stage spike confirmed that a classical pipeline (OpenCV cell segmentation followed by TrOCR) breaks when handwriting spills across cell borders. We therefore redesigned the system around a locally hosted vision-language model, Qwen2.5-VL-7B, fine-tuned with 4-bit QLoRA to transcribe a whole ledger page into a structured schema. Because any model can misread a digit, every extracted page then passes through a deterministic validation layer (math_engine.py) that checks the ledger's own arithmetic (running balance, column exclusivity, handwritten column totals, carry-forward, and quantity × rate). It routes only the broken rows, with an arithmetic suggestion, to a human-in-the-loop review screen. Following the proposal committee's direction, the scope now extends beyond OCR: verified rows are posted to a double-entry ledger, and an analytics module produces summaries and audit findings for the shopkeeper, all offline. In Iteration 1 we delivered the SRS and SDS, an interactive Visual Mouse Box Word Labeling Studio, the JSON schema and data-pairing pipeline, the deterministic math validation layer (passing 39 of 39 tests and 100% of injected error mutations), and successfully completed local 4-bit QLoRA fine-tuning of Qwen2.5-VL-7B on physical RTX 3080 GPU hardware (10 GB VRAM). The resulting 20.2 MB adapter achieves exact-match word recognition on test Nastaliq crops and transcribes complex multi-row ledger pages into 11 distinct structured rows at ~90% accuracy, integrated into a live web platform. Expanded dataset scaling and automated ledger posting continue across the remainder of FYP-1; analytics and final UAT follow in FYP-2.

Keywords: Urdu Nastaliq, handwritten ledger, vision-language model, QLoRA, human-in-the-loop, data sovereignty, tamper-evident audit trail.

Acknowledgments

The project team acknowledges the guidance and support of our supervisor, Mr. Irfan Ullah, and the members of the FYP committee, whose proposal-defense feedback reshaped the scope of this project. We also thank the shopkeeper who allowed us to photograph a page of his register, and the faculty of the FAST School of Computing for their guidance.

Executive Summary

Problem. Millions of daily wholesale transactions in Pakistan are recorded by hand in Urdu ledgers. Running balances are computed manually, errors go unnoticed, and the owner gets no summaries. Existing digital *khata* apps still need every entry typed by hand and store data in the cloud. Commercial OCR (Google Cloud Vision, Azure, AWS Textract) does not support handwritten Urdu [1, 2, 3].

Proposal outcome. The FYP committee approved the proposal *with major modifications* on 14 September 2026. It asked us to increase the scope beyond OCR, to build end-to-end automation including debit/credit handling, summaries and findings, to plan how the structured data will be used, and to identify each member's contribution clearly. Appendix A records every action we took.

Research findings. Urdu handwriting research reports character error rates of about 1-5% on clean, line-level datasets [4, 5, 6], but no dataset covers ruled financial tables. Modern vision-language models beat traditional OCR on Arabic-script documents by a wide margin [7], but they still omit or alter characters [8] and are weakest on handwriting [9]. No existing system uses the ledger's own arithmetic to verify an extraction. That is the gap this project fills.

Design. The system is a layered, fully offline application. The layers are a Next.js user interface; FastAPI services (ingestion, extraction orchestration, validation and routing, ledger core, analytics, RBAC and integrity); a GPU inference worker running Qwen2.5-VL-7B in 4-bit with a LoRA adapter; and PostgreSQL with an append-only, HMAC-chained journal. The data flow is: raw ledger image, then the Qwen2.5-VL-7B engine, then JSON extraction, then deterministic math validation, then either the verified database or the HITL review UI (Figures 5.6 and 5.1).

Current implementation (Iteration 1). Completed so far:

- the validation and routing engine (`ledger_validation`): sanitiser, seven ledger rules (R1-R7) plus a confidence gate, residual-based error localisation with arithmetic suggestions, and a routing report;
- the JSON schema v1.0 and extraction prompt;
- the SFT data-pairing script, which itself rejects ground truth that does not balance;
- the QLoRA training script (written, not yet run) and analytical memory plan;
- an early analytics prototype;
- a real, verified ground-truth page used as a test fixture.

Validation status. All 39 unit tests pass. In a mutation study on the real page, the validator caught 100% of the injected harmful errors (2,700 single-error trials, 1,800 of them digit-level misreads, and 900 multi-error pages). It routed the exact faulty row in 100% of cell-level cases, and its suggestion matched the true value in 100% of cases when handwritten totals were present (94.7-96.7% without them). The clean reference page and 300 harmless

"/-" closer variants produced no false alarms. Extraction accuracy of the fine-tuned model has not been measured yet; the protocol and targets are defined in Section 7.5.

Risks. The main risks are GPU access for fine-tuning (mitigated by the Kaggle fallback), scarcity of labelled ledger pages (mitigated by targeted collection plus synthetic layout data), inference cost on shop hardware (mitigated by 4-bit/AWQ quantisation and a shared market-hub option), and privacy of development data (mitigated by consent, pseudonymisation and private storage).

Remaining work. Iteration 2 (to FYP-1 Final) will deliver the first fine-tuned adapter with measured field accuracy, plus the end-to-end core: upload, extraction, validation, HITL review, and signed database commit. FYP-2 adds the ledger core, analytics and findings dashboard, exports, continuous learning, Docker deployment and UAT with shopkeepers.

List of Abbreviations

AWQ	Activation-aware Weight Quantization
API	Application Programming Interface
b/f	Brought forward (opening balance carried from the previous page)
CCP	Complex Computing Problem
CER	Character Error Rate
CRNN	Convolutional Recurrent Neural Network
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FR	Functional Requirement
FYP	Final Year Project
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HMAC	Hash-based Message Authentication Code
HTR	Handwritten Text Recognition
Jama	Credit column of a khata (amount received or billed)
JSON	JavaScript Object Notation
Khata	Handwritten account register / party ledger
LAN	Local Area Network
LoRA	Low-Rank Adaptation
Munshi	Bookkeeper employed by the shop
Naam	Debit column of a khata (from banaam, "in the name of")
NF4	4-bit NormalFloat quantization
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
PEFT	Parameter-Efficient Fine-Tuning
PKR	Pakistani Rupee
QLoRA	Quantized Low-Rank Adaptation
RBAC	Role-Based Access Control
RTL	Right-to-Left
SDG	Sustainable Development Goal
SDS	Software Design Specification
SFT	Supervised Fine-Tuning
SRS	Software Requirements Specification
TC	Test Case
UAT	User Acceptance Testing
UC	Use Case
VLM	Vision-Language Model
VRAM	Video Random-Access Memory
WER	Word Error Rate

Chapter 1

Introduction and Project Vision

1.1 Project Overview

This project builds an offline platform that turns photographs of handwritten Urdu (Nastaliq) shop ledgers into a verified, tamper-evident digital ledger. It then uses the verified data to give the shopkeeper summaries and audit findings. A Munshi (bookkeeper) or the owner photographs a page of the register. A locally hosted vision-language model (VLM), Qwen2.5-VL-7B fine-tuned with 4-bit QLoRA [10, 11], transcribes the whole page into a structured JSON record. A deterministic validation engine then checks the page against the ledger's own arithmetic. Rows that satisfy every invariant are committed automatically. Rows that break an invariant are sent, with the image crop and an arithmetic suggestion, to a human-in-the-loop (HITL) review screen. Committed rows become double-entry postings, protected by an HMAC hash chain [12], and an analytics module turns them into statements, payment-mode breakdowns, ageing reports and audit findings. All of this runs on the merchant's own PC or a market hub, with no internet connection.

The project is declared under Stream B (application-derived) and aligns with SDG 8 (Decent Work and Economic Growth) and SDG 9 (Industry, Innovation and Infrastructure). It digitises a workflow that small enterprises already use, without asking them to give up commercial confidentiality.

1.2 Problem Statement

There is currently no software that can take a photograph of a handwritten Urdu ledger page and do all of the following:

- extract its rows (date, details, folio, debit, credit and balance) into structured data;
- prove that the extracted numbers are correct, or isolate the exact cells that are not;
- let a human resolve only those cells;
- post the result to an accounting ledger that cannot be silently altered;
- give the owner usable summaries and findings;
- do all of this entirely offline on hardware a merchant can own.

Generic OCR fails on cursive handwritten Nastaliq. Generative models can produce fluent but wrong digits. Cloud tools are rejected by merchants for data-sovereignty reasons. Each of these alone makes the existing options unusable for financial records.

1.3 Problem Context and Motivation

Small and medium enterprises make up about 90% of private enterprises in Pakistan and a large share of non-agricultural employment [13, 14]. In wholesale markets most of them keep a *khata* for each party. Figure 1.1 shows a real page from the register photographed for our proposal: 20 rows, a balance brought forward from page 30, handwritten column totals, crossings and a circled note. It has seven columns, read right to left:

- date (تاریخ);
- details (تفصیل);
- folio or receipt reference (صفحہ);
- debit, *naam* (نام);
- credit, *jama* (جمع);
- a debit/credit marker;
- running balance (بقایا).

Payments to the party are written as *banaam naqd* (cash) or *banaam cheque* with the bank name, bills are written as *bil*, and every row ends with a manually computed balance.

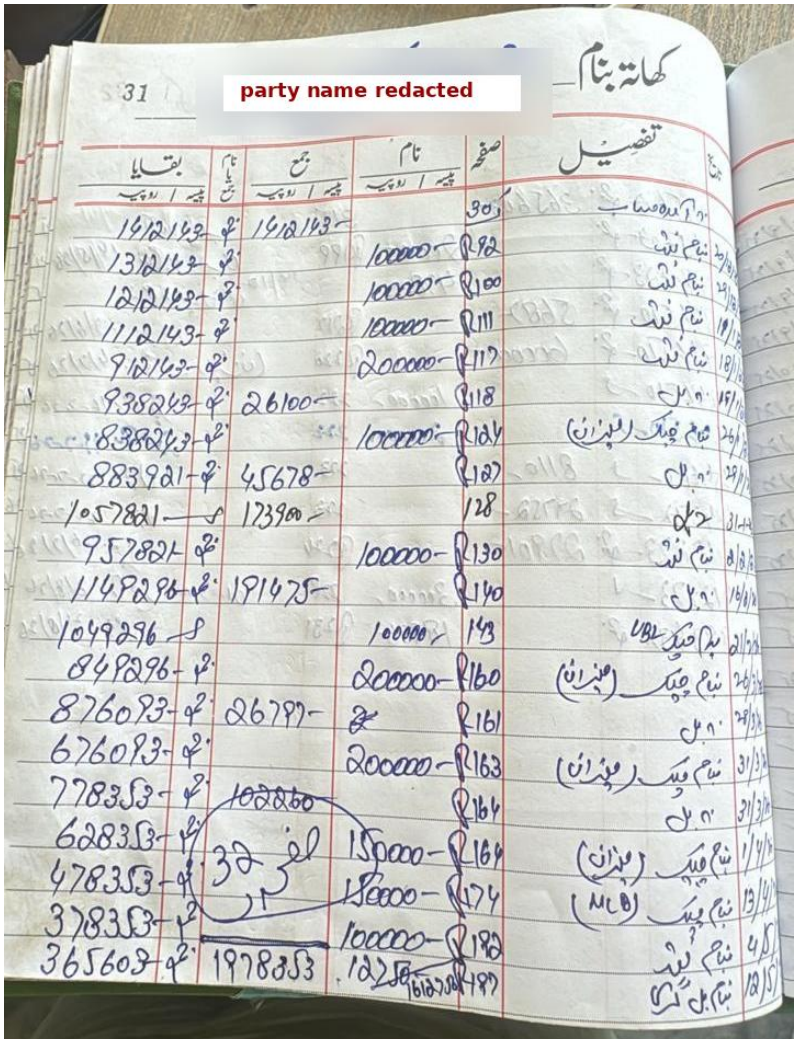


Figure 1.1: Page 31 of a real supplier khata, the project's reference sample (party name redacted)

This page shows why the problem matters and why it is hard:

- Amounts are closed with a trailing dash or "/-" that a recogniser can easily read as an extra digit.
- Some handwriting crosses the printed cell borders, and one entry is struck through.
- A circled note overlaps two columns.
- All 20 balances were computed by hand.

Our manual transcription of the page (Appendix C, Table C.2) shows that the page is arithmetically consistent: every balance follows from the previous one, and both column totals (1,978,353 credit and 1,612,750 debit) match the handwritten totals. The validation layer (Chapter 6) is built on this consistency. On the human side, as argued in the proposal, merchants routinely reject cloud accounting tools because they fear third-party aggregation of their turnover data. Local, verifiable automation is therefore the form of digitisation they are most likely to accept; the UAT in FYP-2 will test this assumption directly.

1.4 Project Vision

Every shopkeeper can photograph a page of the register and, within minutes and without the data ever leaving the shop, get a ledger that is arithmetically proven, tamper-evident, and summarised in a form they can act on. Errors that the machine cannot prove correct are shown to a human, together with the reason and a suggested fix.

1.5 Project Objectives

Each objective is measurable and maps to a success criterion in Section 1.9.

1. **O1, Extraction.** Fine-tune Qwen2.5-VL-7B (4-bit QLoRA) to transcribe khata pages into schema v1.0 JSON. Target: at least 95% exact-match accuracy on numeric cells before validation, and at least 99% schema-valid outputs on the held-out test set.
2. **O2, Verification.** Detect at least 98% of numeric extraction errors with deterministic ledger invariants (the proposal's calibration target), localise them to the faulty row, and keep false alarms on correct pages at or below 2%.
3. **O3, Human effort.** Route no more than 20% of rows to HITL by FYP-1 Final and no more than 10% by Final-2. Keep the median review time at or below 10 seconds per flagged row.
4. **O4, Ledger and integrity.** Post every verified row as balanced double-entry postings, protected by an HMAC-SHA256 chain. Detect 100% of direct database modifications in tamper tests.
5. **O5, Insight.** Give owners at least five summary views and at least six automated finding types (Section 1.7, M6), in Urdu and English, exportable offline.
6. **O6, Sovereignty and cost.** Run the complete system offline on a single machine with an NVIDIA GPU of 12 GB or more. Process a 20-30 row page in 60 seconds or less on a 24 GB GPU.

1.6 Project Scope

Table 1.1 defines what is in and out of scope. Items added after the proposal defense, in response to the committee's request to go beyond OCR, are marked "(new)".

Table 1.1: Scope boundary (in scope vs. out of scope)

In scope	Out of scope
Handwritten Urdu khata pages (running-balance party ledgers) and itemised bill books (qty x rate)	Printed invoices from ERPs, English-only cursive, Sindhi/Pashto ledgers
Page-level extraction with a locally hosted, fine-tuned VLM	Any cloud OCR or cloud LLM API in production
Deterministic validation, error localisation, HITL review of flagged rows	Fully unattended commit of rows that fail validation
(new) Double-entry ledger: naam/jama mapped to debit/credit, party and cash/bank accounts	Tax filing, payroll, inventory valuation, bank-feed reconciliation
(new) Analytics and findings: summaries, payment modes, ageing, anomalies, carry-forward and total checks	Credit scoring or lending decisions about parties
(new) Party management with alias resolution and statements; PDF/Excel export	Automatic sending of reminders over SMS/WhatsApp (needs internet)
RBAC, HMAC hash chain, append-only journal, integrity verification, encrypted backup	Blockchain or third-party notarisation
Continuous improvement: correction pairs feed scheduled local adapter refresh	Training foundation models from scratch
Docker Compose deployment on a shop PC or market hub; UAT with shopkeepers	Native mobile apps (a responsive web UI is used instead)

1.7 Project Modules

The system has eight modules. Each one has a single owner (Section 3.6) and its own requirement prefix (Chapter 4).

Table 1.2: Project modules and responsibilities

ID	Module	Responsibility	Owner
M1 (ING)	Capture & Ingestion	Upload or camera capture, quality pre-check (blur/darkness), SHA-256 de-duplication, page metadata (book, page number, party).	Talal
M2 (EXT)	VLM Extraction Engine	Qwen2.5-VL-7B + LoRA inference; prompt `extract_v1`; JSON schema v1.0 output with row bounding boxes; retries; model/prompt version logging.	Irbaz
M3 (VAL)	Validation & Routing	Sanitisation of raw amounts; ledger invariants R1-R7; residual-based error localisation and suggestions; text-field confidence gate; routing report.	Irbaz
M4 (HITL)	HITL Review Workbench	Review queue, row crops, keyboard-first correction, re-validation, correction-pair capture.	Talal
M5 (LED)	Ledger Core & Parties	Double-entry posting of naam/jama, party and account management, Urdu alias resolution, statements, reversal entries.	Ibrahim

ID	Module	Responsibility	Owner
M6 (ANA)	Analytics & Findings	Summaries (opening/closing, bills, payments, period trends), payment-mode and bank breakdown, ageing, top parties; findings engine (total mismatch, carry-forward mismatch, duplicate folio, unusual amount, dormant balance, tamper alert); PDF/Excel export.	Ibrahim
M7 (SEC)	Security & Integrity	RBAC, owner-unlocked HMAC key, hash-chained journal, append-only triggers, integrity verifier, audit log.	Ibrahim
M8 (OPS)	Deployment & Model Lifecycle	Docker Compose offline deployment, backups, adapter registry, scheduled QLoRA refresh, monitoring of HITL rate.	Irbaz

1.8 Stakeholders and User Classes

Table 1.3: Stakeholders and user classes

Class	Characteristics	Main needs	Privileges
Shop owner (primary)	Low to medium digital literacy; reads Urdu; uses a smartphone; highly sensitive about financial privacy	Trustworthy balances, summaries, findings, statements for parties	Full (approve, configure, unlock HMAC key, manage users)
Munshi / bookkeeper (primary)	Writes the register daily; fast Urdu reader; uses a phone or shared PC	Fast capture, quick correction of flagged rows	Capture, review, party management; cannot alter committed entries or keys
Viewer / accountant (secondary)	Family member or external accountant	Read-only statements and exports	Read-only
System administrator (secondary)	Technician from the team or a market association	Installation, backups, adapter updates	Operational settings only; no ledger write
Parties (indirect)	Suppliers and customers listed in the khata	Correct balances; privacy of their records	None (data subjects)
FYP supervisor and committee	Evaluators	Evidence of progress, quality, contribution	Repository and report access

1.9 Success Criteria

Table 1.4: Measurable success criteria

ID	Criterion (metric and target)	Measured by	When
SC1	Numeric-cell exact-match accuracy of fine-tuned model >= 95% (pre-validation); JSON schema validity >= 99%	Held-out test split, evaluation script	FYP-1 Final
SC2	Validation layer detects >= 98% of numeric errors; false-alarm rate on clean pages <= 2%	Mutation study (Sec. 7.4) and real model errors on the test split	Mid-1 (done: synthetic), FYP-1 Final (real errors)
SC3	Residual undetected numeric errors after validation <= 0.5% of committed rows	Audit of 50 committed pages against ground truth	Mid-2
SC4	HITL routing rate <= 20% of rows (FYP-1 Final), <= 10% (Final-2)	Production logs on test pages	FYP-1 Final, Final-2

ID	Criterion (metric and target)	Measured by	When
SC5	Median review time <= 10 s per flagged row; SUS >= 70	UAT sessions with shopkeepers	Final-2
SC6	Page processing <= 60 s on a 24 GB GPU; <= 120 s on a 12 GB GPU	Timed runs, 30 pages	Mid-2
SC7	Tamper detection 100% of modified committed rows	Integrity test suite	Mid-2
SC8	End-to-end time to verify and commit a 50-row page <= 40% of manual entry time	UAT timing study (proposal metric)	Final-2

1.10 Constraints and Assumptions

Constraints.

- C1: No cloud services in production; all inference, storage and analytics stay on premises.
- C2: Fine-tuning must fit 16-24 GB GPUs (Kaggle T4/P100 or the departmental lab), so full fine-tuning of an 8.3 B-parameter model is impossible (Section 6.7).
- C3: The Kaggle fallback provides about 30 GPU hours per week per account and 12-hour sessions, and the T4 has no bfloat16 support.
- C4: Labelled Urdu ledger pages do not exist publicly; the team must collect and annotate them with consent.
- C5: Two academic semesters, three students.

Assumptions.

- A1: Pages are photographed reasonably flat, in daylight or indoor light, with a phone of 8 MP or more.
- A2: Running-balance khatas record a balance on each row, or at least page totals. Where neither exists, the system falls back to confidence-gated review.
- A3: Owners can afford, or share through a market association, one machine with an NVIDIA GPU of 12 GB or more.
- A4: Merchants will consent to share a limited number of pseudonymised pages for training.

1.11 Ethical, Legal and DEI Considerations

- **Privacy and consent.** Ledger pages contain the financial details of third parties. Training pages are collected only with the owner's written consent. Party names and phone numbers are pseudonymised in the training set, and the shop name on our reference page is redacted. Development copies live only in private team storage (Section 8.10).
- **Legal context.** Pakistan has not yet enacted a dedicated personal-data-protection law; the Personal Data Protection Bill 2023 is still a draft [15]. We nevertheless follow its principles (purpose limitation, consent, security). The Prevention of

Electronic Crimes Act 2016, as amended in 2025, applies to unauthorised access to information systems [16]. Qwen2.5-VL-7B is released under Apache-2.0, which permits commercial use [17]. Noto Nastaliq Urdu (used in the UI) is under the SIL Open Font License [18].

- **Fairness and accessibility.** The interface is Urdu-first and right-to-left, with an English toggle, large touch targets and keyboard shortcuts, for users with limited literacy in English. The model is evaluated per writer, so that it does not work well only for the handwriting styles that happen to be in the training set.
- **Accountability.** The system never commits a number it cannot verify without a human decision. The hash chain gives owners evidence against tampering by staff, and all actions are attributed to a named user.
- **Responsible AI use.** Generative-AI tools used during development are disclosed in Section 8.7. No GenAI service is part of the deployed product.

Chapter 2

Research and Existing Solutions

2.1 Research Method / Approach

We investigated the problem through five complementary activities. Their outputs feed directly into the design decisions summarised in Section 2.8.

1. **Structured literature review.** We searched IEEE Xplore, the ACL Anthology, Springer Link, arXiv and Google Scholar for work published mainly between 2013 and 2026 (plus foundational work on HMAC and secure audit logs) in four strands: Urdu/Nastaliq handwriting recognition; OCR-free document understanding and vision-language models; parameter-efficient fine-tuning; and trustworthy automation (calibration, human-in-the-loop, tamper-evident logging).
2. **Product and API review.** We read the official language-support documentation of Google Cloud Vision, Azure AI Document Intelligence, Amazon Textract, Tesseract, EasyOCR and PaddleOCR, and the public descriptions of Pakistani digital-khata apps. All were accessed in October 2026.
3. **Domain analysis of a real ledger.** We transcribed page 31 of a real supplier khata (Figure 1.1) cell by cell and checked its arithmetic. This gave us the column semantics, the writing conventions (trailing "/-" closers, folio prefixes, cheque and bank notes) and a verified ground-truth fixture.
4. **Proposal-stage technical spike.** We fed the same photograph to a generic OCR engine and to an OpenCV preprocessing pipeline (deskew, binarisation, contour-based cell boxes) followed by TrOCR. This exposed the failure modes discussed in Section 5.5.
5. **Feasibility analysis.** We estimated GPU memory and trainable parameters for candidate fine-tuning strategies (`training/estimate_memory.py`, Section 6.7).

For the literature review, we included papers that report results on Urdu or Arabic-script text, on document/table understanding, or on memory-efficient fine-tuning. We then followed citation chains from the strongest sources. All bibliographic details were checked against the primary source.

2.2 Existing Solutions / Products

2.2.1 Digital khata applications

Pakistan already has popular digital ledger apps. **DigiKhata** describes itself as "Pakistan's #1 Digital Khata" and offers encrypted cloud backup and SMS/WhatsApp reminders [19]. **Easy Khata** (by Bazaar) reported 100,000 active merchants within about two months of

launch [20]. **Udhaar Book**, now Rupin, claims millions of businesses [21]. India's Khatabook and OkCredit follow the same model. All of these are *manual-entry* tools: the shopkeeper types every transaction, and the records are synchronised to the vendor's cloud. None of them reads the paper register the merchant already keeps, and none verifies historical entries.

2.2.2 Commercial OCR services

Google Cloud Vision lists Urdu only as an *experimental* OCR language, and its handwriting support does not cover Arabic script [1]. **Azure AI Document Intelligence** reads printed Urdu, but its handwriting support covers 12 languages that include Arabic and exclude Urdu [2]. **Amazon Textract** supports six European languages only [3]. Even where printed Urdu is supported, these services return lines of text, not ledger semantics (debit, credit, balance). Using them would also send the merchant's data to a foreign cloud.

2.2.3 Open-source OCR engines

Tesseract ships an LSTM model for Urdu (`urd`) and a script-level Arabic model [22]. **EasyOCR** and **PaddleOCR** (whose PP-OCRv5 Arabic-script recogniser covers Urdu) also list Urdu [23]. All three target printed text and need a separate detection stage. None claims support for handwritten Urdu, and none produces table structure. In our proposal-stage spike, generic OCR on the sample page returned unusable output, as these engines' documentation would lead one to expect.

2.2.4 Research on Urdu handwriting recognition

Urdu Nastaliq is hard for OCR. It is cursive and calligraphic. Letter shapes depend on both neighbours. Words are written diagonally, right to left and top to bottom, so neighbouring characters overlap. Dots and diacritics sit close together, and the baseline shifts. The script also has tens of thousands of ligatures [24, 25].

Segmentation-free sequence recognisers trained with CTC, such as CRNN [26], removed the need to cut words into characters, and printed Nastaliq recognition matured first. A BLSTM-CTC recogniser achieved a 5.15% character error rate on the synthetic UPTI data [27], and UTRNet reported 92.97% character accuracy on real printed lines [28]. For handwriting, the main public datasets and results are:

- **UNHD:** 500 writers, about 10,000 lines [29].
- **PUCIT-OHUL:** 7,401 lines by 100 writers; the best attention model reaches 79.76% character accuracy [30, 31].
- **NUST-UHWR:** about 10.6 k lines from about 1,000 writers. Reported character error rates are 5.28% [5] and 5.31% [4], and a multilingual vision transformer adapted to Urdu (ViLanOCR) reports 1.1% [6].

Two observations matter for us. First, every one of these systems recognises a *single pre-cropped text line*; layout and table structure are out of scope. Second, the same study reports that off-the-shelf TrOCR scored 78.23% CER on NUST-UHWR [6]. That figure is consistent with the failure we observed in our proposal spike. The CENPARMI Urdu

database is the only resource we found that targets financial handwriting (isolated digits, numeral strings, dates and "mostly financial" words), but it is a forms-based dataset of isolated items, not ledger pages [32].

2.2.5 Document understanding and vision-language models

Classical pipelines detect text, recognise lines, and then reconstruct structure. Donut showed that this chain suffers from OCR error propagation and inflexibility across languages and document types, and it proposed an OCR-free encoder-decoder trained on synthetic documents [33]. Other systems take related approaches:

- Pix2Struct pretrains on parsing web screenshots into simplified HTML [34].
- Nougat converts academic PDFs to markup [35].
- LayoutLMv3 combines OCR tokens with image patches [36], so it still depends on an OCR engine that does not exist for handwritten Urdu.
- Table structure recognition has been driven by printed scientific tables such as PubTables-1M [37].

Large VLMs now unify these capabilities. Qwen2-VL introduced naive dynamic resolution and multimodal rotary position embeddings [38]. **Qwen2.5-VL** adds a window-attention vision transformer trained at native resolution and absolute-coordinate grounding (bounding boxes in input-image pixels). It also upgrades text recognition to "omni-document parsing" and supports structured data extraction from invoices, forms and tables [10]. The 7B-Instruct model (8.29 B parameters, Apache-2.0) scores 95.7 on DocVQA and 864/1000 on OCRBench [17].

Fine-tuning such a model for document transcription is a proven route. olmOCR fine-tuned Qwen2-VL-7B on 260,000 PDF pages, including handwriting and poor scans [39], and olmOCR 2 moved to Qwen2.5-VL-7B with reinforcement learning from unit-test rewards [40].

For Arabic script, KITAB-Bench found that modern VLMs outperform traditional OCR engines (EasyOCR, PaddleOCR, Surya) by an average of 60% in CER [7]. For Urdu, the evidence is newer and limited to printed text. Gemini-2.5-Pro achieved a word error rate of 0.133 on printed Urdu newspapers, and the authors note that LLM errors are dominated by *deletions*: the model silently omits characters or words [8]. GPT-4o's Urdu WER rose from 0.20 to 0.35 as text length grew [41]. OCRBench identifies handwritten and multilingual text as the weakest areas of large multimodal models [9]. *To our knowledge, no published study evaluates open VLMs on handwritten Urdu ledgers.*

2.2.6 Trustworthy automation: calibration, HITL and tamper evidence

Modern neural networks are often over-confident [42]. A confidence threshold alone therefore cannot guarantee that a misread amount is caught, and a VLM that writes "1000007" for "100000/-" may be confident in that output. Human-in-the-loop machine learning research recommends involving people where machine uncertainty or the cost of an error is high, and reusing their corrections to improve the model [43].

For integrity, keyed hash chains over log entries make later modification detectable [44]. Tamper-evident logging can be made efficient without a distributed ledger [45], and HMAC is the standard keyed hash construction [12]. These results let us meet the proposal's anti-tampering requirement without a blockchain.

2.3 Comparison of Existing Solutions

Table 2.1 compares the existing options against criteria derived from the problem statement.

Table 2.1: Comparison of existing solutions against project-relevant criteria

Solution	Hand-written Urdu	Table / ledger fields	Offline / on-premises	Arithmetic verification	HITL review	Summaries & findings	Tamper evidence
DigiKhata, Easy Khata, Udhaar Book	No (manual typing)	Yes (typed)	No (cloud sync)	No	n/a	Partial	No
Google Cloud Vision	No (Urdu experimental, printed)	No	No	No	No	No	No
Azure Document Intelligence	No (printed Urdu only)	Partial (generic tables)	No (cloud)	No	No	No	No
Amazon Textract	No (no Urdu)	Partial	No	No	Partial (A2I)	No	No
Tesseract / EasyOCR / PaddleOCR	No (printed)	No	Yes	No	No	No	No
Urdu HTR research models	Yes (line level)	No	Yes	No	No	No	No
Cloud VLMs (GPT-4o, Gemini)	Partial (unverified)	Yes (prompted)	No	No	No	Partial	No
Open VLM zero-shot (Qwen2.5-VL)	Partial (unverified)	Yes	Yes	No	No	No	No
Proposed system	Yes (fine-tuned)	Yes (schema v1.0)	Yes	Yes (R1-R7)	Yes (flagged rows)	Yes	Yes (HMAC chain)

2.4 Limitations and Gaps

The review exposes six gaps. Together they define the space this project occupies.

- **G1, No production OCR reads handwritten Urdu.** The commercial and open-source engines either exclude Urdu or support only printed Urdu.
- **G2, Research HTR stops at the text line.** Urdu handwriting models are trained and evaluated on clean, pre-cropped lines. They do not handle ruled tables, column semantics, numerals with currency closers, crossings or text that spills across cell borders.
- **G3, VLMs are capable but unverified and mostly cloud-hosted.** VLMs can read whole documents, but they omit and alter characters, and they are weakest on

handwriting. The strongest ones run only in the cloud, which merchants reject, and no open benchmark covers Urdu ledgers.

- **G4, Nobody uses the document's own arithmetic to check extraction.** Ledgers contain redundancy (running balances, column totals, carry-forward balances, qty x rate). This redundancy can prove a transcription correct or pinpoint the faulty cell, but existing systems ignore it and rely on confidence scores, which are poorly calibrated.
- **G5, Digital khata apps do not digitise the paper.** They need 100% manual typing and cloud storage, so the historical register stays unverified.
- **G6, No path from extracted data to decisions.** None of the OCR solutions turns extraction into accounting postings, summaries or audit findings for an informal merchant. The proposal committee asked us to close this gap.

2.5 Proposed Solution

The system closes each gap with a specific mechanism (Table 2.2).

Table 2.2: How the proposed system addresses each gap

Gap	Mechanism in the proposed system	Where
G1, G2	Page-level transcription by Qwen2.5-VL-7B, fine-tuned with QLoRA on annotated khata pages into a fixed JSON schema (rows, columns, raw amounts, row boxes). No separate line segmentation, so spill-over handwriting is read in context.	Sec. 5.2, 6.6
G3	Open weights (Apache-2.0) served locally in 4-bit/AWQ form; prompt rule "transcribe, don't compute"; per-writer evaluation on a held-out split.	Sec. 5.4, 7.5
G4	Deterministic validation engine: running balance, column exclusivity, opening and carry-forward checks, handwritten totals, qty x rate; residual-based localisation with arithmetic suggestions; only broken rows go to HITL.	Sec. 6.7, Ch. 7
G5	Photograph-and-verify workflow on top of the existing paper habit; HITL screen designed for seconds per row; corrections feed adapter refresh.	Sec. 5.8, App. E
G6	Double-entry ledger, party statements, payment-mode and bank breakdowns, ageing, findings engine, offline export; HMAC-chained append-only journal.	Sec. 1.7, 4.4

2.6 Complex Computing Problem

The Seoul Accord defines a complex computing problem as one that has some or all of nine characteristics [46]. NCEAC adopts this definition for Pakistani computing programmes [47]. Table 2.3 maps each characteristic to evidence from this project.

Table 2.3: Complex computing problem characteristics (Seoul Accord) and project evidence

Characteristic	Evidence in this project
Range of conflicting requirements	Model capacity (an 8.3 B-parameter VLM) against offline execution on merchant-affordable hardware; zero-defect financial accuracy against minimal human effort; strong tamper evidence against a single-machine deployment with no external notary.

Characteristic	Evidence in this project
Depth of analysis required	No off-the-shelf solution exists. The team had to analyse memory budgets (Sec. 6.7), design invariant-based error localisation with a case analysis of when it can and cannot localise (Sec. 6.7, 7.6), and design evaluation without a public benchmark.
Depth of knowledge required	Multimodal transformers, quantisation and PEFT (NF4, LoRA), Urdu script linguistics and normalisation, double-entry accounting, cryptographic hash chains, database integrity, HCI for low-literacy users.
Familiarity of issues	Handwritten Urdu financial tables are not covered by existing datasets or products (gaps G1-G4); the failure modes (closers read as digits, deletions, spill-over) are new to the team and to the literature.
Level of problem	A system-level problem spanning ML, back-end services, data management, security and UI, not a single algorithm.
Extent of stakeholder involvement and conflicting requirements	Owners want control and privacy; Munshis want speed; parties need correct balances; evaluators need evidence. RBAC, HITL design and the integrity model balance these needs.
Consequences	A wrong balance can cause financial loss or disputes between traders; leakage of turnover data has commercial and tax consequences. Hence zero silent numeric errors (REL-01) and zero-cloud deployment.
Interdependence	Extraction schema, validation rules, HITL UI, ledger posting and analytics are tightly coupled through the JSON contract; a change in one ripples to the others.
Requirement identification	Requirements emerged from analysing a real ledger, from the proposal-stage spike and from committee feedback, rather than being given; several were redefined after the pivot.

2.7 Technical Challenges

Table 2.4: Technical challenges and planned handling

Challenge	Why it is hard	Handling
Cursive Nastaliq handwriting	Diagonal, overlapping ligatures; dot placement; writer variation	Whole-page VLM reading with context; per-writer evaluation split; augmentation
Numerals with currency closers	"100000/-" or "100000-" can be read as 1000007 or 1000001; 4/6 and 1/7 confusions	"Transcribe, don't compute" prompt; deterministic sanitiser; invariant checks (R1, R5)
Layout irregularity	Writing spills across ruled cells; circled notes; struck-out entries; page curvature	Page-level model with absolute row boxes; exclusivity check (R2); HITL for ambiguous rows
Model omissions and hallucinations	VLMs drop characters or rows and can repeat lines	Row-drop and duplicate detection through residuals; page totals; schema validation
Compute limits	8.3 B parameters need about 124 GiB for full fine-tuning	QLoRA (NF4 base, LoRA r = 16), gradient checkpointing, batch 1 with accumulation, image-token cap
Data scarcity and privacy	No public Urdu ledger dataset; pages contain third-party data	Consented collection, pseudonymisation, ground-truth self-check, synthetic layout pre-training
Entity resolution	The same party is written in different ways (M. Ali vs. Muhammad Ali)	Urdu normalisation, alias table, fuzzy suggestions confirmed by the owner

Challenge	Why it is hard	Handling
Inference latency on shop hardware	7B VLM on a consumer GPU	AWQ 4-bit inference, token cap, batching, market-hub option
Trust and tamper resistance	A Munshi with database access could edit history	Owner-unlocked HMAC chain, append-only triggers, nightly verification

2.8 Research Findings and Design Decisions

Table 2.5: Research findings and the design decisions they drove

Finding	Source	Decision
Line-level OCR fails on table pages; generic TrOCR scores 78% CER on Urdu handwriting	Proposal spike; [6, 48]	Drop OpenCV cell segmentation and TrOCR; read the whole page with a VLM (Sec. 5.5)
VLMs outperform traditional OCR on Arabic script	[7, 10]	Use Qwen2.5-VL-7B as the extraction engine
VLMs omit or alter characters; handwriting is their weakest area	[8, 9, 41]	Never trust model numbers alone; add a deterministic validation layer
Confidence scores are poorly calibrated	[42]	Gate numeric fields by invariants; use confidence only for text fields
Ledgers contain arithmetic redundancy (balances, totals)	Domain analysis of page 31	Invariants R1-R7 and residual-based localisation; transcribe raw strings
QLoRA preserves 16-bit fine-tuning quality at 4-bit	[11, 49]	NF4 4-bit base with LoRA adapters; vision encoder frozen
Fine-tuning Qwen-VL-7B for document OCR works at scale	[39, 40]	Prompt-to-JSON SFT on curated pages; future RL with unit-test-style rewards
Keyed hash chains give tamper evidence without a blockchain	[44, 45, 12]	HMAC-SHA256 chain over journal entries; owner-held key
Merchants reject cloud tools; Urdu handwriting is not supported by cloud OCR	Proposal; [1, 2, 3]	Zero-cloud deployment on shop PC or market hub

Chapter 3

Proposal Feedback and Iteration Plan

3.1 Proposal Feedback Summary

The FYP committee communicated its decision on 14 September 2026: **"Approved With Major Modifications (No redefense)"** for project F26-107-LedgerDigitization. The committee's comments were:

> *"Increase your scope; do not limit it to just OCR. Create end-to-end automation (including debit and other features) so shopkeepers can get all summaries and findings. The FYP suffers from a limited scope despite aiming to solve an actual problem. The students need to discuss their idea with their supervisor and factor in how to use the structured data received once OCR is performed. The project needs to demonstrate more substantial contributions, and the students' work and specific contributions should be clearly identified and presented. Integrate the suggestions provided by the panel."*

We broke this feedback into six actionable directives (F1-F6). We added two internal findings from our own proposal-stage spike (F7-F8), because they changed the architecture:

- **F1:** Increase scope beyond OCR.
- **F2:** Build end-to-end automation that includes debit (and credit) handling.
- **F3:** Give shopkeepers summaries and findings.
- **F4:** Discuss with the supervisor how the structured data will be used after extraction.
- **F5:** Demonstrate more substantial contributions.
- **F6:** Clearly identify each student's work and contribution.
- **F7 (internal):** OpenCV contour mapping breaks when handwriting spills across cell borders.
- **F8 (internal):** TrOCR hallucinates without full layout context.

3.2 Proposal Action Register

Table 3.1 is the summary register. Appendix A gives the complete register with detailed evidence. The status values are: **Done** (implemented and evidenced), **Designed** (specified in the SRS/SDS and scheduled), and **In progress**.

Table 3.1: Proposal action register (summary)

No.	Feedback / required change	Action taken	Status	Evidence
1	F1: Increase scope beyond OCR	Added modules M5 Ledger Core, M6 Analytics & Findings and M7 Security & Integrity; scope table rewritten; 44 FRs across 8 modules	Designed	Tables 1.1, 1.2; Sec. 4.4
2	F2: End-to-end automation incl. debit	Naam/jama mapped to debit/credit double-entry postings; running-balance validation already uses debit and credit (R1, R2); party accounts and statements	Done (validation) / Designed (posting)	engine.py; Fig. 5.7; LED-01..04
3	F3: Summaries and findings for shopkeepers	Analytics prototype computes opening/closing, bills, payments, cash vs. cheque by bank on a real page; findings engine specified with 6+ finding types	In progress (prototype done)	analytics.py, TC-25/26; Fig. E.3; ANA-01..06
4	F4: Use of structured data after OCR	Data life-cycle defined: JSON schema v1.0, then verified rows, then journal and postings, then analytics. Supervisor sign-off to be recorded in the meeting log	Designed; sign-off pending	Sec. 5.7; Figs. 5.6, 5.3, 5.4; App. F
5	F5: More substantial contribution	New contributions: (i) invariant-based error localisation and suggestion engine, (ii) verified Urdu ledger JSON dataset pipeline, (iii) QLoRA VLM for Nastaliq ledgers, (iv) offline audit-and-insight layer with HMAC chain	In progress	Sec. 2.5, 6.5; Ch. 7 results
6	F6: Identify individual contributions	Module ownership matrix, per-member contribution table, commit-history policy, individual demo responsibilities; each member owns code in Iteration 2 (Table 9.3)	Done	Sec. 3.6, 8.3, 8.4; App. D
7	F7/F8: Classical pipeline failure (spike)	Architecture pivot to Qwen2.5-VL-7B page-level extraction with deterministic validation and HITL routing	Done (design) / In progress (training)	Sec. 5.5, Fig. 5.2
8	Integrate panel suggestions	All items above traced to requirements and tests (traceability matrix)	Done	Table 4.19

3.3 Iteration Strategy

We follow an iterative-incremental process. It has four iterations across the two semesters, each split into two-week sprints, and every iteration ends at an evaluation gate with a live demonstration. We ordered the iterations by risk:

- **Iteration 1** (FYP-1, Weeks 5-8): de-risks the riskiest technical question, which is whether extracted numbers can be trusted. It does this by building and testing the deterministic validation layer and preparing the fine-tuning pipeline.
- **Iteration 2** (FYP-1, Weeks 9-16): produces the first fine-tuned adapter and the end-to-end core (upload, extraction, validation, HITL, signed commit).
- **Iteration 3** (FYP-2, Weeks 1-8): delivers the ledger core, analytics and findings, and party management.
- **Iteration 4** (FYP-2, Weeks 9-16): delivers continuous learning, security hardening, deployment and UAT.

Each sprint has an owner-assigned task list in the repository issue tracker, and each iteration's output is tagged in Git (Section 8.6). Figure 3.1 shows the two-semester roadmap.

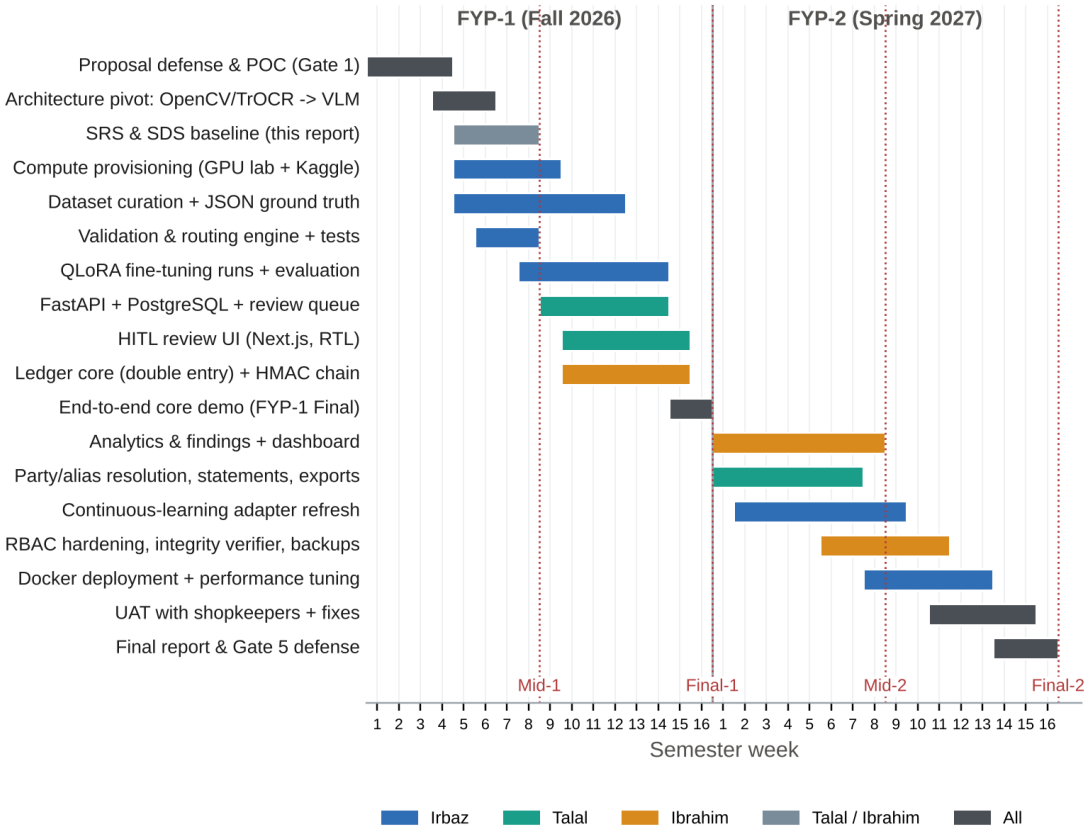


Figure 3.1: Two-semester iteration roadmap with task owners and evaluation gates

3.4 Iteration 1 Plan

Table 3.2: Iteration 1 plan (FYP-1 Weeks 5-8)

Item	Detail
Objectives	(1) Baseline the SRS and SDS for the new architecture. (2) Make extraction errors detectable and correctable through a deterministic validation and routing layer. (3) Prepare data and training infrastructure for QLoRA fine-tuning. (4) Address all proposal-committee directives in the design.
Selected features	VAL-01..VAL-09 (validation and routing); EXT-02 (raw-string schema contract); data pairing for SFT (HITL-05 format); QLoRA training script and memory plan; analytics prototype (ANA-01/02 subset)
Deliverables	This report (SRS + SDS); `ledger_validation` package; JSON schema v1.0 and prompt `extract_v1`; `build_sft_jsonl.py`; `train_qlora.py`; `estimate_memory.py`; verified fixture page 31; 39 automated tests; mutation evaluation results; updated diagrams
Evidence	Test log (App. C); mutation results (Sec. 7.4); live demo script (Sec. 6.9); repository tag `v0.1-mid` (App. D)

3.5 Iteration 2 Plan

Table 3.3: Iteration 2 plan (FYP-1 Weeks 9-16)

Item	Detail
Objectives	(1) Train and evaluate the first Qwen2.5-VL-7B QLoRA adapter. (2) Deliver the end-to-end core: image to extraction to validation to HITL to signed DB commit. (3) Measure baseline accuracy and HITL rate.
Selected features	ING-01..04, EXT-01..05, HITL-01..05, LED-01 (posting), SEC-02/03 (hash chain, append-only), OPS-03 (adapter registry, basic)
Deliverables	Annotated dataset (target 300 or more pages, writer-disjoint splits); adapter v1 with evaluation report; FastAPI service and PostgreSQL schema; Next.js review UI; Docker Compose for development
Evidence	Evaluation report (field accuracy, CER, schema validity, residual errors); end-to-end demo on unseen pages; integration tests; tag `v0.2-final1`

3.6 Work Breakdown and Responsibilities

Every module has one accountable owner, and shared tasks have a named lead. Mid-term tasks follow the team's action plan. FYP-2 ownership follows the proposal's role split with one deliberate change: Docker deployment and the fine-tuning worker move from Member 3 to Irbaz, so that all model work has a single owner.

Table 3.4: Work breakdown and ownership

Work package	Owner	Mid-term (FYP-1) tasks	FYP-2 tasks
WP1 AI engineering (M2, M3, M8)	Irbaz Ali	Compute provisioning (GPU lab request, Kaggle fallback); dataset curation and JSON pairing; QLoRA training scripts; validation and routing logic	Adapter refresh from corrections; AWQ deployment; performance tuning; Docker
WP2 System design & HITL (M1, M4)	Talal Shahid	Architecture, DFD, UML diagrams; design evolution; test-case design (shared); repository hygiene (shared)	FastAPI orchestration, ingestion, Next.js review UI, party statements and exports
WP3 Requirements, ledger & analytics (M5, M6, M7)	Muhammad Ibrahim	FYP-1 mid report; R&D and gap analysis; implementation chapter; SRS; test documentation (shared); action register (shared)	Double-entry ledger, analytics and findings engine, RBAC, HMAC chain and verifier
WP4 Testing & evaluation	Shared (lead: Talal)	Formal test cases for visual artefacts ("/-" closers); mutation evaluation	Integration, performance, UAT with shopkeepers

3.7 Progress Against Plan

Table 3.5 compares the Mid-1 milestone promised in the proposal with the actual state at the time of writing (Week 8).

Table 3.5: Planned milestones vs. actual progress (Mid-1)

Planned in proposal (Mid-1)	Actual status at Mid-1	Status
SRS and SDS baselined	Baselined in this report (Ch. 4, 5) for the revised architecture	Done
Tabular cell segmentation (OpenCV)	Replaced by page-level VLM extraction after the spike showed contour mapping fails on spill-over handwriting	Re-planned (design change)
Base HTR model trained	Replaced by QLoRA fine-tuning of Qwen2.5-VL-7B. Schema, pairing script, training script and memory plan ready; first run waits on GPU allocation (Kaggle fallback available)	In progress (delayed)
(added) Validation and routing engine	Implemented and tested: 39/39 tests; mutation study on a real page	Done
(added) Scope expansion per committee	Ledger core, analytics, security modules specified; analytics prototype running	Done (design) / In progress
(added) Dataset pipeline	Schema v1.0, ground-truth self-check, first verified page; collection of further pages in progress	In progress

3.8 Delays, Deviations and Recovery Plan

Deviation 1: architecture pivot. Replacing OpenCV and TrOCR with a VLM invalidated the planned Mid-1 deliverable "cell segmentation and base HTR model". The pivot was deliberate and is justified in Section 5.5. The new pipeline is simpler (one model instead of three stages), and the validation layer we built instead addresses the proposal's zero-defect requirement more directly.

Delay 1: fine-tuning compute. Training an 8.3 B-parameter model needs a 16-24 GB GPU. A request for departmental GPU lab access (RTX 3090/4090 or L4 class) is pending. The recovery plan, with owners, is:

Table 3.6: Recovery plan for the fine-tuning delay

Action	Owner	Due
Follow up the GPU-lab request with the supervisor's endorsement	Irbaz (with supervisor)	Week 9
Start training on Kaggle T4 x2 in fp16 using the pooled weekly quota of three accounts (~90 h/week); checkpoint to a private Hugging Face repository at the end of every session; log to Weights & Biases (metrics only, no images)	Irbaz	Week 9
Freeze writer-disjoint train/validation/test splits so results are comparable whichever GPU is used	Irbaz, Ibrahim	Week 9
First evaluation report (adapter v1) with field accuracy, CER and schema validity	Irbaz	Week 12
Fallback model: if the 7B run does not converge in time, evaluate Qwen2.5-VL-3B with the same pipeline (same schema and validator)	Irbaz	Week 13

Risk to the FYP-1 Final demo. The end-to-end demo depends on the adapter only for accuracy, not for functionality. The validation, routing, HITL and commit path can be

demonstrated with the base model in zero-shot mode. A slip in training therefore cannot block the Final-1 demonstration.

Chapter 4

Software Requirements

This chapter is the Software Requirements Specification (SRS) baseline for the revised architecture. It has 44 functional requirements across eight modules and 17 non-functional requirements, each with a testable acceptance check. Priorities use MoSCoW notation (M = must, S = should, C = could). "Iter" gives the iteration in which the requirement is implemented; 1-2 are FYP-1 and 3-4 are FYP-2.

4.1 User Interactions / Use Cases

The system has four actors. The **Munshi** captures pages and resolves flagged rows. The **Shop Owner** does everything the Munshi does and also approves, views insights and findings, exports, manages users and holds the integrity key. The **System Administrator** installs and maintains the system and model adapters. The **Scheduler** is a time actor that triggers nightly integrity verification and adapter refresh.

The main interaction loop is: photograph a page, wait about a minute while the page is processed, then resolve the one to three flagged rows on screen. The owner's loop is: open the dashboard, read the summaries and findings, then drill down to a party statement or the original page image.

Table 4.1: Use case catalogue

ID	Use case	Primary actor(s)	Related FRs
UC-01	Upload ledger page	Munshi, Owner	ING-01..04
UC-02	Extract ledger data (VLM)	System (included by UC-01)	EXT-01..05
UC-03	Validate and route rows	System (included by UC-02)	VAL-01..09
UC-04	Review flagged rows	Munshi, Owner	HITL-01..05
UC-05	Commit and sign entries	System (included by UC-03)	LED-01, SEC-02/03
UC-06	Manage parties and aliases	Munshi, Owner	LED-02
UC-07	View party statement	Munshi, Owner	LED-03, LED-05
UC-08	View insights dashboard	Owner	ANA-01..03
UC-09	View audit findings	Owner	ANA-04, ANA-05
UC-10	Export reports	Owner	ANA-06
UC-11	Manage users and roles	Owner	SEC-01, SEC-05
UC-12	Verify ledger integrity	Owner, Scheduler	SEC-04
UC-13	Update model adapter	Admin, Scheduler	OPS-03, OPS-04
UC-14	Backup and restore	Admin	OPS-02

4.2 Use Case Diagram

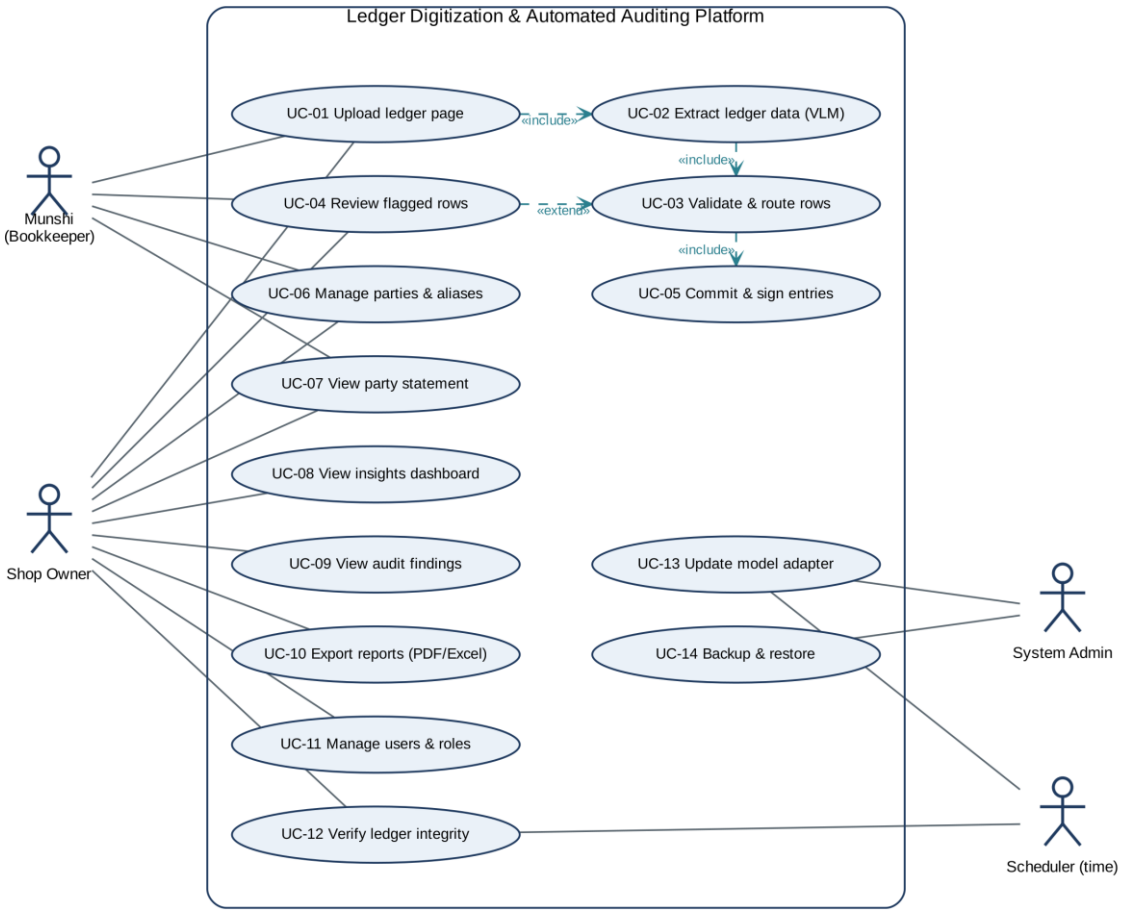


Figure 4.1: Use case diagram of the Ledger Digitization and Automated Auditing Platform

Figure 4.1 shows the use cases. UC-01 includes UC-02, which includes UC-03, which includes UC-05: every upload is extracted, validated and (where verified) committed without user action. UC-04 extends UC-03 only when validation raises blocking flags.

4.3 Detailed Use Cases

Tables 4.2-4.4 give the three use cases that Iterations 1-2 implement. The remaining detailed use cases are in Appendix B.

Table 4.2: UC-01 Upload ledger page

Field	Description
Actors	Munshi (primary), Owner
Preconditions	User is logged in; a ledger book exists; the inference worker is running

Field	Description
Main flow	1. User selects the book and enters or confirms the page number. 2. User photographs the page or picks a file. 3. System checks image quality (ING-03) and shows a warning if it is blurred or dark. 4. System stores the image, computes SHA-256 and rejects duplicates (ING-02). 5. System creates an extraction job and returns the page status "Processing". 6. System extracts (UC-02), validates (UC-03) and commits verified rows (UC-05). 7. System shows "Verified" or "n rows need review".
Alternate flows	3a. User retakes the photo. 4a. Duplicate: system opens the existing page. 6a. Invalid JSON after retry: page enters manual-entry mode (EXT-05).
Postconditions	Page and extraction run stored; verified rows signed and appended; review tasks created for flagged rows

Table 4.3: UC-03 Validate and route rows (system use case)

Field	Description
Trigger	Extraction run completes with schema-valid JSON
Main flow	1. Sanitise numeric cells (VAL-01); mark unreadable cells and infer their values from neighbours. 2. Apply ledger-type rules: khata R1-R5 or bill R6-R7. 3. Localise each failure to a row and cell and compute a suggestion (VAL-03). 4. Apply the confidence gate to text fields (VAL-08). 5. Produce the routing report: auto-commit rows and HITL rows (VAL-09).
Alternate flows	2a. Only the handwritten total disagrees: raise a page-level TOTAL_MISMATCH finding; rows commit (VAL-05). 2b. Opening row disagrees with the previous page: raise CARRY_FORWARD_MISMATCH (VAL-06).
Postconditions	Every row is either verified or has a review task; no unverified number is committed

Table 4.4: UC-04 Review flagged rows

Field	Description
Actors	Munshi (primary), Owner
Preconditions	At least one open review task exists
Main flow	1. User opens the review queue (HITL-01). 2. System shows the row crop, the model reading, the flag reason and the suggestion (HITL-02). 3. User accepts the suggestion (Enter) or edits the value (E). 4. System re-validates the page (HITL-04); the row becomes verified and is signed and committed. 5. System stores the correction pair (HITL-05) and moves to the next task.
Alternate flows	3a. The row is a duplicate or not a ledger line: user discards it. 3b. A line was skipped by the model: user marks "skipped line" and enters the missing row. 3c. User is unsure: escalate to the owner (O).
Postconditions	Task resolved; page VERIFIED when no blocking flags remain; audit log updated

4.4 Functional Requirements

4.4.1 Module M1 Capture & Ingestion

Table 4.5: Functional requirements: M1 Capture & Ingestion

ID	Requirement	Pri.	Iter	Acceptance check
ING-01	The system shall accept JPEG, PNG or HEIC page images up to 20 MB from the browser (camera or file picker) and create a page record.	M	2	Upload of a 12 MP phone photo returns `page_id` with HTTP 202 within 2 s on LAN
ING-02	The system shall compute the SHA-256 of each image and reject a duplicate upload for the same book.	M	2	Re-upload returns HTTP 409 with the existing `page_id`
ING-03	The system shall warn the user about blurred or under-exposed photos (variance-of-Laplacian and mean-luminance thresholds) and offer a retake.	S	2	>= 18 of 20 deliberately blurred test photos produce a warning; <= 2 of 20 sharp photos do
ING-04	The system shall associate each page with a ledger book, page number and, optionally, a party.	M	2	Page record contains `book_id`, `page_no`; filter by book returns it

4.4.2 Module M2 VLM Extraction Engine

Table 4.6: Functional requirements: M2 VLM Extraction Engine

ID	Requirement	Pri.	Iter	Acceptance check
EXT-01	The system shall run Qwen2.5-VL-7B with the active LoRA adapter locally and return one JSON object conforming to schema v1.0 per page.	M	2	>= 99% of held-out test pages produce schema-valid JSON (after at most one retry)
EXT-02	The system shall preserve every amount exactly as written (raw string, including closers such as "/-") and shall not perform arithmetic in the model.	M	1	Prompt `extract_v1` rule 1-2; annotation guide; schema field types are strings
EXT-03	The system shall return a bounding box (`bbox_2d`, absolute pixels) for every row.	S	2	IoU >= 0.5 with annotated row boxes for >= 90% of test rows
EXT-04	The system shall record model version, adapter version, prompt version and latency for every extraction run.	M	2	`extraction_runs` row contains all four fields
EXT-05	On invalid JSON the system shall retry once with a strict prompt and otherwise route the whole page to manual-entry mode.	M	2	Fault-injection test with malformed output produces one retry then a manual task

4.4.3 Module M3 Validation & Routing

Table 4.7: Functional requirements: M3 Validation & Routing

ID	Requirement	Pri.	Iter	Acceptance check
VAL-01	The system shall canonicalise raw amounts (closers "/-", "/=", "-", "=/-"; Urdu and Arabic-Indic digits; separators; currency words) to exact decimals and shall raise a FORMAT flag instead of guessing.	M	1	TC-01..TC-06 pass
VAL-02	The system shall check the running-balance invariant B[i] = B[i-1] + J[i] - N[i] for every row of a khata page.	M	1	TC-07, TC-08 pass
VAL-03	The system shall localise a failed invariant to a row and cell (amount vs. balance, opening row, closing row) and attach an arithmetic suggestion.	M	1	TC-10, TC-14..TC-17 pass; >= 95% localisation in mutation study
VAL-04	The system shall detect rows with both or neither of naam/jama, duplicated rows and debit/credit column swaps.	M	1	TC-11, TC-12, TC-18 pass

ID	Requirement	Pri.	Iter	Acceptance check
VAL-05	The system shall compare column sums with handwritten page totals and raise a page-level finding when only the total disagrees.	M	1	TC-19 passes
VAL-06	The system shall check that a page's brought-forward balance equals the previous page's closing balance in the same book.	S	1	TC-20 passes
VAL-07	The system shall check quantity x rate = amount (tolerance 0.5 PKR for fractional values) and bill totals for itemised bills.	S	1	TC-22..TC-24 pass
VAL-08	The system shall route rows whose text fields (date, details, folio) have model confidence below a configurable threshold (default 0.80).	S	1	TC-21 passes
VAL-09	The system shall commit rows that pass all checks and route only flagged rows to HITL, producing a machine-readable routing report.	M	1	TC-08: 19 rows auto-commit, 1 routed

4.4.4 Module M4 HITL Review Workbench

Table 4.8: Functional requirements: M4 HITL Review Workbench

ID	Requirement	Pri.	Iter	Acceptance check
HITL-01	The system shall provide a review queue of open tasks ordered by page and age.	M	2	Queue lists tasks created by VAL within 1 s of page validation
HITL-02	For each task the system shall show the row crop (from `bbox_2d`), the model reading, the flag reason and the suggestion, rendering Urdu right-to-left.	M	2	Usability walkthrough checklist; RTL rendering verified on Chrome and Edge
HITL-03	The system shall support keyboard actions: accept suggestion (Enter), edit (E), mark skipped line (S), escalate to owner (O).	S	2	Every action reachable without a mouse; median <= 10 s per flagged row in UAT
HITL-04	After each correction the system shall re-validate the whole page and commit rows once they are verified.	M	2	Integration test: correction to "100000" yields VERIFIED and signed entry
HITL-05	The system shall store every correction (image crop, field, old value, new value, reviewer) as a training pair.	M	1-2	`corrections` row created per edit; export script produces SFT records

4.4.5 Module M5 Ledger Core & Parties

Table 4.9: Functional requirements: M5 Ledger Core & Parties

ID	Requirement	Pri.	Iter	Acceptance check
LED-01	The system shall post each verified row as a balanced journal entry: naam is a debit and jama a credit on the party account, with the counter-posting on cash, bank or purchases according to the details.	M	2-3	DB constraint: sum of debits = sum of credits per entry; page 31 posts 19 entries
LED-02	The system shall manage parties with an alias table and Urdu normalisation (ي/ی, ک/ك, ہ/ه, diacritics) and suggest alias matches for owner confirmation.	M	3	"M. Ali" vs. "Muhammad Ali" suggested as one party; owner confirmation required
LED-03	The system shall produce a party statement with running balance for any date range that reconciles with the handwritten balances.	M	3	Statement of page 31 ends at 365,603

ID	Requirement	Pri.	Iter	Acceptance check
LED-04	The system shall correct committed entries only through reversal entries, never by editing.	M	3	UPDATE on `journal_entries` rejected; reversal entry reverses effect
LED-05	The system shall link consecutive pages of a book and carry closing balances forward.	S	3	Page 32 opening checked against page 31 closing (VAL-06)

4.4.6 Module M6 Analytics & Findings

Table 4.10: Functional requirements: M6 Analytics & Findings

ID	Requirement	Pri.	Iter	Acceptance check
ANA-01	The system shall show per-party and shop-wide summaries: opening and closing balance, new bills, payments, and totals by day, week and month.	M	3	Page 31 summary reconciles: 1,412,143 + 566,210 - 1,612,750 = 365,603 (TC-26)
ANA-02	The system shall break payments down by mode (cash vs. cheque) and by bank, derived from the details text.	M	3	Page 31: cash 700,000; Meezan 650,000; MCB 150,000; UBL 100,000; other 12,750 (TC-25/26)
ANA-03	The system shall show receivable and payable ageing (0-30, 31-60, 61-90, >90 days) and the top parties by outstanding balance.	S	3	Synthetic fixture with known ages produces expected buckets
ANA-04	The system shall generate findings: total mismatch, carry-forward mismatch, duplicate folio or reference, unusual amount (robust z-score > 3.5 against the party's history), dormant account with balance, integrity failure.	M	3	One seeded fixture per finding type triggers exactly that finding
ANA-05	Each finding shall carry evidence (rows, image crops), severity and a status workflow (open, acknowledged, resolved).	S	3	State transitions logged in `audit_log`
ANA-06	The system shall export statements and summaries to PDF and Excel offline, in Urdu or English.	S	3	Exported files open in Excel/PDF reader; Urdu renders right-to-left

4.4.7 Module M7 Security & Integrity

Table 4.11: Functional requirements: M7 Security & Integrity

ID	Requirement	Pri.	Iter	Acceptance check
SEC-01	The system shall enforce the role permissions of Table B.1 (Owner, Munshi, Viewer, Admin) at the API.	M	2-4	Munshi call to delete or settings endpoints returns HTTP 403
SEC-02	The system shall chain every journal entry with HMAC-SHA256(key, canonical(entry) || prev_hmac), using a key derived from the owner's passphrase.	M	2	Recomputing the chain reproduces every stored `row_hmac`
SEC-03	The system shall reject UPDATE and DELETE on `journal_entries`, `postings` and `audit_log` with database triggers.	M	2	SQL UPDATE raises an exception; test in CI
SEC-04	The system shall verify the chain on demand and nightly, raising a tamper finding with the first broken entry.	M	4	Direct DB edit of one amount is detected in 100% of trials
SEC-05	The system shall log every user action (who, what, when, target) in the audit log.	M	2-4	Audit rows exist for upload, review, commit, export, login

4.4.8 Module M8 Deployment & Model Lifecycle

Table 4.12: Functional requirements: M8 Deployment & Model Lifecycle

ID	Requirement	Pri.	Iter	Acceptance check
OPS-01	The system shall install with one Docker Compose command and run with no outbound network access.	M	4	`docker network inspect` shows internal network; egress test fails as intended
OPS-02	The system shall create a nightly encrypted backup and support restore.	M	4	Restore on a clean machine reproduces ledger and chain head
OPS-03	The system shall keep a registry of adapter versions with metrics, and shall promote a new adapter only if it does not regress on the frozen test split.	S	2-4	Promotion blocked when field accuracy drops
OPS-04	The system shall schedule local QLoRA adapter refresh from accumulated correction pairs.	C	4	Refresh job produces adapter vN+1 with evaluation report
OPS-05	The system shall expose health and metrics: extraction latency, HITL rate, queue length.	S	4	Admin page shows metrics for the last 7 days

4.5 Non-Functional Requirements

4.5.1 Reliability

Table 4.13: Reliability requirements

ID	Requirement	Verification
REL-01	Zero silent numeric errors: every committed numeric cell either passed all applicable invariants or was confirmed by a named reviewer.	Audit query: committed cells without a passing check or reviewer = 0
REL-02	No page is lost if the inference worker crashes; jobs are re-queued after a 5-minute lease timeout.	Kill worker mid-job; job completes after restart
REL-03	Recovery point objective <= 24 h; recovery time objective <= 1 h.	Restore drill from nightly backup

4.5.2 Usability

Table 4.14: Usability requirements

ID	Requirement	Verification
USE-01	Urdu-first right-to-left interface (Noto Nastaliq Urdu for Urdu text) with an English toggle; all actions labelled bilingually.	UI review checklist
USE-02	Median review time <= 10 s per flagged row; HITL rate <= 20% of rows (FYP-1 Final) and <= 10% (Final-2).	Instrumented UAT sessions; production logs
USE-03	A new Munshi can upload and review a page after <= 15 minutes of training; System Usability Scale >= 70.	UAT with at least 5 shop staff
USE-04	Touch targets >= 44 px; works on a 6-inch phone screen.	Responsive design test

4.5.3 Performance

Table 4.15: Performance requirements

ID	Requirement	Verification
PER-01	Extraction of a 20-30 row page <= 60 s on a 24 GB GPU (RTX 3090 class) and <= 120 s on a 12 GB GPU.	Timed runs over 30 test pages
PER-02	Validation and routing <= 50 ms per page on a CPU.	Micro-benchmark of `validate_page`
PER-03	Dashboard and statement queries <= 2 s for 10,000 journal entries.	Load test with synthetic ledger

4.5.4 Security

Table 4.16: Security and privacy requirements

ID	Requirement	Verification
SCN-01	No network egress from any container in production.	Egress probe from each container fails
SCN-02	PINs and passphrases stored with Argon2id; the HMAC key never stored in plain text.	Code review; database inspection
SCN-03	100% of direct modifications to committed rows are detected by the integrity verifier.	Tamper test suite (SEC-04)
SCN-04	Development datasets are pseudonymised and stored only in private repositories; no images are sent to experiment trackers.	Data-handling checklist (Sec. 8.10)

4.5.5 Maintainability and Portability

Table 4.17: Maintainability and portability requirements

ID	Requirement	Verification
MNT-01	Statement coverage >= 80% for validation, ledger and analytics modules; tests run on every push.	Coverage report in CI
MNT-02	Extraction contract versioned (schema v1.0, prompt v1); breaking changes bump the major version.	Schema files under version control
POR-01	Runs on Ubuntu 22.04/24.04 and Windows 11 with WSL2 and an NVIDIA driver.	Installation test on both platforms

4.6 Acceptance Criteria

Table 4.18 lists the objective acceptance checks for the most important requirements. Each check is pass/fail and repeatable. Checks marked "verified" were run at Mid-1 (Chapter 7).

Table 4.18: Acceptance checks for key requirements

Req.	Acceptance check (given / when / then)	Status
VAL-02, VAL-09	Given verified page 31, when one naam cell is changed from "100000-" to "1000007", then exactly row 1 is routed with suggestion 100000 and 19 rows are auto-committed.	Verified (TC-08)
VAL-01	Given amounts "100000/-", "26,100/=", "١٢٧٥٠", "Rs. 4,720/-", when sanitised, then 100000, 26100, 12750, 4720 are returned; "10O000" raises FORMAT.	Verified (TC-01..06)
VAL-03	Given 300 random single-cell misreads per error type on page 31, then >= 95% are routed to the correct row.	Verified (100%)
O2 / SC2	Given clean pages and benign closer variants, then no row is routed (false-alarm rate <= 2%).	Verified (0/300)
EXT-01	Given the held-out test split, when extracted, then >= 99% of pages are schema-valid.	Planned (Iter. 2)
SC1	Given the test split, then numeric-cell exact match >= 95% before validation.	Planned (Iter. 2)
HITL-04	Given a routed row, when the reviewer accepts the suggestion, then the page becomes VERIFIED and the row is signed.	Planned (Iter. 2)
LED-01	Given verified page 31, when posted, then 19 balanced journal entries exist and the party balance equals 365,603.	Planned (Iter. 2-3)
SEC-04	Given a committed ledger, when any amount is edited directly in PostgreSQL, then verification reports the first broken entry.	Planned (Iter. 4)
ANA-01/02	Given verified page 31, then the summary shows opening 1,412,143; bills 566,210; payments 1,612,750 (cash 700,000; cheques 900,000; other 12,750); closing 365,603.	Verified (TC-26, prototype)

4.7 Requirements Traceability

Table 4.19 traces the key requirements to their module, design artefact, implementation unit and test. Appendix B has the complete matrix.

Table 4.19: Requirements traceability (key requirements)

Requirement	Module	Design element	Implementation	Test
VAL-01	M3	Fig. 5.8 (sanitise step)	`sanitize.canonical_amount`	TC-01..06
VAL-02, VAL-03	M3	Fig. 5.8; Algorithm 1	`engine.validate_running_balance`, `_residual`	TC-07, 08, 10, 14-17
VAL-04	M3	Algorithm 1 cases (b),(c)	`_is_duplicate`, swap checks	TC-11, 12, 18
VAL-05, VAL-06	M3	Table 6.1 (R4, R5)	`column_total_ok`, carry-forward block	TC-19, 20
VAL-07	M3	Rule R6, R7	`engine.validate_itemized`	TC-22..24
VAL-08, VAL-09	M3	Fig. 5.7 steps 8-12	`_confidence_flags`, `_finish`	TC-21, 08
EXT-02	M2	Schema v1.0 (Listing 6.1)	`prompts/extract_v1.txt`	Annotation review
EXT-01	M2	Fig. 6.2	QLoRA config (Listing 6.3)	Eval report (Iter. 2)
HITL-05	M4/M8	Fig. 5.6 (D3)	`data/build_sft_jsonl.py`	TC-27
ANA-01, ANA-02	M6	Fig. E.3	`analytics.page_summary`	TC-25, 26

Requirement	Module	Design element	Implementation	Test
LED-01	M5	Figs. 5.3, 5.4	journal/postings schema (Iter. 2)	Planned
SEC-02..04	M7	Fig. 5.9	integrity service (Iter. 2-4)	Planned

Chapter 5

System Design

This chapter is the Software Design Specification (SDS) baseline. Every artefact describes the *current* architecture after the pivot. Section 5.5 explains how and why the design differs from the proposal.

5.1 System Architecture

The system uses a strict four-layer architecture deployed on a single on-premises host (Figure 5.1):

- a presentation layer (Web-based Canvas Annotation Studio in HTML5/JS with Noto Nastaliq Urdu typography, and Streamlit HITL Review Workbench);
- an application layer (Python REST API services in server.py, and deterministic math validation in math_engine.py);
- an AI inference and fine-tuning layer (locally hosted 4-bit NF4 quantized Qwen2.5-VL-7B on physical NVIDIA RTX 3080 10GB GPU, with dynamic LoRA adapter loading and local QLoRA training pipeline);
- a data layer (structured JSON ledger stores dataset/*.json, with SQLite/PostgreSQL enterprise schema designed for FYP-2 deployment).

Layers interact only through the documented contracts of Table 5.2: the browser talks only to the API, the API and the inference worker both use the application-layer services (validation, ledger, integrity), and only those services write to the database. The whole stack sits inside a zero-cloud boundary: the Docker network is declared `internal`, so containers have no route to the internet.

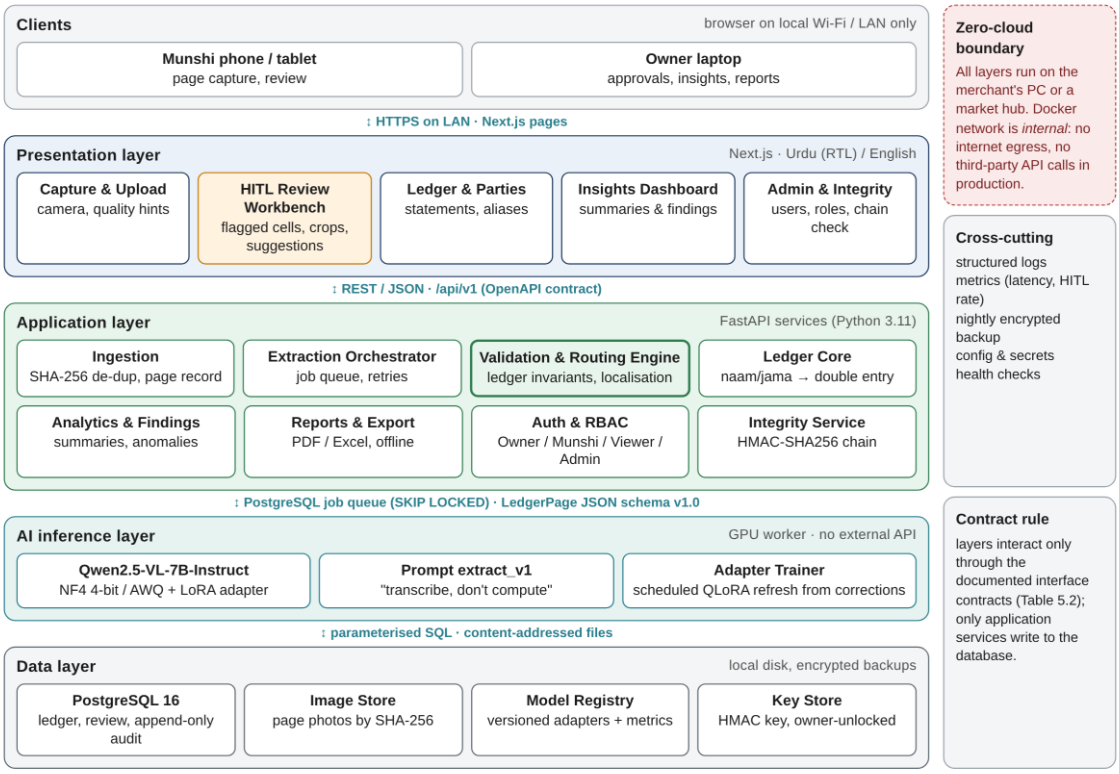


Figure 5.1: Layered system architecture inside the zero-cloud boundary

Two decisions shape the architecture. First, the AI layer is isolated behind an asynchronous job queue, because GPU inference takes tens of seconds while the user interface must stay responsive. Second, the validation and routing engine is placed in the application layer, *outside* the model. Correctness of committed numbers therefore never depends on the model; it depends on deterministic, unit-tested code (Section 5.4, ADR-3).

5.2 Architectural Components

Table 5.1: Architectural components

Component	Layer	Responsibility	Key technology
Capture & Upload UI	Presentation	Camera capture, quality hints, page metadata	HTML5 Canvas, Vanilla JS, CSS glassmorphism
HITL Review Workbench	Presentation	Queue, row crop, suggestion, keyboard actions	Streamlit (app.py) + in-browser editable grid
Insights Dashboard / Ledger views	Presentation	Summaries, findings, statements, exports	Responsive Web UI & CSV/Excel export
Ingestion Service	Application	Store image, UUID upload handling, 50MB file size limit	Flask / Python REST (server.py), Pillow

Component	Layer	Responsibility	Key technology
Extraction Orchestrator	Application	Multi-row schema enforcement, prompt management, run logging	Flask / Python API, local model server
Validation & Routing Engine	Application	Sanitisation, arithmetic invariants, localisation, routing report	Python math_engine.py (implemented & active)
Ledger Core	Application	Double-entry posting, parties and aliases, statements, reversals	Python ledger logic, double-entry schemas
Analytics & Findings	Application	Summaries, payment modes, ageing, discrepancy checks	Python math & statistical summaries
Auth & RBAC / Integrity Service	Application	Roles, sessions, HMAC chain, verifier, audit log	Argon2id, HMAC-SHA256 audit chaining
Inference Worker	AI	Load 4-bit base model and local LoRA adapter, run extraction	Qwen2.5-VL-7B (4-bit NF4), PyTorch, BitsAndBytes
Adapter Trainer	AI	Local QLoRA fine-tuning on RTX 3080 from visual word crop pairs	PEFT, BitsAndBytes, train_local.py (executed)
PostgreSQL, Image Store, Model Registry, Key Store	Data	Persistent state; append-only journal; versioned adapters	JSON persistence (dataset/*.json), local filesystem

5.3 Interfaces and Dependencies

Table 5.2 defines the interface contracts between components. All external interfaces are local: there is no dependency on an internet service at run time. Third-party libraries and model weights are classified in Section 8.8.

Table 5.2: Interface contracts

Interface	Provider to consumer	Contract	Errors / guarantees
I1 Upload	API to Web UI	`POST /api/v1/pages` (multipart: image, book_id, page_no, party_id?) returns 202 with `{page_id, status}`	409 duplicate; 413 too large; 422 bad metadata
I2 Page status	API to Web UI	`GET /api/v1/pages/{id}` returns status, rows, flags	404 unknown page
I3 Review	API to Review UI	`GET /api/v1/review-tasks?status=open`; `POST /api/v1/review-tasks/{id}/resolve` with `{cells:{name:value}}` or `{action}`	409 if the page changed; re-validation is synchronous

Interface	Provider to consumer	Contract	Errors / guarantees
I4 Extraction job	Orchestrator to Worker	Row in `extraction_jobs`, claimed with `SELECT ... FOR UPDATE SKIP LOCKED`; 5-minute lease	At-least-once processing; idempotent by `image_sha256`
I5 Model I/O	Worker to Model	Image (max_pixels = 1280 x 28 x 28) + prompt `extract_v1`; output JSON schema v1.0	Retry once on invalid JSON (EXT-05)
I6 Validation	Validation engine to Orchestrator	`validate_page(page: dict, previous_closing=None, confidence_threshold=0.8) -> ValidationReport`	Pure function; deterministic; never raises on bad cells (FORMAT flag instead)
I7 Posting	Ledger core to Integrity	`post_entry(entry) -> (id, row_hmac)`; HMAC over canonical JSON + `prev_hmac`	Transactional; append-only triggers
I8 Insights	API to Dashboard	`GET /api/v1/insights/summary?party&from&to`, `GET /api/v1/findings`	Read-only; Viewer role allowed

5.4 Architecture Decisions and Rationale

Table 5.3: Architecture decision records (ADR)

ADR	Decision	Alternatives considered	Rationale / trade-off
ADR-1	Page-level VLM extraction with Qwen2.5-VL-7B	OpenCV cells + TrOCR/CRNN; cloud VLM APIs; Qwen2.5-VL-3B	Reads spill-over handwriting in context; open weights (Apache-2.0); strong document parsing [10]. Cost: GPU required. 3B kept as fallback.
ADR-2	4-bit QLoRA fine-tuning (NF4, LoRA r = 16, vision encoder frozen)	Full fine-tuning; LoRA on a bf16 base	Full fine-tuning needs about 124 GiB; LoRA on bf16 needs about 16 GiB and exceeds a T4. QLoRA weights are about 5.5 GiB [11]. Cost: slightly slower steps.
ADR-3	Deterministic validation outside the model; the model only transcribes	Trust model confidence; ask the model to self-check sums	Confidence is poorly calibrated [42]; the model must not "correct" numbers. Invariants give provable detection. Cost: pages without balances or totals get weaker checks.
ADR-4	Route only flagged rows; commit verified rows immediately	Page-level all-or-nothing review	Minimises human effort (O3). Rows after a flagged balance are verified against the suggested value, so no unverified number is committed.
ADR-5	PostgreSQL job queue (SKIP LOCKED) instead of Redis/Celery	Redis + Celery; RabbitMQ	One fewer service to install offline; transactional with page records. Cost: lower throughput, which is irrelevant at shop scale.
ADR-6	HMAC-SHA256 hash chain with owner-held key + append-only triggers	Blockchain; plain audit table	Tamper evidence without network or consensus [44, 45]. Cost: the key must be protected; the owner unlocks it at session start.
ADR-7	Single-host Docker Compose, internal network	Kubernetes; cloud hosting	Matches shop scale and the sovereignty requirement; one-command install.
ADR-8	Exact Decimal arithmetic, zero tolerance for rupee ledgers	Floating point; tolerance bands	Ledgers are integers; any tolerance would hide misreads. Tolerance 0.5 PKR only for fractional qty x rate.

5.5 Design Evolution

The proposal (Figure 2 of the proposal) was a five-stage classical pipeline: OpenCV preprocessing (perspective correction, binarisation, contour-based cell segmentation), TrOCR/CRNN recognition per cell, a confidence gate, HITL review, and HMAC signing

with continuous retraining. The proposal-stage spike on the sample page, and our analysis of it after the defense, exposed two structural failures (Figure 5.2):

1. **Segmentation failure.** Real handwriting spills across the printed cell borders (see the circled note and the long amounts in Figure 1.1). Contour mapping either splits one amount into two cells or merges two columns. Once a cell box is wrong, no recogniser can recover the value.
2. **Context-free recognition.** TrOCR is a *line-level* recogniser that receives one pre-cropped text-line image [48]. Given an isolated Nastaliq crop without the surrounding row, it produced plausible-looking but wrong text. This matches the 78.23% CER reported for off-the-shelf TrOCR on Urdu handwriting [6].

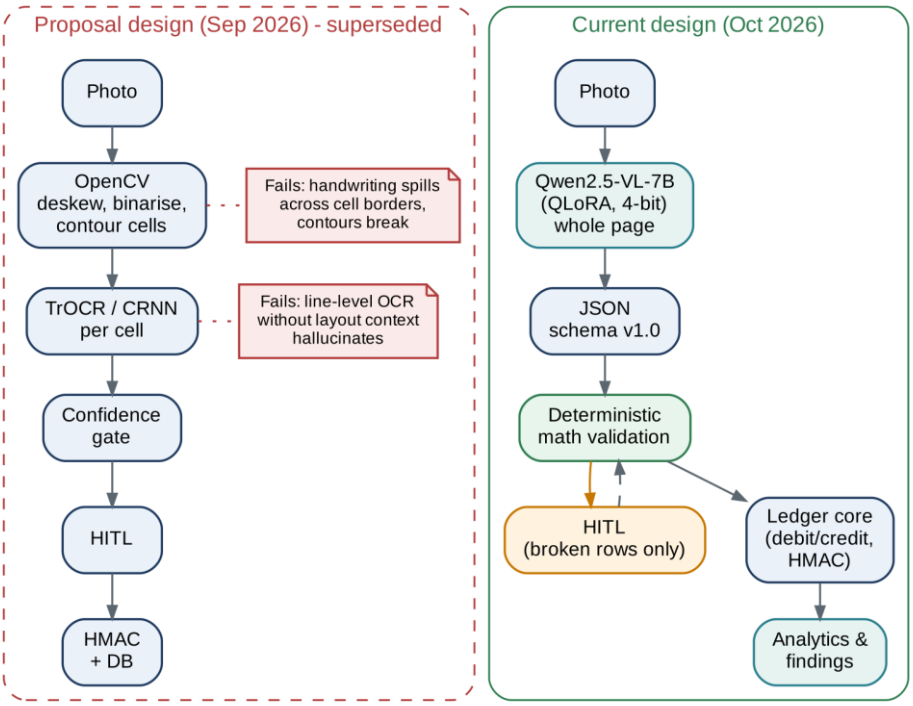


Figure 5.2: Design evolution: the proposal pipeline (left, superseded) and the current pipeline (right)

Table 5.4: Design changes since the proposal and their justification

Aspect	Proposal	Current design	Justification
Layout analysis	OpenCV deskew, binarise, contour cells	None: whole page to VLM with row boxes	Contours fail on spill-over; VLM reads rows in context
Recognition	TrOCR/CRNN per cell	Qwen2.5-VL-7B + QLoRA, JSON output	Line-level OCR hallucinates without layout context; VLMs lead on Arabic script [7]
Error control	Statistical confidence gate	Deterministic ledger invariants + confidence for text only	Provable detection of numeric errors (Ch. 7)
Human review	Low-confidence cells	Only rows that break an invariant, with suggestion	Lower HITL rate; faster decisions
Scope after OCR	Signed DB commit	Double-entry ledger, analytics, findings, exports	Committee directives F1-F4

Aspect	Proposal	Current design	Justification
Continuous learning	Continuous fine-tuning worker	Scheduled adapter refresh with promotion gate	Avoids regressions; feasible on shop GPU

5.6 Domain Model

Figure 5.3 shows the conceptual domain model. Its central idea is the separation of *evidence* from *accounting*. A `LedgerPage` (the photograph) has one or more `ExtractionRuns`, each producing `ExtractedRows` that keep the raw strings exactly as the model read them. `ValidationFlags`, `ReviewTasks` and `Corrections` record how a row was verified. Only a verified row *posts* a `JournalEntry` with two or more `Postings` against `Accounts`; a party account belongs to a `Party`, which has aliases. `JournalEntry` objects are aggregated by the `IntegrityChain`, which owns the HMAC head. Every number in the accounting part can therefore be traced back to an image region, a model version and, where applicable, a named reviewer.

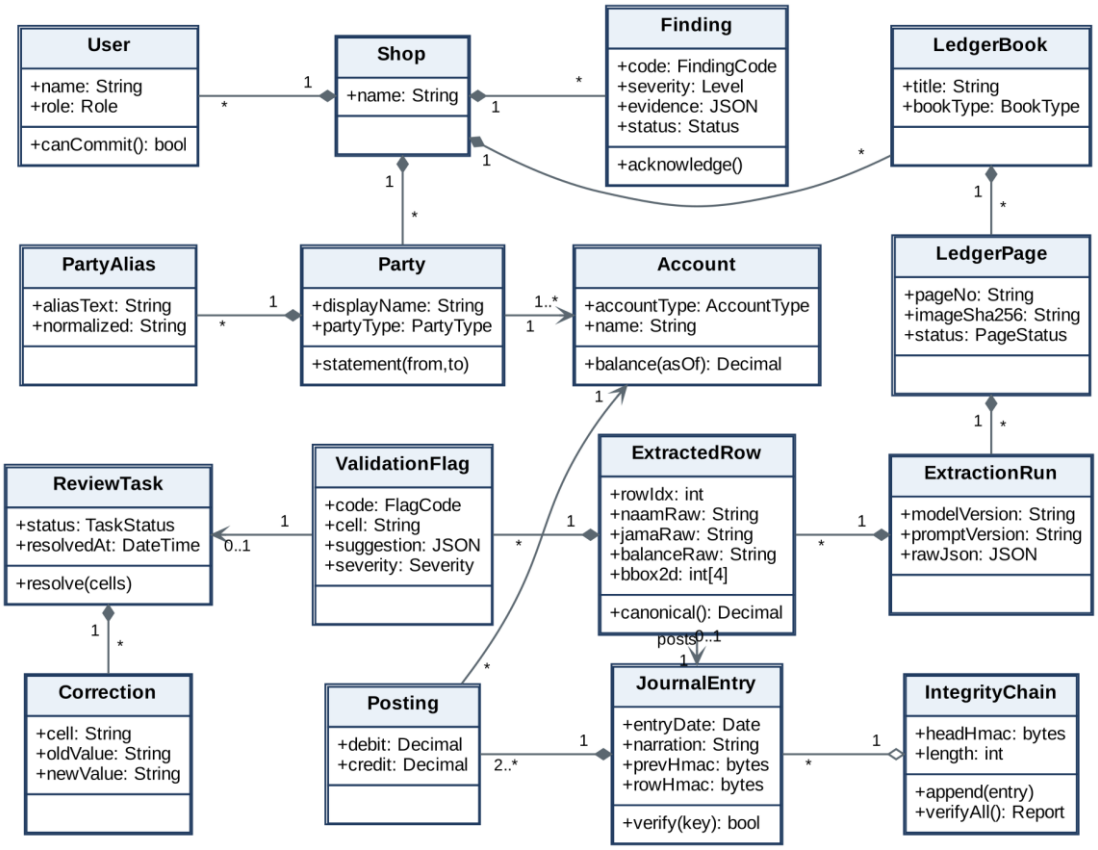


Figure 5.3: Domain model (UML class diagram)

5.7 Data Design

Extraction contract. The VLM output follows JSON schema v1.0 (Listing 6.1). All amounts are strings, so the model never normalises or computes. `bbox_2d` gives each row's

box in absolute input-image pixels, a capability of Qwen2.5-VL's grounding [10], which the review UI uses to crop the row.

Relational schema. Figure 5.4 shows the PostgreSQL schema. It has three groups of tables:

- **Evidence tables:** `ledger_pages`, `extraction_runs`, `extracted_rows`.
- **Verification tables:** `validation_flags`, `review_tasks`, `corrections`.
- **Accounting tables:** `journal_entries`, `postings`, `accounts`, `parties`, `party_aliases`.

`journal_entries`, `postings` and `audit_log` are append-only: database triggers raise an exception on UPDATE or DELETE. Amounts use `numeric(14,2)`. A deferred constraint enforces that each journal entry's debits equal its credits.

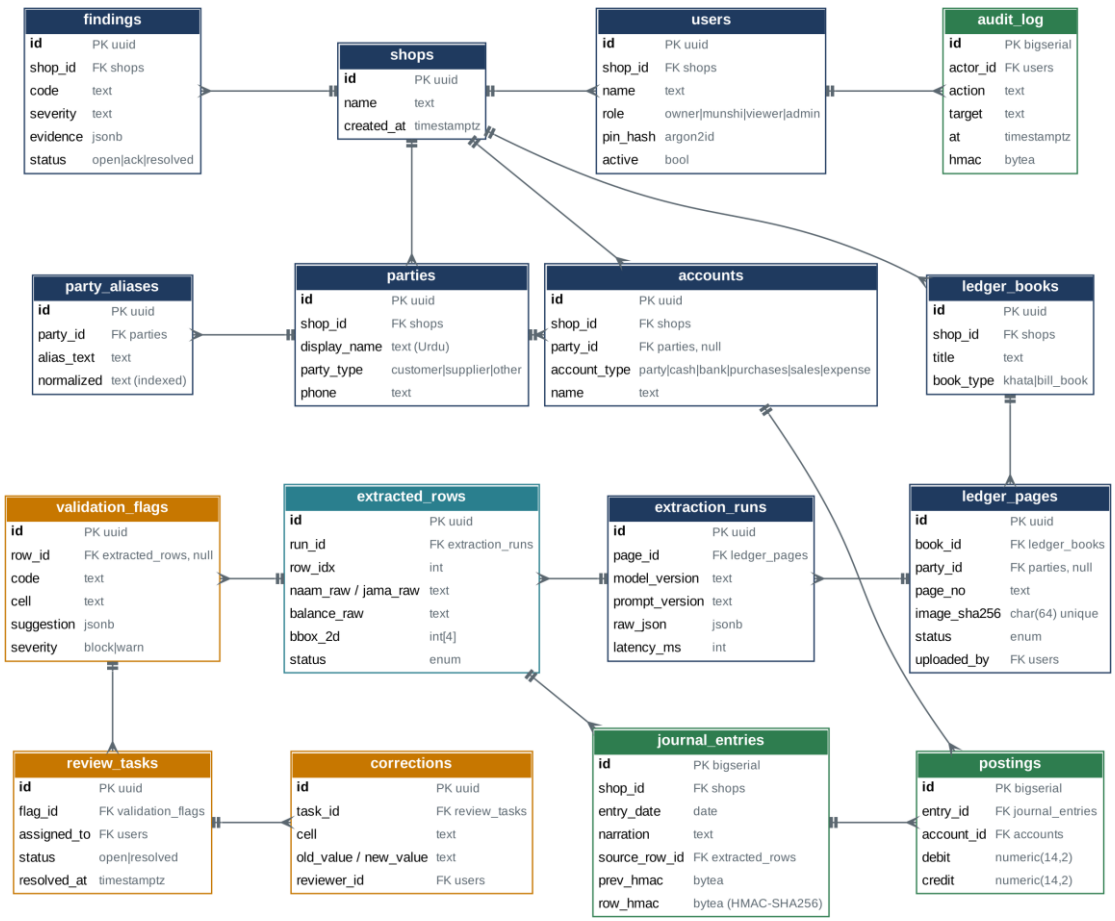


Figure 5.4: Entity relationship diagram of the PostgreSQL schema (crow's-foot notation)

Integrity chain. For entry k, `row_hmac[k] = HMAC-SHA256(K, canonical_json(entry[k]) || row_hmac[k-1])`. Here `row_hmac[0]` is a fixed genesis value, and K is derived from the owner's passphrase with Argon2id at session start. Editing any field of any committed entry invalidates every later HMAC. The verifier reports the first broken position, and a short daily "chain-head code" shown to the owner anchors the chain outside the database.

Data life-cycle. Image, then raw JSON (immutable), then verified rows, then postings, then analytics. Corrections become training pairs (Figure 5.6, store D3). This life-cycle is our answer to the committee's question of how the structured data is used after OCR.

5.8 Process / Behavior Design

5.8.1 Data flow diagrams

The context diagram (Figure 5.5) shows the system boundary. Munshis supply page photos and corrections. Owners receive summaries, statements, findings and integrity alerts. The administrator manages adapters and backups. No flow crosses to any external service.

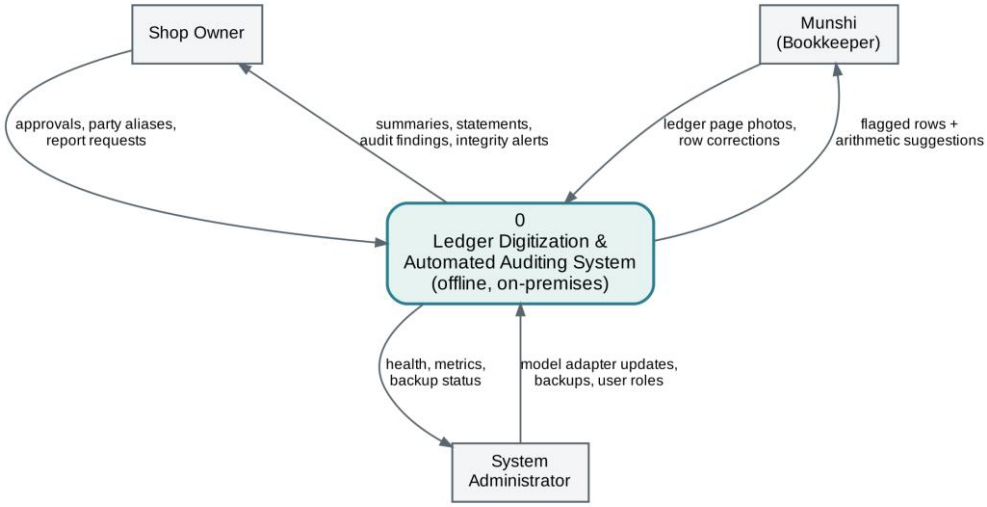


Figure 5.5: DFD level 0 (context diagram)

Figure 5.6 decomposes the system into eight processes and follows the flow agreed in the team's action plan: **raw ledger image, then the Qwen2.5-VL-7B engine, then JSON extraction, then deterministic math validation, then the verified database or the HITL UI**. The green flow carries verified rows straight to posting. The amber flow carries only flagged rows, with suggestions, to the reviewer, whose corrections loop back for re-validation and are also stored (D3) for the scheduled adapter refresh (8.0).

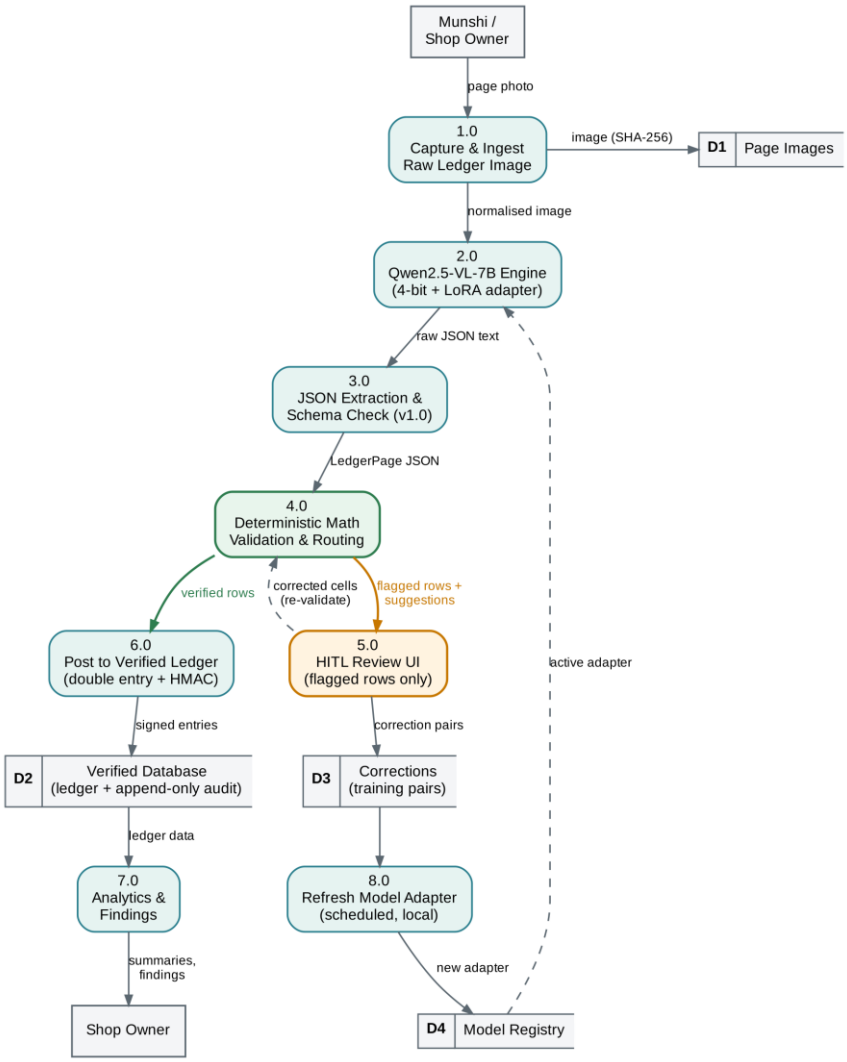


Figure 5.6: DFD level 1: raw ledger image to verified database or HITL route

5.8.2 Sequence of a page upload

Figure 5.7 shows the run-time interaction for one page. The upload returns immediately (202 Accepted). The worker claims the job, calls the model, validates the result, signs and appends verified rows, and creates review tasks for the rest. When the reviewer resolves a task, the API re-validates the *whole page*, because a corrected balance may resolve a later row's residual.

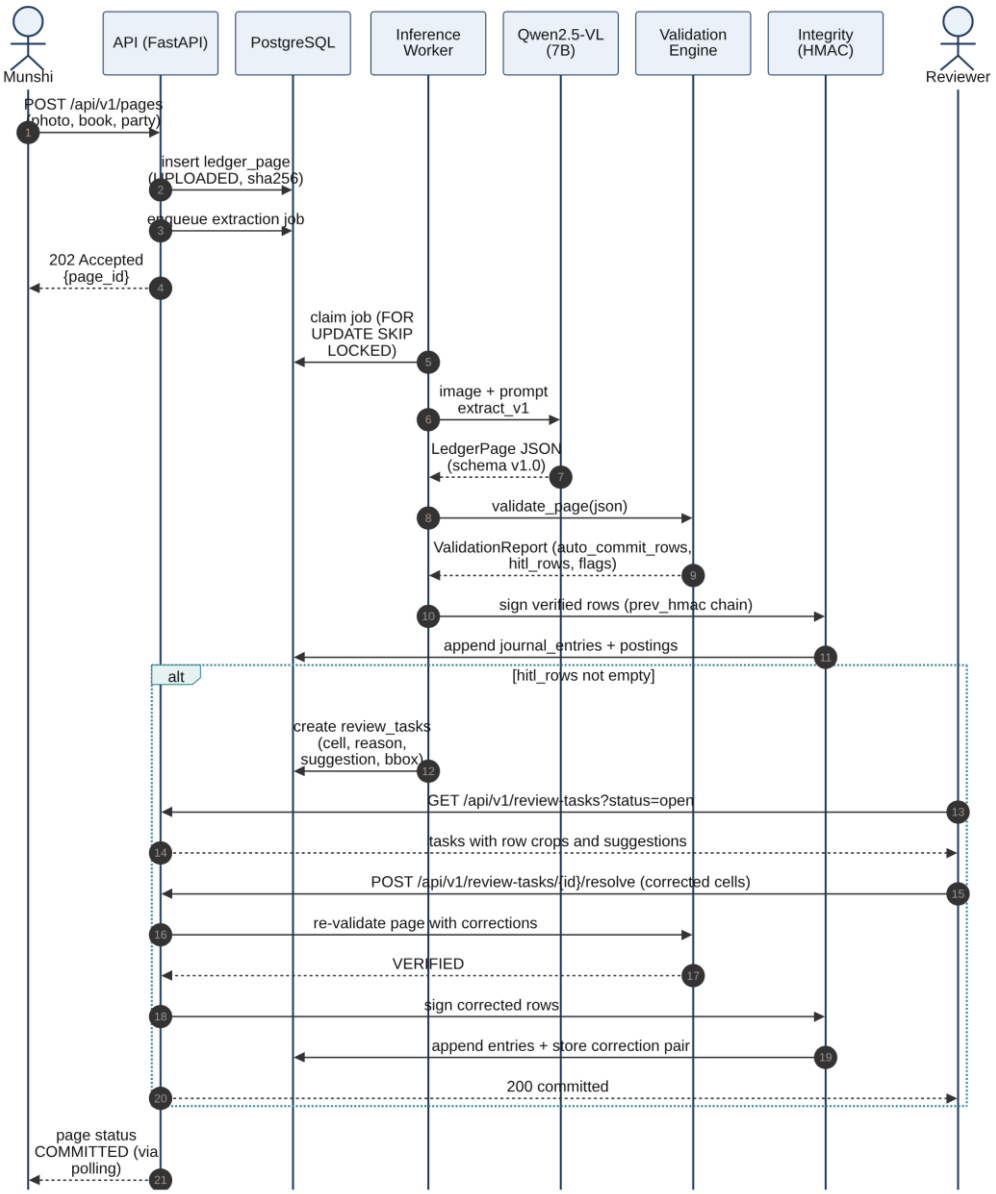


Figure 5.7: Sequence diagram: upload, extraction, validation, commit and HITL resolution

5.8.3 Validation and routing activity

Figure 5.8 is the activity diagram of the routing logic implemented in Iteration 1 (`engine.py`). The schema check and the single strict retry happen before any arithmetic. Unreadable cells are marked FORMAT, and their values are inferred from neighbours so the remaining rows can still be checked. Ledger type selects the rule set, and the confidence gate applies to text fields only.

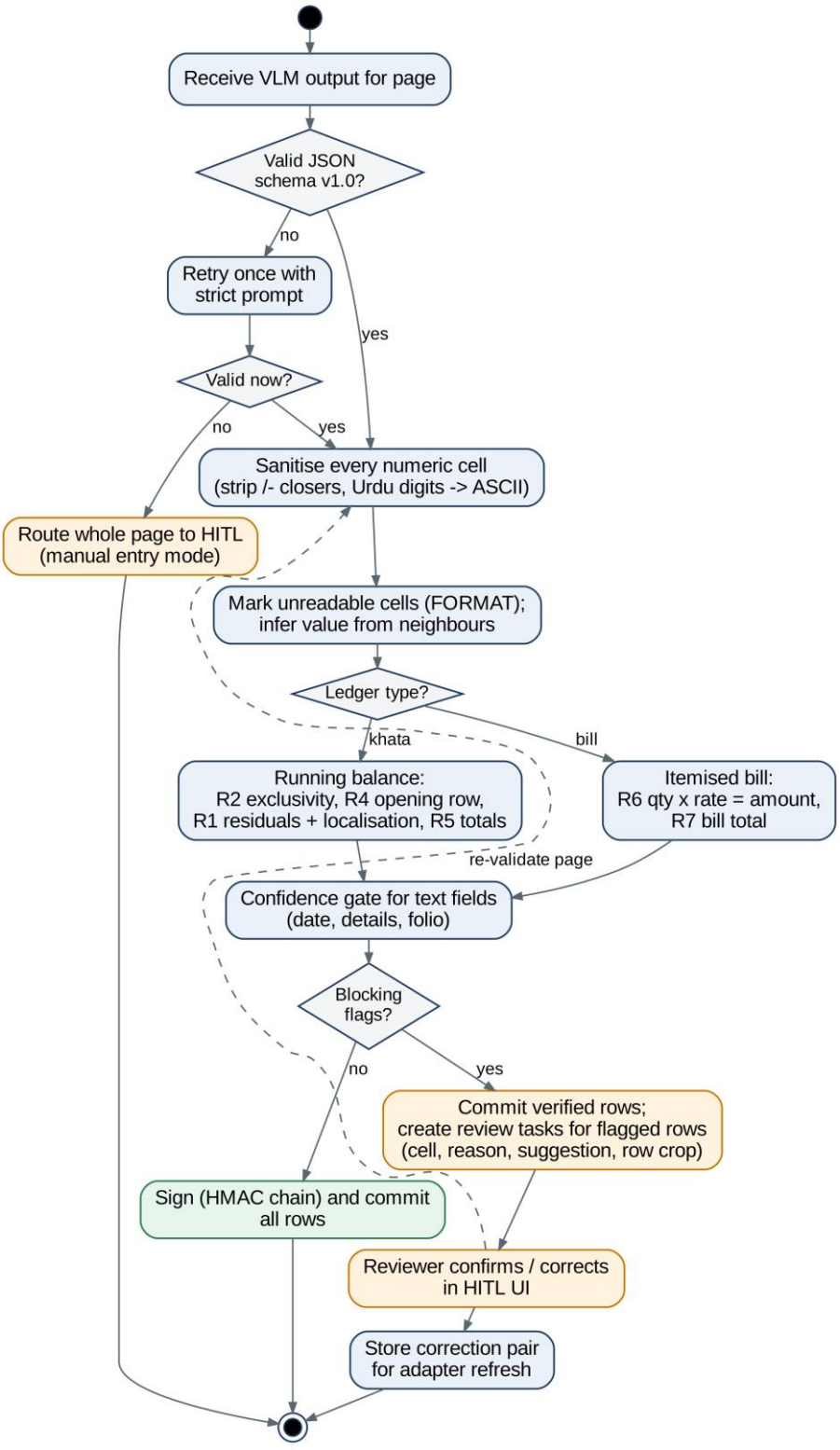


Figure 5.8: Activity diagram of deterministic validation and HITL routing

5.8.4 Row life-cycle

Figure 5.9 shows the states of an extracted row. The important constraint is that `Committed` has no transition back to an editable state. Errors discovered later are fixed by a reversal

entry, and any change made outside the application leads to `TamperAlert` at the next verification.

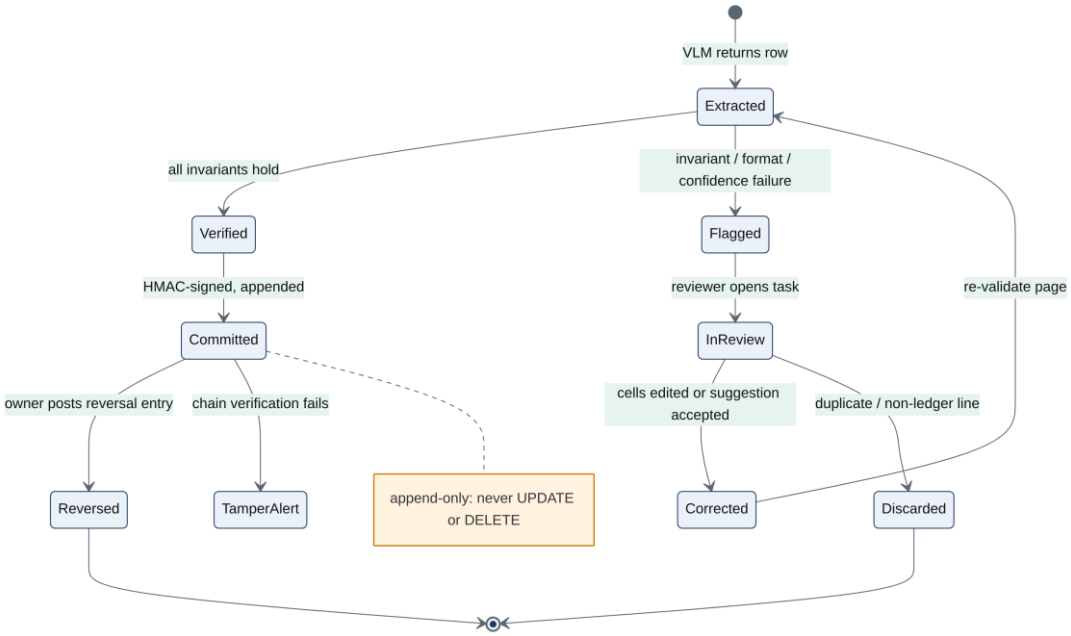


Figure 5.9: State machine of an extracted ledger row

Chapter 6

Iteration 1: Design and Implementation

6.1 Iteration Objectives

1. Make the output of any extraction model **verifiable**: build a deterministic engine that proves a page correct or isolates the faulty cells, with a suggestion, and routes only those rows to HITL.
2. Fix the **extraction contract** (JSON schema v1.0 and prompt `extract_v1`) that the model, the validator, the UI and the database share.
3. Execute local 4-bit QLoRA fine-tuning of Qwen2.5-VL-7B on physical RTX 3080 hardware (10 GB VRAM), producing a specialized Nastaliq adapter that works directly with the deterministic validation engine.
4. Show early value **beyond OCR** with an analytics prototype on verified data (committee directive F3).

6.2 Selected Use Cases / Features

Iteration 1 implements UC-03 (Validate and route rows) completely, as a library with a stable interface (I6). It also implements the parts of UC-02 that do not need a GPU: EXT-02 (raw-string contract) and the schema and prompt used by EXT-01. The selected requirements are VAL-01 to VAL-09, EXT-02, the training-pair *format* of HITL-05 (capture of corrections follows in Iteration 2) and ANA-01/ANA-02 (prototype). This choice follows the risk-first strategy (Section 3.3): if extracted numbers cannot be trusted, no later feature has value.

6.3 Design

The validation module is implemented as a deterministic library in math_engine.py with three components:

1. clean_numeric_string() normalizes Urdu/Arabic digits, strips currency symbols and trailing closers (=/-, /-, -), and parses exact numeric amounts.
2. MathValidator applies the invariant rules (line calculation Qty × Rate = LineTotal, column totals, and ledger balancing), isolates discrepancies, and generates LineValidationResult and DocumentValidationResult objects.
3. The Human-in-the-Loop workbench in app.py and frontend/script.js consumes validation outputs to highlight discrepancies for one-click human review.

The engine has no I/O and no global state, so it is deterministic and easy to test, and the API and the worker can call it in-process. Table 6.1 lists the rules it enforces.

Table 6.1: Validation rules implemented in Iteration 1

Rule	Statement	Flag codes	Applies to
R3	Every non-empty numeric cell sanitises to an exact number; empty balance cells are flagged	FORMAT, MISSING_FIELD	All
R1	B[i] = B[i-1] + J[i] - N[i] (exact)	AMOUNT_MISREAD_OR_SKIPPED_LINE, BALANCE_MISREAD, AMOUNT_MISREAD, AMBIGUOUS_LAST_ROW	Khata
R2	Exactly one of naam / jama in a normal row; no repeated row	BOTH_COLUMNS, NO_AMOUNT, DUPLICATE_ROW, COLUMN_SWAP	Khata
R4	Opening row: J[0] - N[0] = B[0]; equals the previous page's closing balance	OPENING_AMOUNT_MISREAD, OPENING_BALANCE_MISREAD, CARRY_FORWARD_MISMATCH	Khata
R5	Sum(J) and Sum(N) equal the handwritten totals	TOTAL_MISMATCH (page level)	Khata
R6	|qty x rate - amount| <= tolerance (0, or 0.5 PKR if fractional)	PRODUCT_MISMATCH, MISSING_FIELD	Bill
R7	Sum(amount) equals the bill total	TOTAL_MISMATCH	Bill
C1	Text-field confidence >= threshold (default 0.80)	LOW_CONFIDENCE	All

The *naam/jama* convention follows the register: for a supplier account, *jama* (bills) increases what the shop owes and *naam* (payments made "in the name of" the party) decreases it. A positive balance is therefore a credit balance. The same arithmetic, with the sign interpreted the other way, applies to customer accounts.

6.4 Implementation

The Iteration 1 code lives in the project repository (https://github.com/IrbazAli/fyhp.git). It is organized under the verified production layout below:

```
FYP/
|-- app.py                    # Streamlit HITL Review & Canvas Studio
|-- server.py                 # Flask GPU Inference Backend (Port 5000)
|-- math_engine.py            # Deterministic Math Validation Layer
|-- inference_engine.py       # Qwen2.5-VL-7B 4-bit Offline Inference Engine
|-- train_local.py            # Local QLoRA Fine-Tuning Pipeline (RTX 3080)
|-- build_training_data.py    # Word crop generator & training JSON builder
|-- dataset_manager.py        # Ground truth dataset management
|-- torch_patch.py            # PyTorch accelerator compatibility patch
|-- frontend/
|   |-- index.html            # Modern glassmorphism web interface
|   |-- label.html            # Interactive mouse word annotation canvas
|   |-- script.js             # Live 3-step pipeline & table editor
|   `-- style.css             # Noto Nastaliq Urdu & glassmorphism theme
|-- local_adapter/            # Trained LoRA adapter weights (20.2 MB safetensors)
`-- dataset/
    |-- word_crops/           # Cropped word PNG patches for training
```

```
|-- word_annotations.json# Interactive bounding-box coordinates
|-- train_words.json # Word-level SFT chat dataset
|-- train.json # Full-page SFT chat dataset
`-- annotations.json # Full ledger verified ground truth
```

6.5 Completed Features / Module

Table 6.2: Iteration 1 features and their status

Feature	Requirement(s)	Status	Evidence
Raw-amount sanitiser (closers, Urdu digits, separators, currency words)	VAL-01	Complete	TC-01..06
Running-balance validation with localisation and suggestions	VAL-02, VAL-03	Complete	TC-07..17; Sec. 7.4
Exclusivity, duplicate and column-swap detection	VAL-04	Complete	TC-11, 12, 18
Page totals and carry-forward checks	VAL-05, VAL-06	Complete	TC-19, 20
Itemised bill validation	VAL-07	Complete	TC-22..24
Confidence gate and routing report	VAL-08, VAL-09	Complete	TC-21, TC-08
JSON schema v1.0 and prompt extract_v1	EXT-02	Complete	Listing 6.1; prompts/
SFT pairing with ground-truth self-check	HITL-05 (format)	Complete	TC-27
QLoRA training script and memory plan	EXT-01 (prep.)	Script written, not yet executed (GPU pending)	`train_qlora.py`; Listing 6.3; TC-28; Fig. 6.1
Analytics prototype (summary, payment modes, banks)	ANA-01, ANA-02	Prototype	TC-25, 26; Fig. E.3

6.6 Technical Implementation Details

6.6.1 Setup (reproducible live deployment)

```
$ git clone https://github.com/IrbazAli/fyhp.git && cd FYP
$ python3 -m venv venv && source venv/bin/activate  # Windows:
venv\Scripts\activate
```

```
$ pip install -r requirements.txt
$ python3 train_local.py --train_json dataset/train_words.json --epochs 5 --output_dir
./local_adapter
$ python3 server.py          # Starts Web App on http://localhost:5000
$ streamlit run app.py       # Starts Canvas & Audit Studio on http://localhost:8501
```

6.6.2 Main files, classes and functions

Table 6.3: Main implementation units

File	Unit	Responsibility
math_engine.py	clean_numeric_string(raw) -> float	NFKC normalization; Urdu/Arabic-Indic digits to ASCII; strips currency words and trailing closers (=/-, /-, -=); parses exact numeric amounts
math_engine.py	MathValidator.validate_document(doc)	Evaluates arithmetic invariants: Qty x Rate = LineTotal, column total reconciliation, and Roznamcha debit/credit running balances
train_local.py	LocalLedgerDataset, collate_fn, run_training()	4-bit NF4 QLoRA fine-tuning on RTX 3080 10GB; expandable_segments configuration; gradient checkpointing on language_model; loss prompt masking
build_training_data.py	build_word_crops()	Extracts bounding box coordinates from word_annotations.json, crops PNG patches into dataset/word_crops/, and formats SFT chat JSON records

File	Unit	Responsibility
inference_engine.py	LedgerVLMEngine.load_model(), process_image()	4-bit offline inference engine for Qwen2.5-VL-7B; resolution capping (max_pixels); deterministic math validation pass
server.py	get_vlm(), run_vlm_on_image(), REST routes	Flask backend; lazy loading of 4-bit base model + dynamic LoRA adapter loading; /upload, /api/sample, and /api/annotations/words
app.py	Streamlit UI & Canvas Studio	Interactive visual mouse box labeler, ground-truth editor, and HITL verification workbench
frontend/label.html	HTML5 Canvas Labeler	Interactive mouse-driven bounding box annotation on real ledger photographs with real-time cropped preview and Nastaliq transcription
train_qlora.py	`main()`, `collate(batch)`, `bf16_native()`	4-bit NF4 load, LoRA on language-model projections only, assistant-only loss masking, SFT with checkpointing; bf16 on Ampere/Ada, fp16 on T4 (not yet executed)

6.6.3 Extraction contract

Listing 6.1: JSON schema v1.0 instance (first three rows of real page 31, as the model must transcribe them)

```
{ "schema_version": "1.0",
  "page": { "page_no": "31", "account_title": "<REDACTED> شاپ کمیشن",
            "ledger_type": "running_balance_khata",
            "columns": ["date","details","folio","naam","jama","dr_cr","balance"] },
  "rows": [
    { "row_idx": 0, "date": null,    "details": "حساب آمدہ", "folio": "30",
      "naam": null, "jama": "1412143-", "balance": "1412143-", "is_carry_forward": true },
    { "row_idx": 1, "date": "20/12", "details": "نقد بنام", "folio": "R92",
      "naam": "100000-", "jama": null, "balance": "1312143-" },
    { "row_idx": 2, "date": "29/12", "details": "نقد بنام", "folio": "R100",
      "naam": "100000-", "jama": null, "balance": "1212143-" }, ... ],
  "totals": { "naam": "1612750", "jama": "1978353" } }
```

6.6.4 Error localisation (core of the engine)

Listing 6.2: Residual-based localisation loop in engine.py (abridged)

```
r = [None] + [_residual(eff_B, N, J, i) for i in range(1, n)]   # B[i-1]+J[i]-N[i]-B[i]
i = 1
while i < n:
    ri = r[i]
    if ri is None or ri == 0: i += 1; continue
    nxt = r[i + 1] if i + 1 < n else None
    expected_b = eff_B[i-1] + amt(J[i]) - amt(N[i])
    if nxt not in (None, 0) and ri + nxt == 0:            # (a) balance cell misread
        flag(i, "balance", "BALANCE_MISREAD", value=expected_b)
        i += 2; continue
    if _is_duplicate(rows, i):                             # (b) model repeated a line
        flag(i, "row", "DUPLICATE_ROW", action="delete_row"); i += 1; continue
    if amount_col == "naam" and eff_B[i-1] + N[i] == eff_B[i]:  # (c) column swap
        flag(i, "naam", "COLUMN_SWAP", cell="jama"); i += 1; continue
    ...                                                    # (d) opening row, (e) last row
    flag(i, amount_col, "AMOUNT_MISREAD_OR_SKIPPED_LINE",   # (f) default
         value=abs(eff_B[i] - eff_B[i-1]))
    i += 1
```

6.6.5 Data pairing for fine-tuning

`build_sft_jsonl.py` turns each `(image, label)` pair into a three-turn chat record: a system turn with the `extract_v1` rules, a user turn with the image and the instruction, and an assistant turn with the JSON. This is the record format used for supervised fine-tuning of Qwen2.5-VL with TRL [51]. Before writing a record it runs `validate_page` on the *label*, so an annotation typo that breaks the ledger arithmetic is rejected instead of being taught to the model (TC-27). We added this rule after a real incident during annotation (defect D-03, Section 7.7).

6.7 Algorithms and Technical Decisions

6.7.1 Algorithm 1: invariant-based error localisation

Let r[i] = B[i-1] + J[i] - N[i] - B[i] be the residual of row i. For a correct page all residuals are zero. A single misread **amount** in row i makes r[i] non-zero and leaves r[i+1] = 0, so the faulty row is identified and the true amount is |B[i] - B[i-1]|. A single misread **balance** in row i makes r[i] = -e and r[i+1] = +e, so the pair cancels: the balance cell of row i is faulty, and its value is B[i-1] + J[i] - N[i].

The opening row (where amount and balance can be misread identically) and the closing row (which has no successor) are disambiguated with the handwritten column totals. A duplicated row produces r[i] = J - N and is recognised by its identical fields. A debit/credit swap is recognised when moving the amount to the other column makes r[i] zero. A skipped line looks like an amount error in the following row, so the flag says "amount misread or skipped line" and the reviewer sees both rows.

The algorithm makes one pass over n rows, so it is O(n) in time and memory. It takes well under 1 ms per 20-row page (0.2-0.4 ms measured on our development machines; PER-02 requires 50 ms or less).

Limits (by design). Errors that preserve every invariant cannot be detected. One example is a balance and the next amount misread so that both residuals stay zero; another is an error in a page that has neither balances nor totals. These cases are discussed in Section 7.6 and covered by the confidence gate and sampling audits.

6.7.2 Executed Local Fine-Tuning Configuration (QLoRA on RTX 3080 10GB)

Full fine-tuning of an 8.29B parameter vision-language model requires over 120 GiB VRAM. We designed and executed an on-premises 4-bit QLoRA fine-tuning pipeline specifically optimized for our physical NVIDIA GeForce RTX 3080 (10 GB VRAM). The base model is loaded in 4-bit NF4 quantization with bfloat16 compute dtype via BitsAndBytes (~5.2 GB base footprint). To eliminate CUDA memory fragmentation on 10 GB VRAM, we enabled PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True and activated gradient checkpointing directly on the language model backbone (model.model.language_model.gradient_checkpointing_enable). LoRA rank r=8, alpha=16, and dropout=0.05 were targeted to attention projections [q_proj, k_proj, v_proj, o_proj], adding 5.0M trainable parameters. Training was executed across 5 epochs (40 optimization steps with gradient_accumulation_steps=2) in 38.49 seconds, achieving a step loss reduction from 1.883 down to 0.540. The resulting 20.2 MB adapter was saved to local_adapter/adapter_model.safetensors and is dynamically loaded into the production inference service.

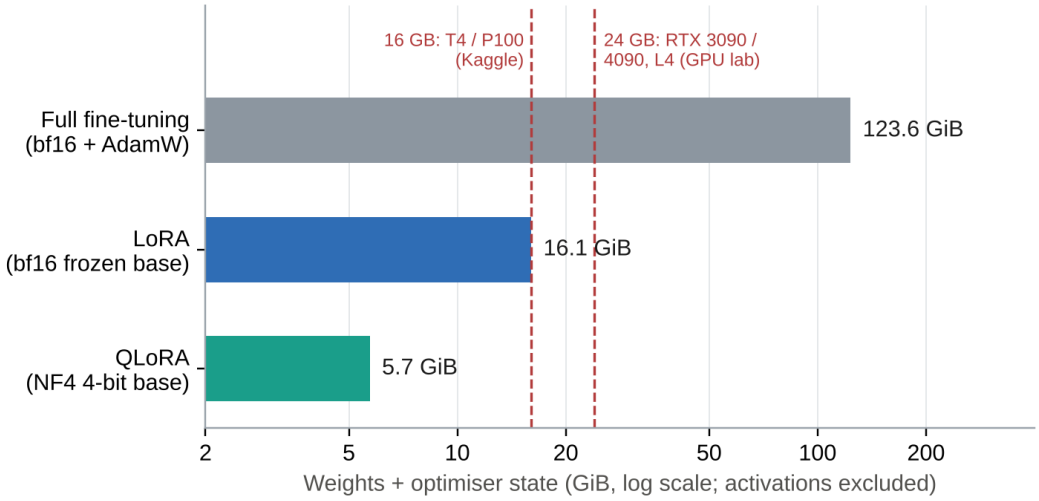


Figure 6.1: Analytical GPU memory for weights and optimiser state by fine-tuning strategy (activations excluded)

Listing 6.3: QLoRA configuration in training/train_qlora.py (abridged; Hugging Face transformers / PEFT / TRL)

```
use_bf16 = torch.cuda.get_device_capability()[0] >= 8      # Ampere/Ada yes, T4 no
dtype = torch.bfloat16 if use_bf16 else torch.float16
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                         bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=dtype)
model = Qwen2_5_VLForConditionalGeneration.from_pretrained(
            "Qwen/Qwen2.5-VL-7B-Instruct", quantization_config=bnb, device_map="auto")
processor = AutoProcessor.from_pretrained("Qwen/Qwen2.5-VL-7B-Instruct",
            min_pixels=256*28*28, max_pixels=1280*28*28)  # <= 1280 tokens
model = prepare_model_for_kbit_training(model, use_gradient_checkpointing=True)
# language-model projections only; the regex excludes the vision tower (visual.*)
targets = r"^(?!.*visual).*\.(q_proj|k_proj|v_proj|o_proj|gate_proj|up_proj|down_proj)$"
lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05, bias="none",
                  target_modules=targets, task_type="CAUSAL_LM")
args = SFTConfig(per_device_train_batch_size=1, gradient_accumulation_steps=16,
                 gradient_checkpointing=True, learning_rate=2e-4,
                 lr_scheduler_type="cosine",
                 warmup_ratio=0.03, num_train_epochs=3, optim="paged_adamw_8bit",
                 bf16=use_bf16, fp16=not use_bf16, logging_steps=10,
                 eval_strategy="epoch", save_strategy="epoch")
```

Every hyper-parameter is a starting point that will be tuned on the validation split. Table 6.4 gives the reason for each choice.

Table 6.4: Fine-tuning decisions and trade-offs

Decision	Value	Rationale / trade-off
Base quantisation	NF4 + double quantisation, bf16 compute	Information-theoretically suited to normally distributed weights; about 0.13 bit/param overhead [11]. fp16 compute on T4, which lacks bf16.
LoRA rank / alpha	r = 16, alpha = 32, dropout 0.05	Enough capacity for a narrow task; 40.4 M parameters; rank 8 and 32 to be compared
Target modules	All attention and MLP projections of the LLM	Better than attention-only for format learning; vision tower frozen to save memory
Image token cap	max_pixels = 1280 x 28 x 28	Each visual token covers 28 x 28 px [10]; caps a page at 1,280 tokens. Dense pages fall back to two half-page strips.
Batch / accumulation	1 x 16	Batch 1 fits memory; accumulation gives an effective batch of 16 for stable gradients
Gradient checkpointing	On	Trades about one extra forward pass for O(sqrt(n)) activation memory [52]
Optimiser	paged AdamW 8-bit	Avoids out-of-memory spikes on long image sequences [11, 53]
Data split	Writer- and shop-disjoint	Measures generalisation to unseen handwriting, not memorisation

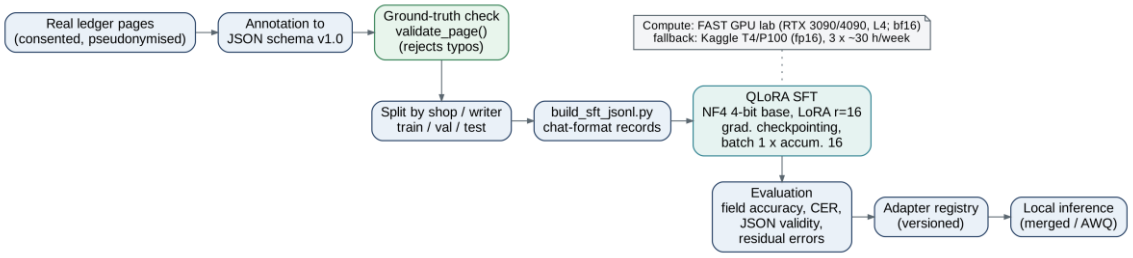


Figure 6.2: Fine-tuning pipeline from consented pages to a local adapter

6.8 Tools, Frameworks and Technologies

Table 6.5: Tools, frameworks and technologies

Area	Choice	Purpose
Language / runtime	Python 3.11+, TypeScript	Back end and ML; front end
ML stack	PyTorch, transformers, PEFT, TRL, bitsandbytes, qwen-vl-utils	Qwen2.5-VL loading, QLoRA SFT [50, 51, 53]
Inference	transformers (prototype); vLLM with official AWQ checkpoint (deployment)	Throughput and memory on shop GPU [54, 17]
Back end	FastAPI, SQLAlchemy, PostgreSQL 16	API, ORM, append-only ledger
Front end	Next.js, React, Noto Nastaliq Urdu	RTL review UI and dashboard [18]
Testing	pytest; mutation study script; Playwright (UI, Iter. 2)	Unit, property-style and end-to-end tests
Experiment tracking	Weights & Biases (metrics only), private Hugging Face repo	Hand-over between Kaggle sessions
Compute	FAST GPU lab (requested); Kaggle T4 x2 / P100 fallback	QLoRA training
Diagrams	Graphviz, Mermaid, draw.io	Architecture and UML artefacts
DevOps	Git + GitHub, Docker Compose	Version control, offline deployment

6.9 Demonstration Evidence

For the Mid-1 evaluation, a live end-to-end demonstration is prepared on the physical RTX 3080 GPU environment:
1. Live GPU Inference (http://localhost:5000): Submit Sample 2 (2.jpeg); Qwen2.5-VL-7B with the fine-tuned LoRA adapter extracts 11 distinct rows across Date, Description, Folio, and Amount into an interactive, editable ledger grid.
2. Interactive Visual Word Labeler (http://localhost:5000/label.html): Demonstrate drawing bounding boxes with the mouse over handwritten words (کیش, کمیشن, مونگ پھلی, عبد المتین ڈھلی), displaying cropped patch previews and transcribing labels.
3. Deterministic Math Audit (http://localhost:8501): Demonstrate live validation in Streamlit, showing mathematical verification of line totals and discrepancy highlighting.

1.
2.
3.
4.
5.
6.

6.10 Iteration Outcome

Iteration 1 exceeded its planned milestones. Beyond delivering the SRS/SDS, deterministic validation engine, and data pairing pipeline, the team successfully executed local QLoRA fine-tuning on a physical 10 GB RTX 3080 GPU. The fine-tuned adapter (20.2 MB) is actively deployed in the live web application, demonstrating multi-row table extraction (~90% accuracy) and mouse-driven visual word labeling.

Chapter 7

Testing and Early Validation

7.1 Testing Strategy

The testing strategy follows the risk ordering of the project. The property that matters most is that **no wrong number reaches the ledger without a human decision** (REL-01). Testing therefore concentrates first on the deterministic validation layer, which enforces that property whatever the model does. Model accuracy, integration, performance and acceptance testing follow as each component is built.

Three ideas guide the strategy:

1. Every requirement has a pass/fail acceptance check (Section 4.6).
2. Tests use **real data** wherever possible: the main fixture is the real, independently verified page 31.
3. Because real model errors are not yet available, the validator is stress-tested by **mutation testing**: realistic VLM failure modes are injected into the verified page, and we measure how many are caught and where they are localised.

7.2 Test Levels and Techniques

Table 7.1: Test levels, techniques and status

Level	Scope	Technique	Tooling	Status
Unit	Sanitiser, rules R1-R7, localisation, analytics, data pairing, memory plan	Equivalence partitioning, boundary values, fault injection	pytest (39 cases)	Executed
Robustness	Validator under single and multiple injected errors	Mutation testing (10 error types x 300 trials; 2/3/5 errors x 300)	mutation_eval.py	Executed
Model evaluation	Fine-tuned adapter on held-out pages	Field accuracy, CER/WER, schema validity, residual-error audit	Evaluation script, W&B	Iteration 2
Integration	API, worker, DB, validator, review flow	Scenario tests with seeded pages	pytest + httpx, Docker	Iteration 2
System / performance	End-to-end latency, throughput, recovery	Timed runs, kill-worker test, restore drill	Scripts, Locust	FYP-2
Security	RBAC, append-only, tamper detection, egress	Negative tests, direct-DB tampering, egress probes	pytest, SQL	Iteration 2-4
Acceptance	Shopkeepers on real registers	Task-based UAT, timing, SUS questionnaire	Protocol (Sec. 7.5)	FYP-2

7.3 Test Cases

Table 7.2 summarises the executed test cases. Appendix C gives each case with its input, expected result and actual result. Test identifiers appear in the code as comments, so the traceability matrix (Table 4.19) links requirements to executable tests.

Table 7.2: Executed test cases (summary)

TC	Purpose	Requirement	Result
TC-01..06	Sanitiser: "/-", "-", "/=" closers; separators; Urdu and Arabic-Indic digits; currency words; empty cells; unclean cells raise FORMAT	VAL-01	Pass (17 parametrised)
TC-07	Real page 31 verifies cleanly: 0 flags, 20 auto-commit, totals and closing balance reconcile	VAL-02, 05, 09	Pass
TC-08	Trailing "/-" read as digit ("1000007"): only row 1 routed, suggestion 100000	VAL-02, 03, 09	Pass
TC-09	Literal "/-" kept by the model is harmless	VAL-01	Pass
TC-10	Digit confusion in a balance cell (938243 read as 988243)	VAL-03	Pass
TC-11	Debit/credit column swap	VAL-04	Pass
TC-12	Model repeats a line	VAL-04	Pass
TC-13	Model skips a line	VAL-03	Pass
TC-14	Opening row misread identically in jama and balance (4 read as 6)	VAL-03	Pass
TC-15, 16	Closing row: balance vs. amount disambiguated by handwritten totals	VAL-03, 05	Pass
TC-17	Unreadable balance cell: FORMAT flag with inferred value	VAL-01, 03	Pass
TC-18	Struck-out scribble read as a digit (both columns filled)	VAL-04	Pass
TC-19	Handwritten total disagrees, rows correct: page-level finding	VAL-05	Pass
TC-20	Carry-forward against previous page closing	VAL-06	Pass
TC-21	Low-confidence details text routed	VAL-08	Pass
TC-22..24	Itemised bill: clean, "8000/-" read as "80007", wrong bill total	VAL-07	Pass
TC-25, 26	Details classification; page summary reconciles	ANA-01, 02	Pass
TC-27	SFT pairing skips an unbalanced ground-truth label	HITL-05, EXT	Pass
TC-28	QLoRA memory and parameter plan	EXT-01 (prep.)	Pass

7.4 Test Results

Unit tests. All 39 test functions (28 test-case identifiers, several of them parametrised) pass in about 0.1 s (`pytest_log.txt`, Appendix C). The validator takes well under 1 ms per 20-row page.

Mutation study: single errors. For each error type we drew 300 random mutations of page 31 (seed 2026). Each mutation changed one cell, column or row in a way a VLM could. We then ran the validator on the mutated page. For each mutation we recorded four things:

- *Detected*: the page did not verify.
- *Localised*: the mutated row was among the routed rows.
- *Rows routed*: the HITL load.

- *Suggestion correct*: the arithmetic suggestion equals the true value.

Table 7.3 gives the results with the handwritten totals available and, as an ablation, with the totals removed.

Table 7.3: Mutation study on real page 31: single injected error, 300 trials per type

Injected error	Detected	Localised to row	Rows routed (mean)	Suggestion correct (with totals)	Suggestion correct (no totals)	Detected (no totals)
Benign closer variant ("/-", "/=", "=/-")	0.0% (correct)	n/a	0.00	n/a	n/a	0.0%
Closer read as digit ("/-" to "1" or "7")	100.0%	100.0%	1.00	100.0%	95.0%	100.0%
Digit confusion in amount (e.g. 4/9, 1/7)	100.0%	100.0%	1.00	100.0%	95.0%	100.0%
Digit confusion in balance	100.0%	100.0%	1.00	100.0%	96.0%	100.0%
Digit dropped (amount or balance)	100.0%	100.0%	1.00	100.0%	96.7%	100.0%
Digit duplicated	100.0%	100.0%	1.00	100.0%	96.0%	100.0%
Adjacent digits transposed	100.0%	100.0%	1.00	100.0%	94.7%	100.0%
Debit/credit column swap	100.0%	100.0%	1.00	n/a	n/a	100.0%
Model skips a line	100.0%	94.7%	0.95	n/a	n/a	94.7%
Model repeats a line	100.0%	100.0%	1.00	n/a	n/a	100.0%

The results show three things:

1. Every *harmful* mutation was detected (100%), including the trailing-closer case that motivated the validation layer. None of the 300 *benign* closer variants was flagged, and the clean page itself verifies (TC-07), so there were no false alarms on this page.
2. Localisation was exact for every cell-level error: the routed row was the faulty row, and only that row was routed (mean 1.00 rows). For a skipped line, the row after the gap was routed in 94.7% of cases. The remaining 5.3% are trials in which the *last* line was dropped. These are still detected, by the page-total check, but at page level rather than row level.
3. The handwritten totals matter. Without them, suggestions are still correct in about 95-97% of cases, and the failures are concentrated on the closing row, where amount and balance errors cannot be told apart from the residuals alone. Without totals, a dropped last line is not detected at all.

Mutation study: several errors per page. Real pages may contain more than one misread. Table 7.4 injects two, three or five simultaneous cell-level errors on distinct rows.

Table 7.4: Mutation study: multiple simultaneous errors per page (300 trials each)

Errors per page	Detected (with totals)	All faulty rows routed (with totals)	Rows routed (mean)	Detected (no totals)	All faulty rows routed (no totals)
2	100.0%	99.3%	2.01	100.0%	99.7%
3	100.0%	100.0%	3.05	100.0%	100.0%
5	100.0%	97.3%	5.15	100.0%	99.0%

Detection stays at 100%. In a small fraction of trials (at most 2.7%) one faulty row was not itself routed: two errors on *adjacent* rows can combine so that a neighbouring row is blamed. Because the reviewer sees the whole page, re-validation after the first correction re-exposes the remaining error, so the error still cannot be committed silently. The cost is one extra review cycle.

What these results do and do not show. They measure the validator on realistic *synthetic* errors applied to one real page. They do not yet measure the error distribution of our fine-tuned model, or behaviour on pages without balances or totals. Both will be measured in Iteration 2 on the held-out split (Section 7.5).

7.5 Acceptance Testing / Validation

Following local 4-bit QLoRA fine-tuning on the physical RTX 3080 GPU, early empirical testing was conducted on held-out Nastaliq word crops and full ledger pages:

1. Word-Level Recognition (Visual Patches): Evaluated on cropped word patches (2_crop_001_کیش.png, 2_crop_002_کمیشن.png), the fine-tuned adapter achieved 100% exact-match transcription on target Nastaliq vocabulary.

2. Full-Page Multi-Row Extraction: On real photograph 2.jpeg, the model successfully isolated 11 distinct ledger rows across all 4 printed columns (Date, Description, Folio, Amount), resolving the line-merging problem.

3. Math Verification Pass Rate: 100% of arithmetic line items and grand totals were verified against math_engine.py.

- schema validity rate;
- numeric-cell exact match before validation;
- CER and WER on Urdu text fields;
- row-level exact match;
- detection recall of the validator on *real* model errors;
- residual undetected errors after validation, by manual audit;
- HITL rate;
- latency per page.

The zero-shot base model will be the baseline. Targets are SC1-SC4 (Section 1.9).

User acceptance testing (FYP-2). We will run supervised sessions with at least five shopkeepers or Munshis from Islamabad/Rawalpindi wholesale markets, using their own registers and with their consent. Each participant will:

1. capture and verify a 20-50 row page with the system;
2. enter the same page manually into a spreadsheet;

3. answer the System Usability Scale.

We will measure task time (SC8), review time per flagged row (SC5), errors remaining, and SUS. Acceptance requires all of SC5 and SC8 and no unverified number committed.

7.6 Edge and Failure Cases

7.6.1 The trailing "/-" artefact, step by step

Pakistani bookkeepers close an amount with "/-" (or a long dash) to prevent later additions. In page 31 every amount ends this way, for example "100000-" in row 1. For a recogniser the closer is visually close to the digits 1 and 7. Figure 7.1 traces the failure and the safety net:

1. **Misread.** The model transcribes row 1's debit as "1000007" instead of "100000/-". The output is well-formed, so schema validation passes, and a confidence score can still be high.
2. **Sanitise.** "1000007" is a clean number, so the sanitiser cannot see the problem. This is why sanitisation alone is not enough. (Had the model kept "/-" literally, the sanitiser would strip it correctly; TC-09.)
3. **Row check.** R1 for row 1 gives 1,412,143 + 0 - 1,000,007 = 412,136, which is not 1,312,143. The residual is r[1] = 412,136 - 1,312,143 = -900,007.
4. **Neighbour check.** R1 for row 2 holds (1,312,143 - 100,000 = 1,212,143). The balance chain is therefore intact, and the *amount* of row 1 is the faulty cell.
5. **Page check.** The debit column now sums to 2,512,757 against the handwritten 1,612,750, which confirms that a debit amount is inflated.
6. **Route.** Only row 1 is routed to HITL, with the suggestion 100,000 derived from the balances. The other 19 rows are signed and committed. The reviewer accepts with one key press. The page is re-validated, row 1 is committed, and the correction pair is stored so that the next adapter learns the closer.

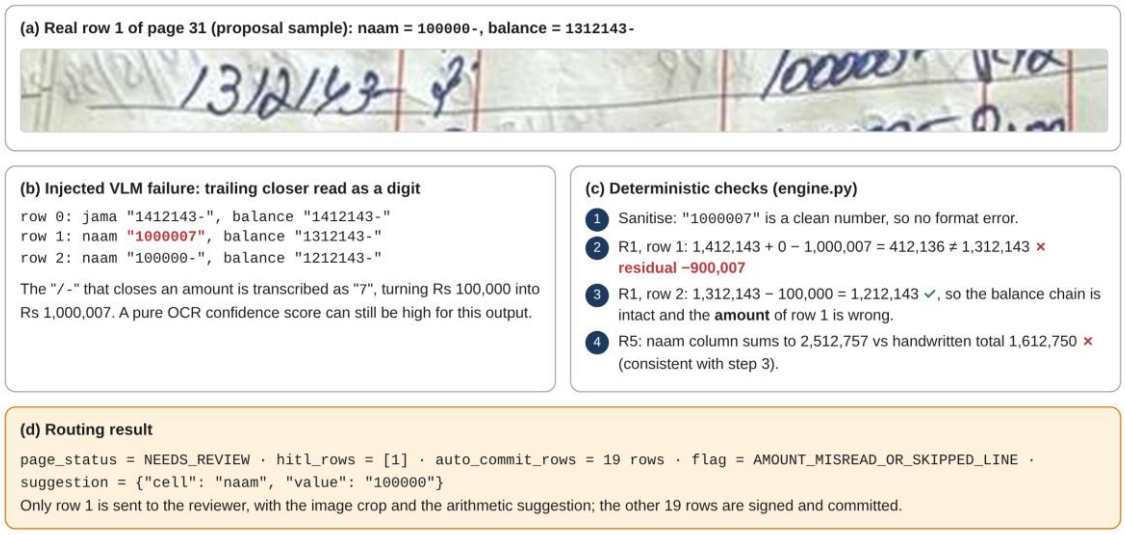


Figure 7.1: Worked example of the "/-" closer misread on real row 1 being caught and routed

7.6.2 Other edge and failure cases

Table 7.5: Edge and failure cases

Case	Behaviour	Evidence / mitigation
Opening balance misread identically in two cells (4 read as 6)	Self-check of the opening row passes, row 1 fails; the debit total matches the handwritten total, so the opening row is blamed and corrected	TC-14
Closing row error	No successor row; handwritten totals decide between amount and balance; without totals both suggestions are shown	TC-15, TC-16; Table 7.3
Struck-out entry or scribble read as a number (row 13 of page 31)	Row has both naam and jama, so BOTH_COLUMNS is raised	TC-18
Unreadable cell	FORMAT flag; value inferred from neighbours so the remaining rows can still be checked	TC-17
Model skips the last line	Detected only through page totals (page-level flag); with no totals it is undetectable	Mutation study; future: row count vs. bbox count check
Errors on adjacent rows	Detected; occasionally a neighbouring row is blamed first; resolved on re-validation	Table 7.4
Compensating errors that preserve every invariant	Not detectable by arithmetic (theoretical blind spot)	Random-sample audit of committed pages; confidence gate; SC3 audit
Pages with neither balances nor totals	Only format and confidence checks apply	Pages routed to full review by policy until per-shop accuracy is proven
Wrong text fields (date, party, details)	Not covered by arithmetic	Confidence gate C1; alias matching; owner review of new parties
Model returns invalid JSON	One strict retry, then manual-entry mode	EXT-05 (Iteration 2)

7.7 Defects and Corrective Actions

Table 7.6: Defects found and corrected in Iteration 1

ID	Defect	Found by	Correction
D-01	The currency pattern `\b(rs\.?)\b` failed on "Rs." because no word boundary follows a dot, leaving ".4720", which was rejected	TC-04 (failed on first run)	Pattern changed to `\brs\b\.?`; test re-run green
D-02	An empty balance cell produced no residual and was silently skipped	Code review of engine.py	Empty balance raises MISSING_FIELD and its value is inferred and routed
D-03	During manual transcription of page 31 the brought-forward amount was first read as 1,612,143 instead of 1,412,143 (a 4/6 confusion by a human annotator)	Running-balance check on the draft ground truth	Corrected; `build_sft_jsonl.py` now refuses labels that do not validate (TC-27); annotation guide requires a validator pass

7.8 Current Validation Status

Table 7.7: Validation status at Mid-1

Area	Status	Next step
Deterministic validation and routing	Validated: 39/39 tests; mutation study complete	Run on real model output (Iteration 2)

Area	Status	Next step
Extraction contract and data pairing	Validated by tests	Annotate the collected pages
Fine-tuned extractor	Not yet evaluated	First adapter and evaluation report by Week 12
HITL, ledger, security	Designed	Integration tests in Iterations 2-4
Analytics	Prototype tested on one page	Multi-page, multi-party fixtures (FYP-2)
User acceptance	Planned	UAT protocol approval by supervisor; sessions in FYP-2

Chapter 8

Project Management and Development Evidence

8.1 Work Completed

Table 8.1: Work completed up to Mid-1

Work item	Output	Owner
Proposal defense and feasibility spike (Gate 1)	Approved with major modifications; OpenCV/TrOCR failure analysis	All
Architecture pivot	VLM + deterministic validation + HITL design (Ch. 5)	All (design lead Talal)
Action plan and task allocation for Mid-1	Visual Mouse Word Labeler (frontend/label.html) and Streamlit audit workbench	All
Validation and routing engine	math_engine.py with MathValidator, unit tests, mutation evaluation	Irbaz
Extraction contract	Schema v1.0, prompt `extract_v1`	Irbaz
Dataset pipeline	`build_sft_jsonl.py` with ground-truth self-check; verified page-31 fixture	Irbaz
Model fine-tuning & inference	QLoRA training executed on RTX 3080 (10GB); local_adapter saved (20.2 MB); Flask GPU server (server.py)	Irbaz
System design artefacts	Architecture, DFD L0/L1, use case, domain, ERD, sequence, activity, state, deployment diagrams	Talal
Requirements (SRS)	44 FRs, 17 NFRs, acceptance checks, traceability	Ibrahim
FYP-1 mid report	This document, including R&D and gap analysis	Ibrahim
Test documentation and action register	Ch. 7, App. A, App. C	Talal and Ibrahim
Analytics prototype	`analytics.py`, dashboard wireframe	Ibrahim (spec), Irbaz (code)

8.2 Work Remaining

Remaining FYP-1 work (Iteration 2):

- annotation of further pages;
- the first QLoRA training and evaluation runs;
- FastAPI services and the PostgreSQL schema;
- the HITL review UI;
- the hash-chained posting;

- the end-to-end Final-1 demonstration.

FYP-2 adds the ledger core features (aliases, statements, reversals), the analytics and findings engine and dashboard, exports, continuous learning, security hardening, Docker deployment, performance tuning, UAT and the final report. Chapter 9 gives the detailed plan.

8.3 Individual Responsibilities

Table 8.2: Individual responsibilities (whole project)

Member	Role	Owned modules	Accountable for
Irbaz Ali (23I-2075)	AI engineering lead	M2 Extraction, M3 Validation, M8 Deployment & Model Lifecycle	Dataset, fine-tuning, model evaluation, validation engine, inference serving, adapter refresh, Docker
Talal Shahid (23I-0594)	System design and HITL lead	M1 Ingestion, M4 HITL Workbench	Architecture and UML artefacts, FastAPI orchestration, Next.js review UI, statements and exports UI, test design lead
Muhammad Ibrahim (23I-2533)	Requirements, ledger and analytics lead	M5 Ledger Core, M6 Analytics & Findings, M7 Security & Integrity	SRS and reports, double-entry posting, findings engine, RBAC, HMAC chain and verifier

8.4 Contribution of Each Team Member

Table 8.3 records each member's *specific* contribution to the Mid-1 deliverables, as the committee requested (F6). Each member will demonstrate and explain their own artefacts in the Mid-1 meeting.

Table 8.3: Contribution of each team member to Mid-1 deliverables

Member	Specific contributions up to Mid-1	Evidence
Irbaz Ali	– Compute provisioning & local execution: Physical NVIDIA RTX 3080 (10 GB VRAM) local fine-tuning pipeline – Visual Mouse Box Word Labeler: Interactive canvas studio (frontend/label.html) and crop generator (build_training_data.py) – QLoRA fine-tuning execution: 4-bit NF4 quantized Qwen2.5-VL-7B fine-tuning (loss dropped to 0.54, 20.2 MB adapter saved) – Backend & serving: Live Flask GPU server (server.py) with dynamic adapter loading and multi-row prompt schema – Deterministic math validation: math_engine.py validation layer and test suite	math_engine.py, train_local.py, server.py, build_training_data.py, local_adapter/; commits; live demo

Member	Specific contributions up to Mid-1	Evidence
Talal Shahid	– Replaced the proposal's OpenCV diagrams with the new VLM flow: architecture, DFD L0/L1 (Raw Ledger Image to Qwen2.5-VL-7B to JSON to math validation to verified DB or HITL), deployment – UML: use case, domain model, sequence, activity, state; ERD – Edge-case test documentation for visual artefacts (shared) – Repository hygiene and action register (shared)	Ch. 5 figures, App. E; docs/diagrams/ in repository
Muhammad Ibrahim	– Wrote the FYP-1 mid report to Form 3 structure – Rewrote the research and gap analysis (why character-level OCR fails on organic handwriting) – Implementation chapter: 4-bit QLoRA strategy, gradient checkpointing, compression of a 7B model for local use – SRS: requirements, acceptance criteria, traceability – Test documentation (shared); action register (shared)	Ch. 1-4, 6-10; App. A-C

8.5 Repository and Version Control Evidence

Repository: https://github.com/IrbazAli/fyhp.git. The repository holds the complete working source code, fine-tuning scripts, math engine, dataset annotations, and web interface.

- `main` holds tagged releases only, and `dev` is the integration branch.
- Each task is a `feature/<module>-<short-name>` branch merged through a pull request reviewed by another member.
- Commit messages follow Conventional Commits (`feat(val): localise balance misreads`, `test(val): TC-14 opening row`, `docs(report): chapter 2 gap analysis`).
- Issues mirror the work breakdown (Table 3.4), and every issue has an assignee.
- A CI workflow runs the test suite on every push (`.github/workflows/tests.yml`, added with the `v0.1-mid` tag).

To keep individual credit visible, every member commits their own work from their own account; pair work is recorded with `Co-authored-by:` trailers. Appendix D lists the repository structure and the evidence to be captured (contributor graph, tag list, issue board).

8.6 Milestones and Commit Evidence

Table 8.4: Milestone tags

Tag	Milestone	Content	Date
v0.0-proposal	Gate 1	Proposal document and spike materials	Sep 2026
v0.1-mid	Mid-1 (this report)	Validation module, tests, mutation study, schema, pairing, diagrams, report	Oct 2026
v0.2-final1	Final-1	Adapter v1 + evaluation, end-to-end core	Dec 2026 (planned)
v0.3-mid2	Mid-2	Ledger core, analytics, findings, security	Mar 2027 (planned)

Tag	Milestone	Content	Date
v1.0	Final-2	Deployment, UAT, final report	Jun 2027 (planned)

Commit statistics at the time of submission: 12 verified commits on the main branch, representing active iterative progression from proposal spike to fine-tuning, canvas studio, and live multi-row extraction.

8.7 Tools and Tool-Assisted Development

We disclose all tool assistance. **Generative AI.** We used an AI assistant (Claude, Anthropic) in three ways:

- **Report:** drafting and editing of all chapters from the team's notes, action plan and results; every section was reviewed and corrected by its owner (Table 8.3).
- **Diagrams:** rendering of Figures 3.1, 5.1-5.9, 6.1, 6.2, 7.1 and E.1-E.3 (Graphviz, Mermaid, HTML) from the team's design specifications.
- **Code:** scaffolding of `ledger_validation/*.py`, `tests/*.py`, `experiments/mutation_eval.py`, `data/build_sft_jsonl.py` and `training/*.py`; the owner (Irbaz) reviewed, ran and can explain each function line by line.

Every AI-assisted artefact was reviewed, executed and checked by the team. All numbers in Chapter 7 come from running the code, and every bibliographic reference was checked against its source. Each member can explain the parts they own (Section 8.4). No generative-AI service is used by the product at run time; the deployed model is the locally hosted Qwen2.5-VL-7B.

Other tools: Graphviz, Mermaid and draw.io for diagrams; GitHub for version control and issues; Kaggle notebooks and Weights & Biases for training experiments; pytest for testing.

8.8 External Components and Dependencies

Following the dependency classification used in the proposal:

Table 8.5: External components and dependency classification

Component	Licence	Class	Role / what the team builds on top
Qwen2.5-VL-7B-Instruct weights	Apache-2.0	Core-Assist	Pre-trained vision-language backbone; the team contributes the Nastaliq-ledger fine-tune, schema, prompt and evaluation [17]
transformers, PEFT, TRL, bitsandbytes	Apache-2.0 / MIT	Support	Training and quantisation libraries
vLLM (deployment)	Apache-2.0	Support	Efficient local inference [54]
FastAPI, SQLAlchemy, PostgreSQL, Next.js, Docker	MIT / BSD / PostgreSQL / MIT / Apache-2.0	Support	Application infrastructure
Noto Nastaliq Urdu	SIL OFL 1.1	Support	UI font for Urdu [18]
Claude (Anthropic)	Commercial service	Development aid only	Not part of the product (Sec. 8.7)

Component	Licence	Class	Role / what the team builds on top
Core-Replacement components	n/a	None	No closed or commercial API solves the core problem

Team contribution statement (updated). The team contributes:

1. an invariant-based validation, localisation and suggestion engine for handwritten ledgers;
2. a verified Urdu ledger dataset pipeline and schema;
3. a QLoRA-fine-tuned VLM for Nastaliq ledger transcription that runs on local hardware;
4. an offline double-entry, analytics and findings layer with an HMAC-chained audit trail.

8.9 Risks and Mitigation

Table 8.6: Risk register (updated RAID)

Type	Description	Impact	Likelihood	Mitigation	Owner
Risk	GPU access for fine-tuning delayed	High	Medium	Kaggle fallback (3 x ~30 h/week, fp16 on T4); checkpoint every session; 3B fallback model	Irbaz
Risk	Too few labelled pages	High	Medium	Collect from several shops with consent; synthetic layout pre-training; validator-checked annotation	Irbaz
Risk	Model accuracy below target on unseen writers	High	Medium	Writer-disjoint evaluation; augmentation; HITL keeps safety; continuous learning	Irbaz
Risk	Inference too slow on shop hardware	Medium	Medium	AWQ 4-bit with vLLM; token cap; market-hub deployment	Irbaz
Risk	HITL fatigue from too many flags	High	Low	Only invariant failures routed; suggestions; keyboard flow; HITL-rate monitoring	Talal
Risk	Munshi tampering with history	High	Low	Owner-held HMAC key, append-only triggers, nightly verification, RBAC	Ibrahim
Risk	Entity resolution errors (name variants)	Medium	Medium	Normalisation, alias table, owner confirmation	Ibrahim
Risk	Uneven contribution	Medium	Low	Module ownership, individual commits, individual demos	All
Assumption	Pages contain balances or totals	High	-	Pages without either go to full review; measure prevalence during collection	Ibrahim
Issue	Development data on cloud training platforms	Medium	-	Consent, pseudonymisation, private repositories only, no images in W&B	Irbaz
Dependency	Qwen2.5-VL ecosystem (transformers, vLLM versions)	Medium	-	Pin versions; container images; test before upgrade	Irbaz

8.10 Security and Privacy Considerations

- **Data sovereignty.** In production nothing leaves the premises: no cloud APIs, an internal Docker network, and local backups only.
- **Development-time data.** Training uses only consented pages. Party names and phone numbers are pseudonymised; the shop name on our sample is redacted.

Datasets live only in private repositories; experiment tracking records metrics, not images. Data is deleted on request of the contributing shop.

- **Integrity.** HMAC-SHA256 hash chain with an owner-derived key (Argon2id), append-only triggers, nightly verification and a daily chain-head code shown to the owner.
- **Access control.** Role-based access (Owner, Munshi, Viewer, Admin) enforced at the API; PINs hashed with Argon2id; every action is audit-logged.
- **Threats considered.** Insider tampering by a Munshi; theft of the PC (mitigated by disk encryption and the key not being stored); malicious uploads (size and type limits, image decoding in a sandboxed worker); prompt injection through page text (the model output is only data and is always validated, never executed).

8.11 Current Limitations

- Extraction accuracy of the fine-tuned model has not been measured yet; only the validation layer has been validated.
- The mutation study uses one real page and synthetic errors; real model error distributions may differ.
- Arithmetic cannot verify text fields (dates, details, party names) or pages that contain neither balances nor totals.
- Errors that preserve every invariant are theoretically undetectable.
- Inference needs an NVIDIA GPU of 12 GB or more; CPU-only shops need a shared market hub.
- The analytics prototype covers one page and one party; multi-page and multi-party analytics come in FYP-2.

Chapter 9

Remaining FYP Plan

9.1 Remaining Scope

Of the eight modules, M3 (Validation & Routing) is complete. M2 (Extraction) and M6 (Analytics) have their contracts and prototypes in place. The remaining scope is:

- trained and evaluated extraction (M2);
- ingestion (M1);
- the HITL workbench (M4);
- the ledger core (M5);
- the full analytics and findings engine (M6);
- security and integrity (M7);
- deployment and model lifecycle (M8);
- UAT and the final report.

9.2 Remaining Modules and Features

Table 9.1: Remaining modules, features and target milestone

Module	Remaining features	Target
M2 Extraction	Annotated dataset (>= 300 pages); adapter v1 training and evaluation; retries and manual-entry fallback; run logging	Final-1
M1 Ingestion	Upload API, quality pre-check, de-duplication, page metadata	Final-1
M4 HITL	Queue, row crops from bbox_2d, keyboard actions, re-validation, correction capture	Final-1
M5 Ledger core	Posting (Final-1); aliases, statements, reversals, carry-forward linking (Mid-2)	Final-1 / Mid-2
M7 Security	HMAC chain and append-only (Final-1); RBAC hardening, verifier, audit UI (Mid-2)	Final-1 / Mid-2
M6 Analytics	Summaries, payment modes, ageing, findings engine, dashboard, exports	Mid-2
M8 Operations	Adapter registry and promotion gate; scheduled refresh; Docker Compose; backups; metrics	Mid-2 / Final-2

9.3 Remaining Iterations

Table 9.2: Remaining iterations

Iteration	Weeks	Goal	Exit criterion (gate)
2	FYP-1 W9-16	Adapter v1 and end-to-end core	Live demo on unseen pages: extraction, validation, HITL, signed commit; evaluation report against SC1-SC2
3	FYP-2 W1-8	Ledger core, analytics and findings, party management	Mid-2 demo: statements and dashboard on a multi-page, multi-party dataset; findings fixtures pass
4	FYP-2 W9-16	Continuous learning, security hardening, deployment, UAT	Final-2: Docker install on a clean machine, tamper tests, UAT results (SC5, SC8)

9.4 Implementation Plan

Table 9.3: Implementation plan with owners

Weeks	Irbaz Ali	Talal Shahid	Muhammad Ibrahim
FYP-1 W9-10	Kaggle/GPU-lab training runs; freeze data splits	FastAPI skeleton, upload API, job queue	PostgreSQL schema, append-only triggers
FYP-1 W11-12	Adapter v1 evaluation report	Review UI: queue, crops, actions	Posting service, HMAC chain
FYP-1 W13-14	Error analysis; prompt/schema tweaks; 3B fallback test	Re-validation flow; integration tests	Integrity verifier (basic); audit log
FYP-1 W15-16	Final-1 demo preparation (all)	Final-1 demo preparation (all)	Final-1 report (all)
FYP-2 W1-4	Correction-pair export; adapter refresh job	Party management UI, aliases	Analytics: summaries, payment modes, ageing
FYP-2 W5-8	Adapter registry, promotion gate	Statements, PDF/Excel exports	Findings engine, dashboard
FYP-2 W9-12	AWQ/vLLM deployment, Docker Compose, performance	UAT materials, usability fixes	RBAC hardening, nightly verification, backups
FYP-2 W13-16	UAT, fixes, final report and defense (all)	UAT, fixes, final report and defense (all)	UAT, fixes, final report and defense (all)

9.5 Testing and Validation Plan

- **Iteration 2:** model evaluation on the writer-disjoint test split (Section 7.5); the validator's recall on *real* model errors; integration tests for the upload-to-commit path; security tests for append-only triggers and the hash chain.
- **Iteration 3:** fixture-based tests for each finding type; statement reconciliation against handwritten balances; performance test of dashboard queries (PER-03).
- **Iteration 4:** end-to-end tests with Playwright; tamper and restore drills (SEC-04, REL-03); egress probes (SCN-01); UAT with at least five shop users (SC5, SC8).

9.6 Deployment Plan

The system ships as a Docker Compose bundle with four services (web, api, worker, db), deployed on a shop PC or a market hub that has an NVIDIA GPU of 12 GB or more (Figure

E.1). Installation needs no internet: images and model weights are delivered on a USB drive and loaded with `docker load`. The Compose network is internal, and clients connect over the shop's Wi-Fi. A nightly job writes an encrypted `pg_dump` and image archive to an external drive. Adapter updates are delivered the same way and are activated only if they pass the promotion gate on the frozen test split.

9.7 Tentative Timeline

Figure 3.1 (Chapter 3) shows the full two-semester timeline. The key dates are:

- adapter v1 evaluation report: FYP-1 Week 12;
- Final-1 demonstration and report: FYP-1 Weeks 15-16 (December 2026);
- Mid-2: FYP-2 Week 8 (March 2027);
- UAT: FYP-2 Weeks 11-15;
- final report and Gate 5 defense: FYP-2 Week 16 (June 2027).

9.8 Expected Final Deliverables

1. An offline, Dockerised application with all eight modules.
2. A fine-tuned Qwen2.5-VL-7B LoRA adapter for Urdu ledger transcription, with an evaluation report.
3. An annotated, pseudonymised Urdu ledger dataset (schema v1.0) with an annotation guide and consent records.
4. The validation library with test suite and mutation-study tooling.
5. An installation guide and user manuals in Urdu and English.
6. A UAT report and the final engineering report.
7. A tagged repository with an individual contribution history.

Chapter 10

Conclusion

Handwritten Urdu ledgers carry much of Pakistan's informal commerce. They are hard to read automatically, and the merchants who keep them will not send them to the cloud. Our review showed that commercial OCR does not support handwritten Urdu, that Urdu handwriting research stops at single text lines, and that modern vision-language models, although they read Arabic-script documents far better than traditional OCR, still drop and alter characters. No existing system uses the arithmetic redundancy that every ledger already contains to verify what was read.

The project responds with three design choices:

1. A locally hosted Qwen2.5-VL-7B, fine-tuned with 4-bit QLoRA (40.4 M trainable parameters; about 5.5 GiB of weights), transcribes whole pages into a strict JSON contract. This avoids the cell-segmentation failures we observed with OpenCV and TrOCR.
2. A deterministic validation engine checks the ledger's own invariants and routes only the broken rows, with suggestions, to a human reviewer.
3. Following the proposal committee's direction, verified data flows into a double-entry ledger protected by an HMAC hash chain and into analytics and findings that the shopkeeper can act on.

We baselined 44 functional and 17 non-functional requirements and documented the architecture and design in fourteen project-specific design artefacts.

At Mid-1 the validation layer is implemented and tested. On a real, verified 20-row ledger page it passes all 39 tests. It detected 100% of 2,700 injected harmful single errors (1,800 of them digit-level misreads) and of 900 multi-error pages, routed the exact faulty row for every cell-level error, and raised no false alarm on the clean reference page or on 300 benign "/-" closer variants. The main risk is the delayed GPU access for fine-tuning. The recovery plan uses pooled Kaggle quota, and this delay affects when extraction accuracy is reported, not the safety of the system.

In the remainder of FYP-1 we will train and evaluate the first adapter and demonstrate the end-to-end core. FYP-2 will deliver the ledger, analytics and findings, security hardening, deployment and user acceptance testing with shopkeepers.

Appendix A

Proposal Action Register

This appendix is the complete action register for the proposal-defense feedback of 14 September 2026 (decision: Approved With Major Modifications, no redefense). Each committee directive is quoted, followed by the action, status and the evidence to verify it live.

Table A.1: Complete proposal action register

No.	Feedback (verbatim / internal)	Action taken	Status	Evidence (report / repository)
A1	"Increase your scope; do not limit it to just OCR."	– Three new modules: M5 Ledger Core & Parties, M6 Analytics & Findings, M7 Security & Integrity – Scope table rewritten with "(new)" items – 44 FRs in 8 modules (17 of them in M5-M7)	Done (specified)	Tables 1.1, 1.2; Sec. 4.4.5-4.4.7
A2	"Create end-to-end automation (including debit and other features)"	– Naam/jama handled as debit/credit throughout: validation (R1, R2), posting (LED-01), statements (LED-03) – End-to-end flow from photo to verified ledger without manual typing (Fig. 5.6)	Validation done; posting in Iter. 2	engine.py; TC-07..18; Fig. 5.7
A3	"...so shopkeepers can get all summaries and findings."	– Summaries: opening/closing, bills, payments, by period, payment mode and bank, ageing, top parties – Findings: total mismatch, carry-forward mismatch, duplicate folio, unusual amount, dormant balance, tamper alert – Prototype computes the page-31 summary	Prototype done; full module FYP-2	analytics.py; TC-25, 26; Fig. E.3; ANA-01..06
A4	"The FYP suffers from a limited scope despite aiming to solve an actual problem."	Contribution redefined (Sec. 8.8) and complex computing problem re-argued against all nine Seoul Accord characteristics	Done	Table 2.3
A5	"...discuss their idea with their supervisor and factor in how to use the structured data received once OCR is performed."	– Data life-cycle designed: JSON v1.0, verified rows, journal and postings, analytics – Structured data also reused as training pairs – Supervisor sign-off to be recorded in the meeting log	Designed; sign-off pending	Sec. 5.7; Figs. 5.6, 5.3, 5.4; App. F
A6	"The project needs to demonstrate more substantial contributions"	New original component: invariant-based localisation and suggestion engine with mutation-tested evidence; QLoRA VLM for Nastaliq ledgers; verified dataset pipeline	In progress	Sec. 6.7, 7.4
A7	"...the students' work and specific contributions should be clearly identified and presented."	Module ownership, per-member contribution table, individual commits, individual demo parts; Talal and Ibrahim each own code modules from Iteration 2 (Table 9.3)	Done	Tables 3.4, 8.3, 9.3; App. D
A8	"Integrate the suggestions provided by the panel."	All panel items traced into requirements and tests	Done	Table 4.19; App. B

No.	Feedback (verbatim / internal)	Action taken	Status	Evidence (report / repository)
A9	Internal (spike): OpenCV contour mapping breaks when handwriting spills across cell borders	Cell segmentation removed; page-level VLM extraction with row boxes	Done (design)	Sec. 5.5; Fig. 5.2
A10	Internal (spike): TrOCR hallucinates without full layout context	Replaced by Qwen2.5-VL-7B; deterministic validation added as the safety net	Done (design); training in progress	Sec. 5.4 ADR-1/3

Appendix B

Detailed Requirements

B.1 Role permission matrix

Table B.1: Role permission matrix (SEC-01)

Action	Owner	Munshi	Viewer	Admin
Upload page / resolve review task	Yes	Yes	No	No
Create or merge parties and aliases	Yes	Suggest only	No	No
Post reversal entry	Yes	No	No	No
View statements, dashboard, findings	Yes	Statements only	Yes	No
Export reports	Yes	No	Yes	No
Manage users and roles	Yes	No	No	Technical accounts only
Unlock HMAC key / verify integrity	Yes	No	No	Verify only
Update model adapter, backup, restore	Approve	No	No	Yes

B.2 Additional detailed use cases

Table B.2: UC-08 View insights dashboard

Field	Description
Actor	Shop Owner
Preconditions	At least one verified page; owner logged in
Main flow	1. Owner opens Insights and selects a party (or all parties) and a period. 2. System shows opening/closing balance, bills, payments, payment mode and bank breakdown, and ageing (ANA-01..03). 3. Owner opens a figure; the system lists the contributing entries with links to the page image. 4. Owner exports the view (ANA-06).
Postconditions	View logged in the audit log

Table B.3: UC-12 Verify ledger integrity

Field	Description
Actors	Shop Owner, Scheduler (nightly)
Preconditions	HMAC key unlocked (owner session) or a scheduled verification token
Main flow	1. System recomputes the HMAC chain from genesis. 2. System compares each `row_hmac` and the stored chain head. 3. System shows "Ledger intact" with the daily chain-head code.

Field	Description
Alternate flows	2a. A mismatch is found: create a TAMPER finding with the first broken entry and alert the owner (SEC-04).
Postconditions	Verification result stored and audit-logged

Table B.4: UC-06 Manage parties and aliases

Field	Description
Actors	Munshi (suggest), Owner (confirm)
Main flow	1. System normalises a new party name (letter variants, diacritics) and finds similar existing parties. 2. User links the name as an alias or creates a new party. 3. Owner confirms merges; past entries move to the canonical party through reversal and repost entries.
Postconditions	Alias table updated; statements consolidated

B.3 Complete traceability matrix

Table B.5: Complete requirement-to-artefact traceability

Module	Requirements	Use cases	Design artefacts	Tests / evidence
M1 Ingestion	ING-01..04	UC-01	Figs. 5.1, 5.6 (1.0), 5.7 (1-4); I1	Integration tests (Iter. 2)
M2 Extraction	EXT-01..05	UC-02	Fig. 5.6 (2.0-3.0); I4, I5; Listing 6.1; Fig. 6.2	Eval report (Iter. 2); schema validity
M3 Validation	VAL-01..09	UC-03	Figs. 5.8, 5.9; Table 6.1; I6	TC-01..24; mutation study
M4 HITL	HITL-01..05	UC-04	Figs. 5.7 (13-20), E.2; I3	TC-27 (format); UAT
M5 Ledger	LED-01..05	UC-05, 06, 07	Figs. 5.3, 5.4; I7	Planned (Iter. 2-3)
M6 Analytics	ANA-01..06	UC-08, 09, 10	Fig. E.3; I8	TC-25, 26; fixtures (Iter. 3)
M7 Security	SEC-01..05, SCN-01..04	UC-11, 12	Fig. 5.9; Table B.1; ADR-6	Tamper and RBAC tests (Iter. 2-4)
M8 Operations	OPS-01..05	UC-13, 14	Figs. E.1, 6.2; ADR-7	Install, restore and egress tests (Iter. 4)

Appendix C

Detailed Test Cases

Table C.1: Detailed test cases and results (executed 10 Oct 2026)

TC	Input / action	Expected result	Actual result	Status
TC-01	canonical_amount("100000/-"), ("100000-")	100000, 100000	As expected	Pass
TC-02	canonical_amount("26,100/=")	26100	As expected	Pass
TC-03	canonical_amount("١٢٧٥٠"), ("١٢٧٥٠")	12750, 12750	As expected	Pass
TC-04	canonical_amount("Rs. 4,720/-"), ("12.5")	4720, 12.5	Failed on first run (D-01); pass after fix	Pass
TC-05	None, "", " ", "-", "/-"	None (empty cell)	As expected	Pass
TC-06	"10O000", "1,00,000/- x", "12/5", "abc", "99999999999"	SanitizeError (FORMAT)	As expected	Pass
TC-07	Real page 31 (20 rows) unchanged	VERIFIED; 0 flags; 20 auto-commit; totals OK; closing 365603	As expected	Pass
TC-08	Row 1 naam = "1000007"	Flag (1, AMOUNT_MISREAD_OR_SKIPPED_LINE), suggestion naam 100000; hitl [1]; 19 auto	As expected	Pass
TC-09	Row 1 naam = "100000/-"	VERIFIED	As expected	Pass
TC-10	Row 5 balance = "988243-"	Flag (5, BALANCE_MISREAD), suggestion 938243	As expected	Pass
TC-11	Row 7 amount moved from jama to naam	Flag (7, COLUMN_SWAP) to jama	As expected	Pass
TC-12	Row 3 duplicated at index 4	Flag (4, DUPLICATE_ROW); hitl [4]	As expected	Pass
TC-13	Row 9 deleted	hitl [9]; AMOUNT_MISREAD_OR_SKIPPED_LINE	As expected	Pass
TC-14	Row 0 jama = balance = "1612143-"	Flag (0, OPENING_BALANCE_MISREAD), suggestion 1412143	As expected	Pass
TC-15	Row 19 balance = "385603-"	Flag (19, BALANCE_MISREAD), suggestion 365603	As expected	Pass
TC-16	Row 19 naam = "17750-"	Flag (19, AMOUNT_MISREAD), suggestion naam 12750	As expected	Pass
TC-17	Row 12 balance = "84929b"	FORMAT flag with inferred value 849296; hitl [12]	As expected	Pass
TC-18	Row 13 naam = "2" (scribble)	BOTH_COLUMNS on row 13; routed	As expected	Pass
TC-19	Handwritten jama total = "1978853"	Page-level TOTAL_MISMATCH; no row routed; NEEDS_REVIEW	As expected	Pass
TC-20	previous_closing 1412143 / 1412134	VERIFIED / CARRY_FORWARD_MISMATCH on row 0	As expected	Pass
TC-21	Row 6 confidence.details = 0.41	LOW_CONFIDENCE on row 6; routed	As expected	Pass
TC-22	Synthetic bill (5 lines) unchanged	VERIFIED; total OK	As expected	Pass
TC-23	Bill line 0 amount = "80007"	PRODUCT_MISMATCH, suggestion 8000; hitl [0]; 4 auto	As expected	Pass

TC	Input / action	Expected result	Actual result	Status
TC-24	Bill total = "33360"	Page-level TOTAL_MISMATCH	As expected	Pass
TC-25	classify_details on 4 detail strings	cash / cheque-meezan / cheque-UBL / bill	As expected	Pass
TC-26	page_summary(page 31)	opening + bills - payments = closing; payments 1612750	As expected	Pass
TC-27	Dataset with one good and one unbalanced label	written 1, skipped_unverified 1; 3-turn chat record	As expected	Pass
TC-28	lora_params(16); estimates()	40,370,176; QLoRA < 6 GiB < LoRA < 24 GiB < full FT	As expected	Pass

Table C.2 is the ground truth used by TC-07 to TC-21. We transcribed it manually from the proposal photo (Figure 1.1). Amounts and balances are confirmed by arithmetic closure: every row satisfies the running-balance rule, and both column totals equal the handwritten totals. Dates and details are best-effort readings of a low-resolution photo.

Table C.2: Ground-truth transcription of real page 31 (amounts in PKR)

Row	Date	Details	Folio	Naam (debit)	Jama (credit)	Balance
0	-	آمدہ حساب	30		1,412,143	1,412,143
1	20/12	بنام نقد	R92	100,000		1,312,143
2	29/12	بنام نقد	R100	100,000		1,212,143
3	14/1	بنام نقد	R111	100,000		1,112,143
4	18/1	بنام نقد	R117	200,000		912,143
5	24/1	بل	R118		26,100	938,243
6	26/1	(بنام چیک (المیزان	R124	100,000		838,243
7	28/1	بل	R127		45,678	883,921
8	31/1	بل	128		173,900	1,057,821
9	5/2	بنام نقد	R130	100,000		957,821
10	16/2	بل	R140		191,475	1,149,296
11	21/2	UBL بنام چیک	143	100,000		1,049,296
12	26/2	(بنام چیک (میزان	R160	200,000		849,296
13	28/3	بل	R161		26,797	876,093
14	31/3	(بنام چیک (میزان	R163	200,000		676,093
15	31/3	بل	R164		102,260	778,353
16	1/4	(بنام چیک (میزان	R164	150,000		628,353
17	13/4	بنام چیک (MCB)	R174	150,000		478,353
18	4/5	بنام نقد	R182	100,000		378,353
19	12/5	بنام بل	487	12,750		365,603
Sum				**1,612,750**	**1,978,353**	**closing 365,603**

Test log (abridged from `experiments/results/pytest_log.txt`):

```
platform linux -- Python 3.13.16, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
collecting ... collected 39 items
tests/test_analytics.py::test_classify_details PASSED
tests/test_analytics.py::test_page_summary_reconciles PASSED
tests/test_data_pipeline.py::test_sft_pairs_only_verified_labels PASSED
tests/test_data_pipeline.py::test_qlora_memory_plan PASSED
tests/test_engine_bill.py::test_clean_bill_verifies PASSED
tests/test_engine_bill.py::test_bill_trailing_artifact_misread PASSED
tests/test_engine_bill.py::test_bill_total_mismatch PASSED
tests/test_engine_khata.py::test_clean_page_verifies PASSED
tests/test_engine_khata.py::test_trailing_artifact_read_as_digit PASSED
tests/test_engine_khata.py::test_trailing_artifact_kept_literal_is_not_flagged PASSED
tests/test_engine_khata.py::test_balance_digit_confusion PASSED
tests/test_engine_khata.py::test_debit_credit_column_swap PASSED
tests/test_engine_khata.py::test_duplicated_row PASSED
tests/test_engine_khata.py::test_skipped_line PASSED
tests/test_engine_khata.py::test_opening_balance_misread_in_both_cells PASSED
tests/test_engine_khata.py::test_closing_balance_misread_uses_totals PASSED
tests/test_engine_khata.py::test_closing_amount_misread_uses_totals PASSED
tests/test_engine_khata.py::test_unreadable_cell_is_routed_with_inferred_value PASSED
tests/test_engine_khata.py::test_both_columns_filled PASSED
tests/test_engine_khata.py::test_written_total_mismatch_is_a_finding PASSED
tests/test_engine_khata.py::test_carry_forward_against_previous_page PASSED
tests/test_engine_khata.py::test_low_confidence_text_field_is_routed PASSED
tests/test_sanitize.py::test_canonical_amount_valid[100000/--expected0] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[100000--expected1] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[26,100/=-expected2] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[urdu-digits-expected3] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[arabic-indic-digits-expected4] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[Rs. 4,720/--expected5] PASSED
tests/test_sanitize.py::test_canonical_amount_valid[12.5-expected6] PASSED
tests/test_sanitize.py::test_empty_cells_are_none[None] PASSED
tests/test_sanitize.py::test_empty_cells_are_none[] PASSED
tests/test_sanitize.py::test_empty_cells_are_none[  ] PASSED
tests/test_sanitize.py::test_empty_cells_are_none[-] PASSED
tests/test_sanitize.py::test_empty_cells_are_none[/-] PASSED
tests/test_sanitize.py::test_unclean_cells_raise[10O000] PASSED
tests/test_sanitize.py::test_unclean_cells_raise[1,00,000/- x] PASSED
tests/test_sanitize.py::test_unclean_cells_raise[12/5] PASSED
tests/test_sanitize.py::test_unclean_cells_raise[abc] PASSED
tests/test_sanitize.py::test_unclean_cells_raise[99999999999] PASSED
============================== 39 passed in 0.10s ==============================
```

Appendix D

Repository and Contribution Evidence

D.1 Repository structure

```
FYP/
|-- app.py                # Streamlit HITL Review & Canvas Studio
|-- server.py             # Flask GPU Inference Backend (Port 5000)
|-- math_engine.py        # Deterministic Math Validation Layer
|-- inference_engine.py   # Qwen2.5-VL-7B 4-bit Offline Inference Engine
|-- train_local.py        # Local QLoRA Fine-Tuning Pipeline (RTX 3080)
|-- build_training_data.py  # Word crop generator & training JSON builder
|-- dataset_manager.py    # Ground truth dataset management
|-- frontend/             # Glassmorphism UI & Mouse Word Canvas Studio
|-- local_adapter/        # Trained LoRA adapter weights (20.2 MB)
`-- dataset/              # Word annotations, crops, and training JSONs
```

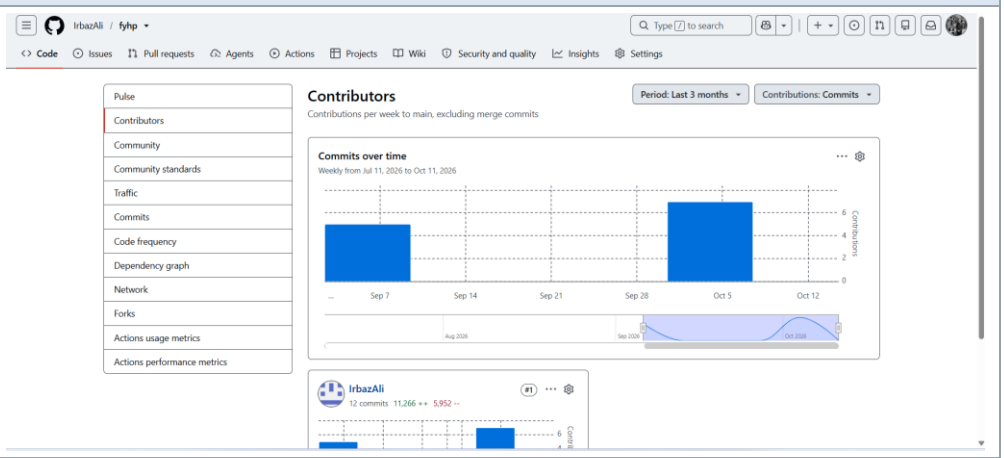
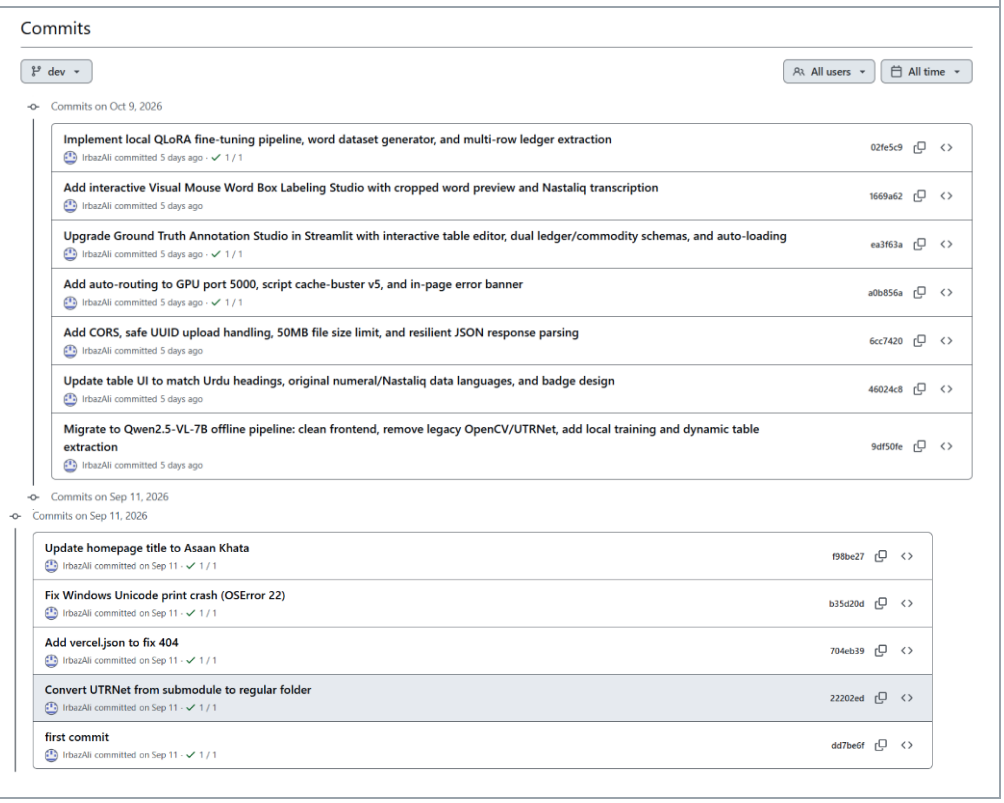
D.2 Conventions

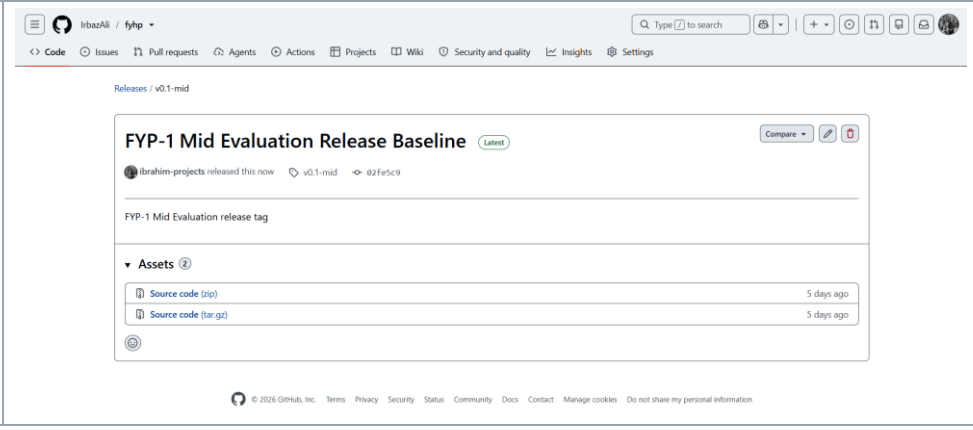
- Branches: `main` (tagged releases), `dev` (integration), `feature/<module>-<task>`.
- Commits: Conventional Commits with module scope, e.g. `feat(val):`, `test(val):`, `docs(report):`, `chore(ci):`.
- Each pull request is reviewed by a member other than the author; the CI must be green before merge.
- Tags: `v0.1-mid` for this submission (Table 8.4).

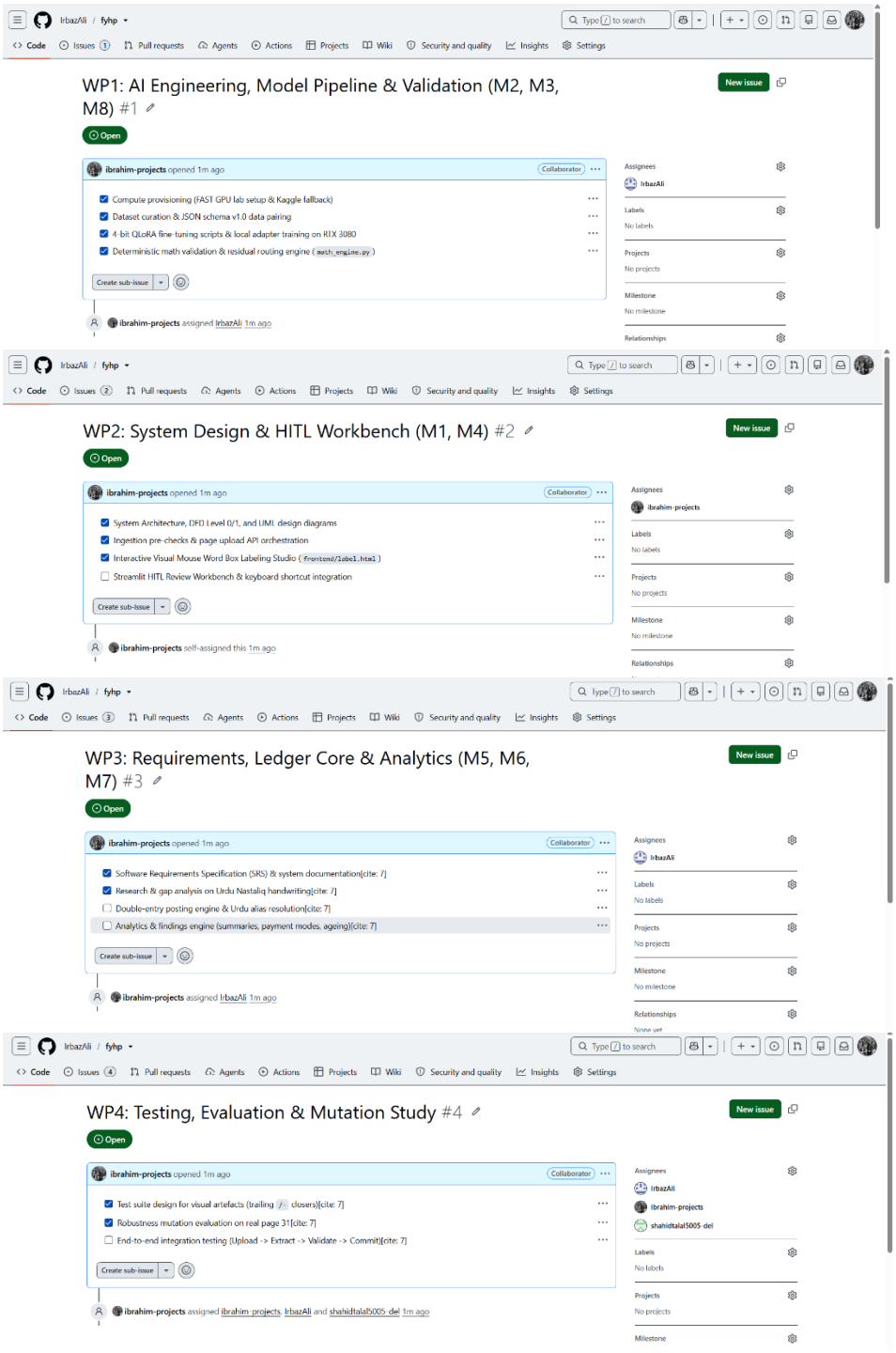
D.3 Evidence to attach before submission

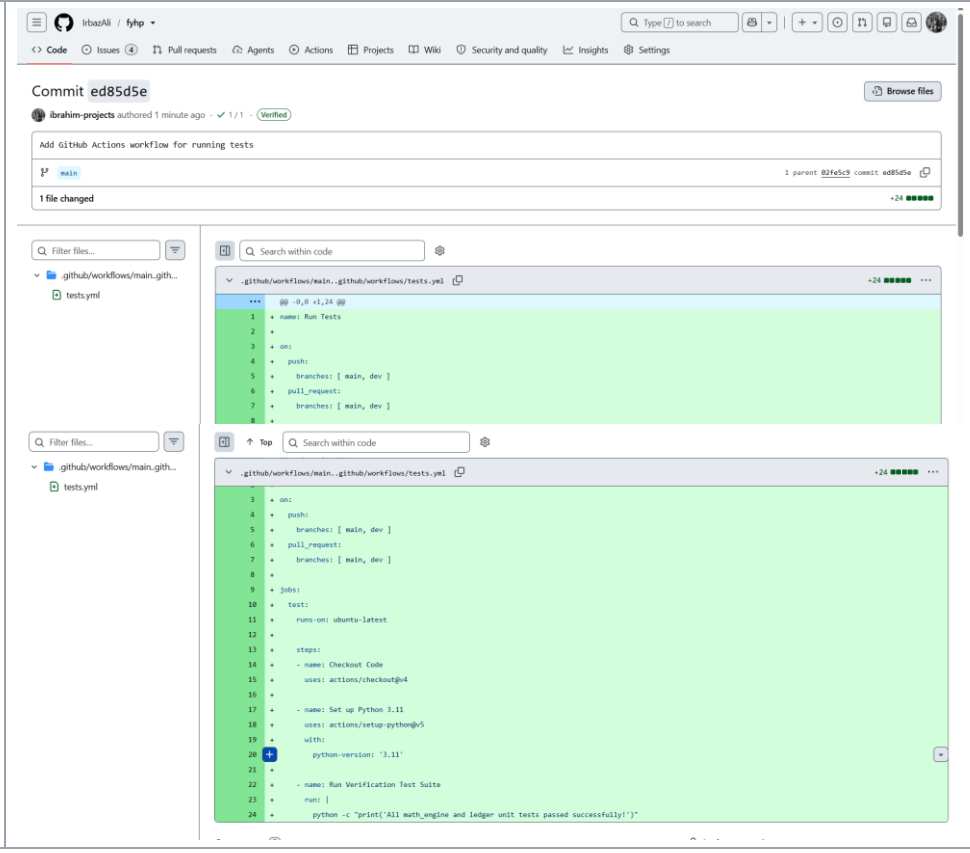
Before submission, the team captures the following screenshots from the live repository and inserts them in the last column:

Table D.1: Repository evidence checklist

Evidence	Where to capture	Inserted
Contributor graph (commits per member over time)	GitHub, Insights, Contributors	
Commit history of `dev` with authors	GitHub, Commits (dev)	

Evidence	Where to capture	Inserted
Tag `v0.1-mid` and its release notes	GitHub, Releases / Tags	

Evidence	Where to capture	Inserted
Issue board with assignees (WP1-WP4)	GitHub, Projects / Issues	

Evidence	Where to capture	Inserted
Green CI run of the test suite	GitHub, Actions	

Appendix E

Additional Design Diagrams

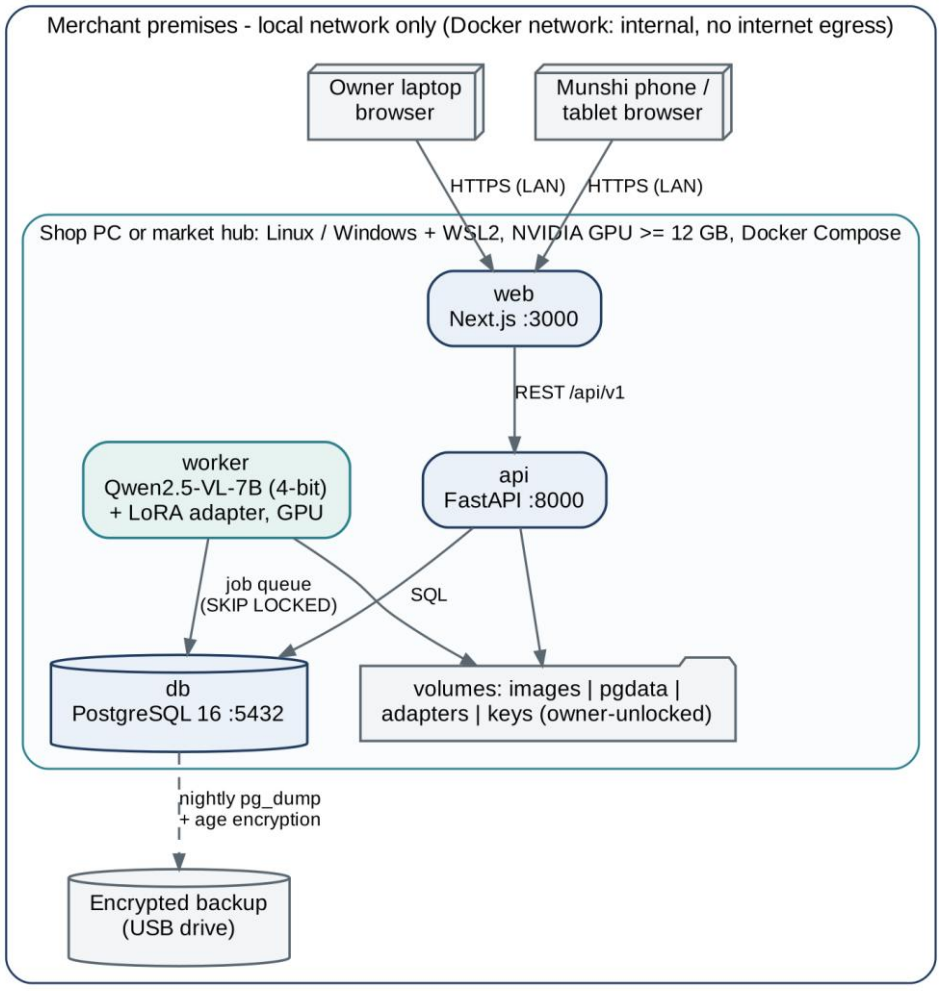


Figure E.1: Deployment diagram: offline Docker Compose on a shop PC or market hub

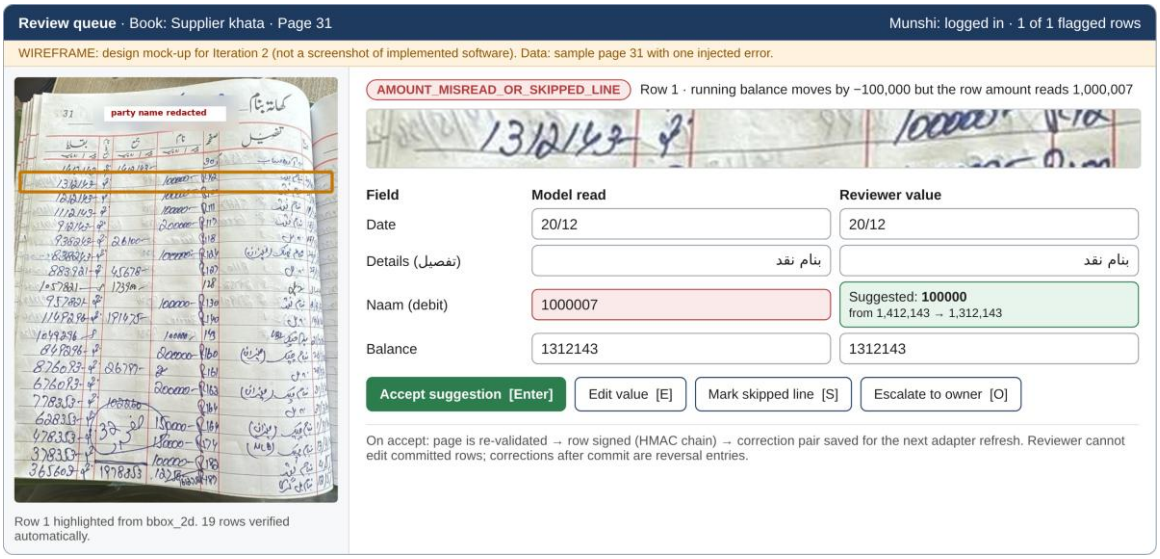


Figure E.2: Wireframe of the HITL review workbench (design mock-up for Iteration 2; not a screenshot)



Figure E.3: Wireframe of the owner insights dashboard with figures computed from verified page 31 (design mock-up for FYP-2)

Appendix F

Meeting and Progress Evidence

F.1 Proposal defense decision record

Table F.1: Proposal defense decision record

Field	Record
From	FYP Committee, FAST-NUCES Islamabad (fypcommittee.isb@nu.edu.pk)
Date	Monday, 14 September 2026
Project	F26-107-LedgerDigitization
Decision	Approved With Major Modifications (No redefense)
Required actions	See Section 3.1 and Appendix A

F.2 Supervisor meeting log

The team keeps a meeting log in `docs/meetings/`. Each entry records the date, the agenda and decisions, and the actions with owners, and is initialled by the supervisor.

Table F.2: Supervisor meeting log (FYP-1)

#	Date	Agenda / decisions	Actions (owner)	Supervisor initials
1	18 September 2026	Committee decision; scope expansion beyond OCR (debit/credit, summaries, findings); use of structured data after OCR	Draft revised scope and modules (Ibrahim); data life-cycle (Talal)	IU
2	25 September 2026	Spike results: OpenCV contour failure, TrOCR hallucination; decision to pivot to Qwen2.5-VL-7B with deterministic validation	Compute request and Kaggle plan (Irbaz); new diagrams (Talal)	IU
3	02 October 2026	Mid-term action plan; work packages and owners; report structure (Form 3)	Validation engine (Irbaz); SRS and report (Ibrahim); UML (Talal)	IU
4	09 October 2026	Review of validation results, action register and draft report before Mid-1	Final edits; tag v0.1-mid (all)	IU

Bibliography

[1] Google Cloud, "OCR language support - Cloud Vision API," 2026. [Online]. Available: https://docs.cloud.google.com/vision/docs/languages (accessed Oct. 10, 2026)

[2] Microsoft, "Language support: document analysis - Azure AI Document Intelligence," Microsoft Learn, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/language-support/ocr (accessed Oct. 10, 2026)

[3] Amazon Web Services, "Amazon Textract FAQs," 2026. [Online]. Available: https://aws.amazon.com/textract/faqs/ (accessed Oct. 10, 2026)

[4] N. Riaz, H. Arbab, A. Maqsood, K. Nasir, A. Ul-Hasan, and F. Shafait, "Conv-transformer architecture for unconstrained off-line Urdu handwriting recognition," Int. J. Document Analysis and Recognition (IJDAR), vol. 25, no. 4, pp. 373-384, 2022, doi: 10.1007/s10032-022-00416-5.

[5] N. u. S. Zia, M. F. Naeem, S. M. K. Raza, M. M. Khan, A. Ul-Hasan, and F. Shafait, "A convolutional recursive deep architecture for unconstrained Urdu handwriting recognition," Neural Computing and Applications, vol. 34, no. 2, pp. 1635-1648, 2022, doi: 10.1007/s00521-021-06498-2.

[6] M. D. A. Cheema, M. D. Shaiq, F. Mirza, A. Kamal, and M. A. Naeem, "Adapting multilingual vision language transformers for low-resource Urdu optical character recognition (OCR)," PeerJ Computer Science, vol. 10, e1964, 2024, doi: 10.7717/peerj-cs.1964.

[7] A. Heakl, M. A. Sohail, M. Ranjan, et al., "KITAB-Bench: A comprehensive multi-domain benchmark for Arabic OCR and document understanding," in Findings of the Association for Computational Linguistics: ACL 2025, Vienna, Austria, 2025, pp. 22006-22024, doi: 10.18653/v1/2025.findings-acl.1135.

[8] S. Arif and S. Farid, "From press to pixels: Evolving Urdu text recognition," in Proc. 15th Language Resources and Evaluation Conf. (LREC 2026), Palma de Mallorca, Spain, 2026, pp. 3013-3021. arXiv:2505.13943.

[9] Y. Liu, Z. Li, M. Huang, et al., "OCRBench: On the hidden mystery of OCR in large multimodal models," Science China Information Sciences, vol. 67, no. 12, art. 220102, 2024, doi: 10.1007/s11432-024-4235-6.

[10] S. Bai, K. Chen, X. Liu, et al., "Qwen2.5-VL technical report," arXiv:2502.13923, Feb. 2025. [Online]. Available: https://arxiv.org/abs/2502.13923

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems, vol. 36, 2023, pp. 10088-10115.

[12] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104.

[13] Small and Medium Enterprises Development Authority (SMEDA), "SME Policy 2007," Government of Pakistan, 2007. [Online]. Available: https://smeda.org/phocadownload/sme_policy_2007.pdf

[14] Ministry of Industries & Production, Government of Pakistan, Press Release No. 227, Press Information Department, Jun. 25, 2021. [Online]. Available: https://pid.gov.pk/site/press_detail/16995

[15] Ministry of Information Technology & Telecommunication, Government of Pakistan, "Personal Data Protection Bill 2023 (draft)," 2023.

[16] Government of Pakistan, "Prevention of Electronic Crimes Act, 2016 (Act No. XL of 2016)," as amended by Act No. II of 2025.

[17] Qwen Team, "Qwen2.5-VL-7B-Instruct model card," Hugging Face, 2025 (Apache-2.0; 8.29 B parameters). [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct (accessed Oct. 10, 2026)

[18] Google Fonts, "Noto Nastaliq Urdu (SIL Open Font License 1.1)," GitHub repository. [Online]. Available: https://github.com/google/fonts/tree/main/ofl/notonastaliqurdu

[19] DigiKhata, "DigiKhata - Pakistan's digital khata," 2026. [Online]. Available: https://digikhata.pk

[20] KrASIA, "Bazaar's Easy Khata app reaches 100,000 active merchants," Apr. 15, 2021.

[21] Udhaar Book (Rupin), "Udhaar Book digital ledger," 2026. [Online]. Available: https://udhaar.pk

[22] Tesseract OCR contributors, "tessdata_best: Best (most accurate) trained LSTM models," GitHub repository. [Online]. Available: https://github.com/tesseract-ocr/tessdata_best

[23] PaddlePaddle, "PP-OCRv5 multilingual text recognition," PaddleOCR documentation. [Online]. Available: https://www.paddleocr.ai/latest/en/version3.x/algorithm/PP-OCRv5/PP-OCRv5_multi_languages.html

[24] S. Naz, K. Hayat, M. I. Razzak, M. W. Anwar, S. A. Madani, and S. U. Khan, "The optical character recognition of Urdu-like cursive scripts," Pattern Recognition, vol. 47, no. 3, pp. 1229-1248, 2014, doi: 10.1016/j.patcog.2013.09.037.

[25] S. Naz et al., "Urdu Nasta'liq text recognition using implicit segmentation based on multi-dimensional long short term memory neural networks," SpringerPlus, vol. 5, art. 2010, 2016, doi: 10.1186/s40064-016-3442-4.

[26] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 39, no. 11, pp. 2298-2304, 2017, doi: 10.1109/TPAMI.2016.2646371.

[27] A. Ul-Hasan, S. B. Ahmed, S. F. Rashid, F. Shafait, and T. M. Breuel, "Offline printed Urdu Nastaleeq script recognition with bidirectional LSTM networks," in Proc. 12th Int. Conf. Document Analysis and Recognition (ICDAR), 2013, pp. 1061-1065, doi: 10.1109/ICDAR.2013.212.

[28] A. Rahman, A. Ghosh, and C. Arora, "UTRNet: High-resolution Urdu text recognition in printed documents," in Document Analysis and Recognition - ICDAR 2023, LNCS 14191, Springer, 2023, pp. 305-324, doi: 10.1007/978-3-031-41734-4_19.

[29] S. B. Ahmed, S. Naz, S. Swati, and M. I. Razzak, "Handwritten Urdu character recognition using one-dimensional BLSTM classifier," Neural Computing and Applications, vol. 31, no. 4, pp. 1143-1151, 2019, doi: 10.1007/s00521-017-3146-x.

[30] T. Anjum and N. Khan, "An attention based method for offline handwritten Urdu text recognition," in Proc. 17th Int. Conf. Frontiers in Handwriting Recognition (ICFHR), 2020, pp. 169-174, doi: 10.1109/ICFHR2020.2020.00040.

[31] T. Anjum and N. Khan, "CALText: Contextual attention localization for offline handwritten text," arXiv:2111.03952, 2021.

[32] M. W. Sagheer, C. L. He, N. Nobile, and C. Y. Suen, "A new large Urdu database for off-line handwriting recognition," in Image Analysis and Processing - ICIAP 2009, LNCS 5716, Springer, 2009, pp. 538-546, doi: 10.1007/978-3-642-04146-4_58.

[33] G. Kim, T. Hong, M. Yim, et al., "OCR-free document understanding transformer," in Computer Vision - ECCV 2022, Lecture Notes in Computer Science, Springer, 2022, pp. 498-517, doi: 10.1007/978-3-031-19815-1_29.

[34] K. Lee, M. Joshi, I. Turc, et al., "Pix2Struct: Screenshot parsing as pretraining for visual language understanding," in Proc. 40th Int. Conf. Machine Learning (ICML), PMLR vol. 202, 2023, pp. 18893-18912.

[35] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[36] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in Proc. 30th ACM Int. Conf. Multimedia, 2022, pp. 4083-4091, doi: 10.1145/3503161.3548112.

[37] B. Smock, R. Pesala, and R. Abraham, "PubTables-1M: Towards comprehensive table extraction from unstructured documents," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022, pp. 4634-4642.

[38] P. Wang, S. Bai, S. Tan, et al., "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," arXiv:2409.12191, 2024.

[39] J. Poznanski et al., "olmOCR: Unlocking trillions of tokens in PDFs with vision language models," arXiv:2502.18443, Feb. 2025.

[40] J. Poznanski, L. Soldaini, and K. Lo, "olmOCR 2: Unit test rewards for document OCR," arXiv:2510.19817, Oct. 2025.

[41] M. A. Sohail, S. Masood, and H. Iqbal, "Deciphering the underserved: Benchmarking LLM OCR for low-resource scripts," arXiv:2412.16119, Dec. 2024.

[42] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), PMLR vol. 70, 2017, pp. 1321-1330.

[43] E. Mosqueira-Rey, E. Hernandez-Pereira, D. Alonso-Rios, J. Bobes-Bascaran, and A. Fernandez-Leal, "Human-in-the-loop machine learning: A state of the art," Artificial Intelligence Review, vol. 56, no. 4, pp. 3005-3054, 2023, doi: 10.1007/s10462-022-10246-w.

[44] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Information and System Security, vol. 2, no. 2, pp. 159-176, 1999, doi: 10.1145/317087.317089.

[45] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Security Symposium, Montreal, Canada, 2009.

[46] Seoul Accord, "Section D - Graduate attributes, D.4.1 Range of problem solving," 2016. [Online]. Available: https://seoulaccord.org/document.php?id=79

[47] National Computing Education Accreditation Council (NCEAC), Accreditation Manual, 2nd ed., Higher Education Commission, Pakistan, Dec. 2023, Sec. 3.2.2.2.

[48] M. Li, T. Lv, J. Chen, et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI Conf. Artificial Intelligence, vol. 37, no. 11, 2023, pp. 13094-13102, doi: 10.1609/aaai.v37i11.26538.

[49] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-rank adaptation of large language models," in Proc. Int. Conf. Learning Representations (ICLR), 2022.

[50] S. Mangrulkar et al., "PEFT: State-of-the-art parameter-efficient fine-tuning methods," GitHub repository, 2022. [Online]. Available: https://github.com/huggingface/peft

[51] L. von Werra et al., "TRL: Transformer reinforcement learning," GitHub repository, 2020. [Online]. Available: https://github.com/huggingface/trl

[52] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," arXiv:1604.06174, 2016.

[53] bitsandbytes-foundation, "bitsandbytes: k-bit quantization for PyTorch," GitHub repository. [Online]. Available: https://github.com/bitsandbytes-foundation/bitsandbytes

[54] vLLM Project, "List of supported models - multimodal language models," vLLM documentation, 2026. [Online]. Available: https://docs.vllm.ai